MULTI-LINE INSURANCE POLICY AND CLAIMS MANAGEMENT SYSTEM

College Name : Anjalai Ammal Mahalingam Engineering College

Dept Name : Information Technology

TEAM ID : SWTID-2026-7407

Lavanya K	820423205045	s89514627@gmail.com
Durgadevi K	820423205028	kdurga2306@gmail.com
Lakshmi Nandhidha L	820423205044	nandhidhalakshmanan05@gmail.com
Akalya R	820423205009	ravichandranabinesh12@gmail.com

Multi-Line Insurance Policy and Claims Management System

User Story

The Multi-Line Insurance Policy and Claims Management System provides a unified Salesforce environment for managing insurance activities across Vehicle, Property, and Life insurance. Insurance personnel can maintain customer and policy information in an organized manner while handling quotations, policy servicing, and claims through defined business processes. The system supports policy-specific information, assists staff in recording claim events, and provides controlled access to insurance records. Automated processes help reduce routine manual work, while dashboards provide operational information to users responsible for monitoring policies and claims. The solution is intended to improve coordination between insurance teams, maintain reliable records, and support a consistent workflow from policy creation through claim settlement.

1. INTRODUCTION

1.1 Project Overview :

The Multi-Line Insurance Policy and Claims Management System is developed on the Salesforce platform to provide an organized environment for handling different insurance operations under a single application. It supports Vehicle, Property, and Life Insurance by maintaining their policy and claim information in a structured manner. The system uses Salesforce configuration and development features to reduce dependency on disconnected records and manual coordination between insurance teams.

The application provides dedicated policy management capabilities that allow authorized users to capture customer information, insurance details, coverage information, and other policy-related data. Different policy categories can be handled according to their individual requirements, while automated processes assist users during policy creation and related activities. Business rules can also be applied to ensure that important information is entered correctly before records are processed.

The claim management functionality allows users to register incidents against existing policies and maintain the information required during claim evaluation. Based on the insurance information associated with a claim, the system can support appropriate assignment and processing activities. Claims requiring additional review can be moved through defined approval stages so that responsible personnel can examine and authorize them before further action is taken.

The solution also provides Salesforce-based interfaces and monitoring capabilities that help users view relevant insurance information from a common location. Access to

records is controlled according to user responsibilities, allowing different insurance teams to work with the information required for their roles.

Overall, the project establishes a structured insurance management environment that connects policy administration and claim processing while reducing manual coordination, improving record organization, supporting controlled workflows, and providing better visibility into ongoing insurance activities.

1.2 Purpose :

The purpose of the Multi-Line Insurance Policy and Claims Management System is to create a unified platform where insurance organizations can manage policy-related and claim-related activities in an efficient and systematic manner. The project focuses on simplifying the movement of information between different stages of the insurance lifecycle and providing users with a consistent method for handling records.

The system is intended to make policy administration more organized by providing appropriate structures for different insurance categories. It helps users collect required information, maintain policy records, and perform routine activities through predefined processes rather than relying entirely on manual coordination. Data validation mechanisms are used to maintain the quality and consistency of important records.

For claims operations, the project provides a structured method for recording incidents, associating them with existing policies, assigning responsibility, and monitoring their progress. Approval mechanisms can be used when a claim requires review from personnel with higher authorization. Salesforce dashboards and related interfaces further assist users in understanding the current workload and status of insurance activities.

Overall, the project aims to establish a reliable and scalable Salesforce environment that simplifies insurance administration, organizes claim operations, improves information consistency, supports user-specific activities, and provides a common platform for managing multiple insurance services.

2.1 Problem Statement (Customer Problem Statement Template)

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Policy Administrator	maintain customer policy records	information is difficult to organize	customer details are maintained across different records	confused and concerned

PS-2	Insurance Agent	collect required information for a new policy	I need to enter details from multiple sources	each insurance product has its own information requirements	tired and time-consuming
PS-3	Insurance Agent	prepare suitable coverage for customers	comparing customer requirements is difficult	policy information is not presented in a consistent format	uncertain and less confident
PS-4	Policy Manager	monitor active policies	policy changes are difficult to track	updates can occur at different stages of the policy lifecycle	concerned about missing important changes
PS-5	Claims Adjuster	review reported incidents	supporting claim details are scattered	information may be received through different channels	frustrated and delayed
PS-6	Claims Coordinator	keep track of pending claims	claim progress is not clearly visible	different claims can be at different processing stages	overwhelmed and concerned
PS-7	Senior Claims Reviewer	review claims requiring additional attention	important cases are difficult to identify quickly	claims have different financial values and processing requirements	pressured and cautious
PS-8	Insurance Operations Manager	maintain consistent insurance records	incorrect or incomplete information can enter the system	users handle different types of insurance transactions	concerned about record reliability
PS-9	Customer	receive updates about my	I do not always know	communication during	uncertain and dissatisfied

		insurance request	the current progress	processing may be inconsistent	
PS-10	Insurance Management	understand the overall workload of policy and claim teams	operational information is difficult to consolidate	activities are handled by different users and processes	concerned about productivity

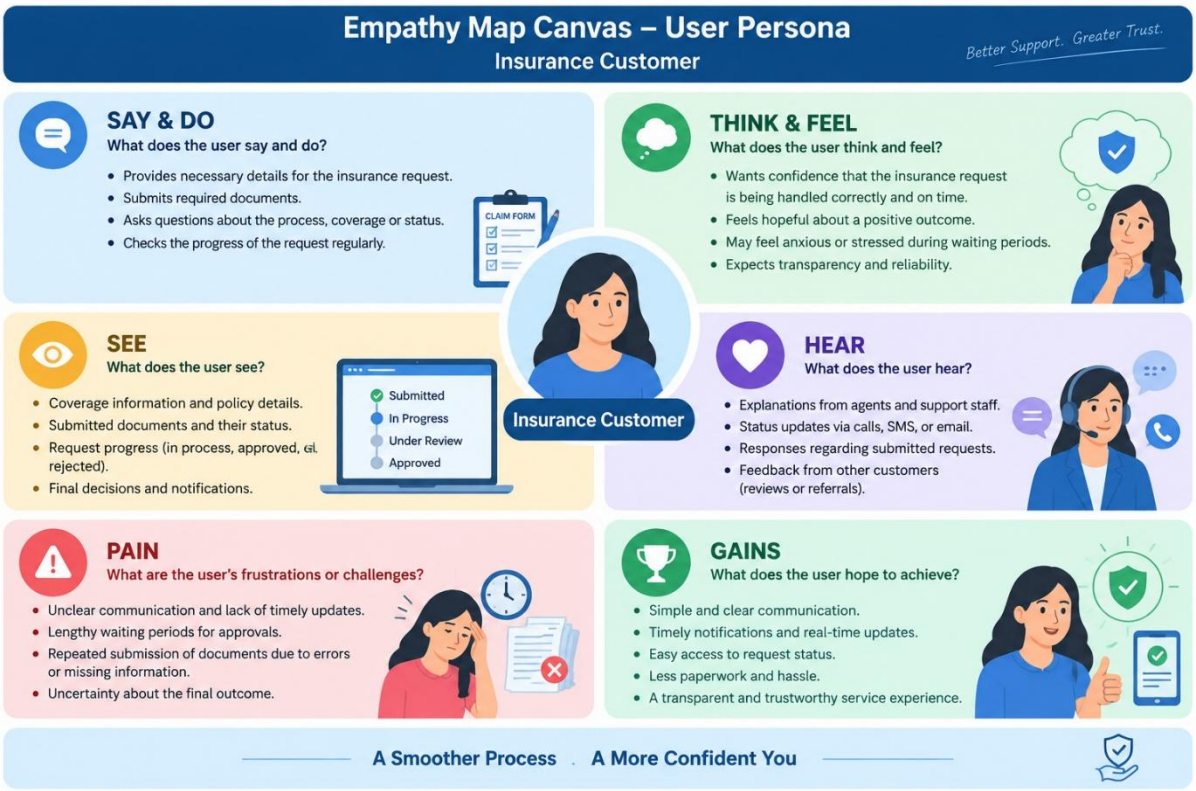


2.2 Empathy Map Canvas

User Persona: Insurance Customer

- **Think & Feel:** Wants confidence that the insurance request is being handled correctly and on time.
- **Hear:** Explanations from agents, status updates, and responses regarding submitted requests.
- **See:** Coverage information, submitted documents, request progress, and final decisions.
- **Say & Do:** Provides necessary details, submits documents, asks questions, and checks the progress of the request.

- **Pain:** Unclear communication, lengthy waiting periods, repeated document submission, and uncertainty about outcomes.
- **Gain:** Simple communication, timely notifications, easy access to request status, and a transparent service experience.



2.3 Brainstorming & Idea Prioritization

Ideas Considered

● **Paper-Based Documentation** – Customer, policy, and incident details recorded in physical files.

● **Basic Spreadsheet System** – Insurance records maintained in separate spreadsheets for individual activities.

● **Independent Web Application** – A dedicated application developed specifically for handling insurance operations.

● **Salesforce-Based Solution** – A configurable platform for managing insurance information, workflows, user activities, and analytics.

Final Selection

Salesforce-Based Solution selected due to:

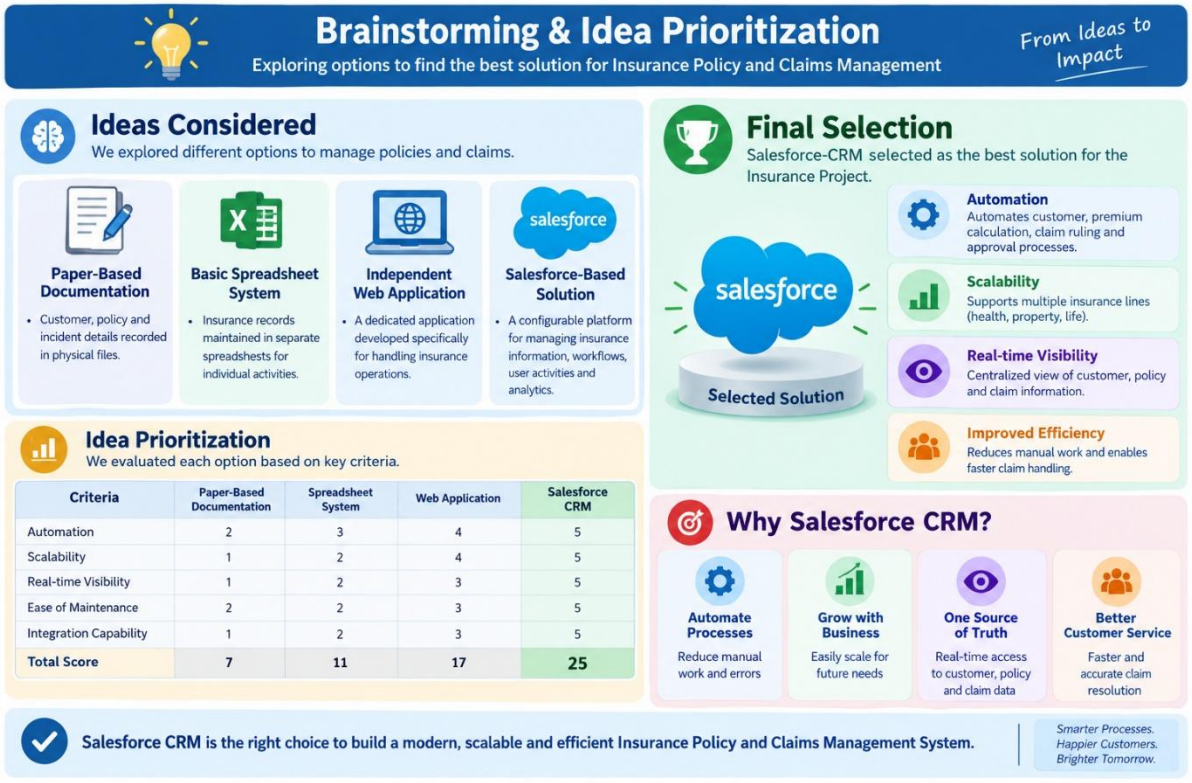
● **Centralized Data Management** – Keeps customer, policy, and claim records within a connected environment.

● **Process Flexibility** – Allows different insurance categories to follow their respective business requirements.

● **Controlled Access** – Provides user-based access so employees can work according to their responsibilities.

● **Workflow Support** – Enables structured movement of records through various stages of policy and claim operations.

● **Better Monitoring** – Helps managers and operational teams track ongoing activities through reports and dashboards.



3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stages:

● **Customer Registration** – Customer details are collected and recorded.

● **Insurance Requirement Identification** – The customer's coverage needs are understood.

● **Product Exploration** – Available insurance options are examined according to the customer's requirement.

● **Application Submission** – Required personal and insurance-related information is provided.

● **Eligibility Verification** – Submitted information is checked against the applicable requirements.

● **Quotation Preparation** – An estimated insurance cost is prepared from the available details.

● **Policy Activation** – The selected insurance plan is confirmed and made active.

● **Incident Reporting** – The customer reports an event covered under the policy.

● **Claim Assessment** – Submitted incident details and supporting information are examined.

● **Decision Processing** – The claim is evaluated and an appropriate decision is recorded.

● **Settlement Completion** – Approved claim activities are completed according to the applicable process.

● **Service Follow-Up** – The customer receives the relevant outcome and service information



3.2 Solution Requirements

Functional Requirements

FR No	Functional Requirement
FR-01	The application shall enable authorized users to register and maintain customer insurance profiles.

FR-02	The application shall provide dedicated policy configurations for Vehicle, Property, and Life insurance.
FR-03	The application shall capture coverage-related information appropriate to the selected insurance category.
FR-04	The application shall guide agents through the required information during policy application.
FR-05	The application shall generate an insurance quotation from the information entered by the user.
FR-06	The application shall store quotation information for further policy processing.
FR-07	The application shall check mandatory policy information before allowing further processing.
FR-08	The application shall enable authorized users to register incidents against active insurance policies.
FR-09	The application shall capture essential claim information including incident details, claimant information, and requested amount.
FR-10	The application shall associate each claim with its corresponding customer and policy.
FR-11	The application shall assign claim cases to the responsible processing team.
FR-12	The application shall identify cases that require additional managerial examination.
FR-13	The application shall support review decisions with appropriate status information and remarks.
FR-14	The application shall provide claims personnel with a consolidated view of their assigned cases.
FR-15	The application shall display relevant claim information such as case reference, insurance category, claimant, amount, and current stage.
FR-16	The application shall allow users to narrow displayed claim information according to the selected insurance category or processing condition.
FR-17	The application shall update related records automatically when defined business events occur.

FR-18	The application shall restrict access to records according to configured organizational responsibilities.
FR-19	The application shall provide appropriate privileges to agents, claims personnel, and managerial users.
FR-20	The application shall provide reports and dashboards for monitoring insurance operations and business activity.

Non-Functional Requirements

ID	Non-Functional Requirement	Description
NFR-01	**Ease of Use**	The application should provide a straightforward interface that allows users to complete common insurance activities without unnecessary complexity.
NFR-02	**Responsiveness**	Common operations such as opening records, submitting information, and viewing claim details should respond within an acceptable period.
NFR-03	**Capacity**	The solution should accommodate increasing numbers of customers, policies, and claims without requiring a complete redesign.
NFR-04	**Accuracy**	Entered information should remain consistent across related customer, policy, and claim records.
NFR-05	**Confidentiality**	Sensitive insurance information should be accessible only to users with the required authorization.
NFR-06	**Maintainability**	The application should be organized into manageable Salesforce configuration and development components for easier updates.
NFR-07	**Dependability**	Automated activities should operate consistently and produce predictable results under normal operating conditions.
NFR-08	**Adaptability**	The solution should permit changes to insurance products and business requirements without extensive modification of the complete application.

NFR-09	**Information Availability**	Authorized users should be able to obtain current policy and claim information from a centralized environment.
NFR-10	**Learnability**	The application should be sufficiently intuitive that insurance staff can understand the main functions with limited training.

3.3 Data Flow Diagram

Flow:

Customer → Insurance Agent → Policy Application → Verification → Policy Record → Quotation → Active Policy → Incident Report → Claim Record → Claim Assessment → Claim Decision → Policy/Claim Update → Dashboard & Reports

Main Data Flow

Step1–CustomerInformation
→ The customer provides personal details and insurance requirements to the Insurance Agent.

Step 2 – Insurance Application
→ The agent enters the required customer and coverage information into the Salesforce application.

Step 3 – Information Verification
→ The system checks the submitted information and identifies missing or invalid mandatory details.

Step 4 – Policy Record Creation
→ Verified information is stored as a policy record under the relevant insurance category.

Step 5 – Quotation Preparation
→ The entered policy details are processed to determine the applicable quotation information.

Step 6 – Policy Activation
→ After the required confirmation, the policy information is updated and made available for future servicing.

Step 7 – Incident Registration
→ When an insured event occurs, the responsible user records the incident against the corresponding policy.

Step 8 – Claim Assessment
→ Claims personnel examine the reported incident, supporting information, and associated policy details.

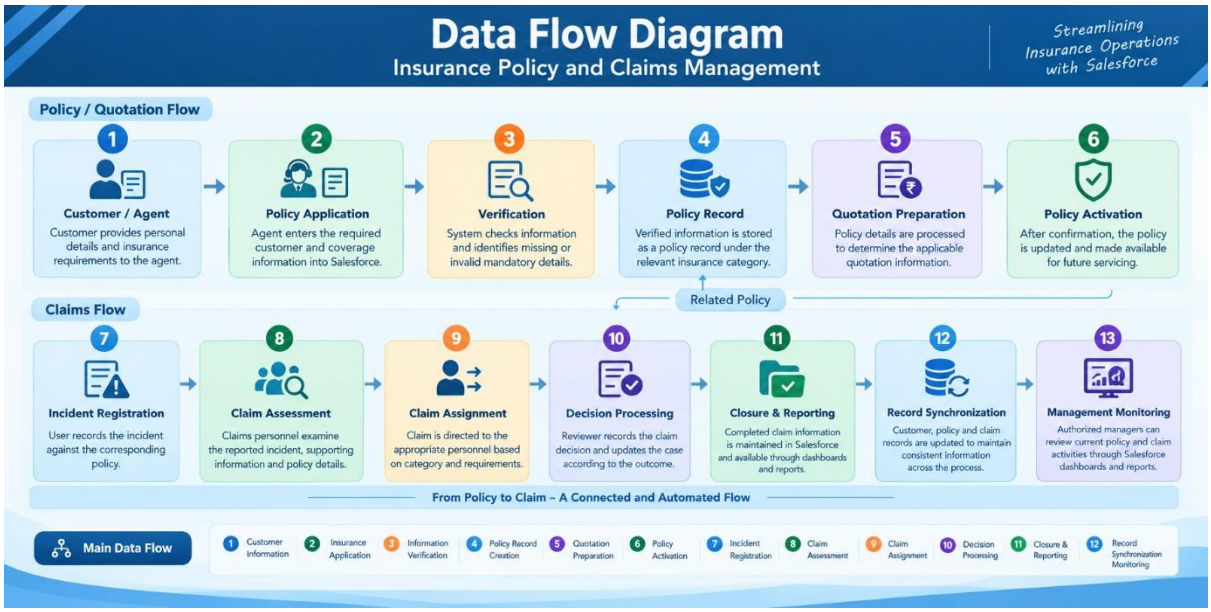
Step 9 – Claim Assignment
→ The claim is directed to the appropriate personnel for further examination based on its category and processing requirements.

Step 10 – Decision Processing
→ The responsible reviewer records the claim decision and updates the case according to the outcome.

Step 11 – Closure & Reporting
→ Completed claim information is maintained in Salesforce and made available through dashboards and reports for authorized users.

Step 12 – Record Synchronization
→ Related customer, policy, and claim records are updated to maintain consistent information throughout the insurance process.

Step 13 – Management Monitoring
→ Authorized managers can review current policy and claim activities through Salesforce dashboards and reports to monitor ongoing operations.



3.4 Technology Stack

Layer	Technology Used
User Interface	Salesforce Lightning Experience, Lightning Web Components (LWC)

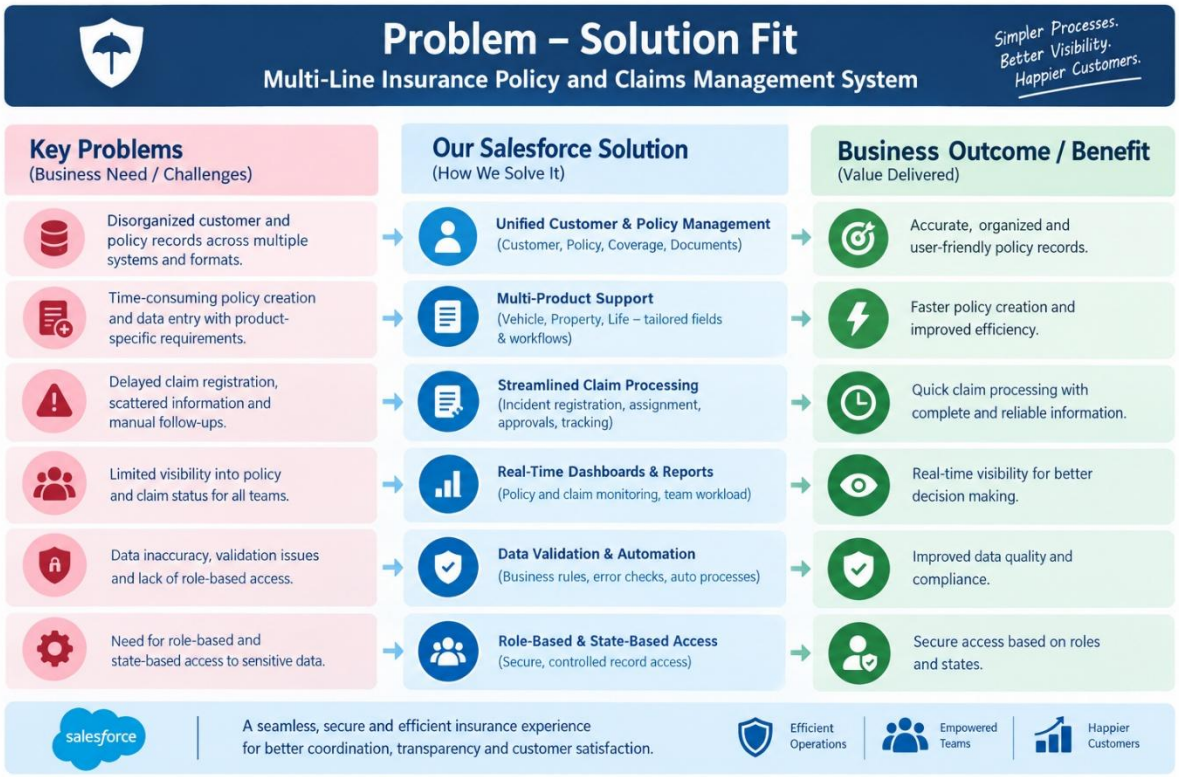
Application Logic	Apex Classes, Apex Controllers
Data Storage	Salesforce Custom Objects, Contact, User
Policy Management	Policy Custom Object, Record Types, Custom Fields, Field Sets
Claim Management	Claim Custom Object, Lookup Relationships, Claim Status Fields
Automation	Screen Flows, Record-Triggered Flows, Approval Processes
Validation	Validation Rules, Required Fields, Field-Level Validation
Premium Processing	Apex Premium Calculation Logic
Claim Routing	Salesforce Flow, Queue Assignment, Approval Workflow
Security	Profiles, Permission Sets, Role Hierarchy, Sharing Rules
Reporting	Salesforce Reports, Dashboards, List Views
Testing	Apex Test Classes, Flow Testing, Functional Testing

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

The Salesforce-based Multi-Line Insurance Policy and Claims Management System is designed to address the operational requirements of managing different insurance products within a single platform. It provides a structured environment for maintaining customer, policy, quotation, and claim information while supporting the different processing requirements of Vehicle, Property, and Life Insurance.

Salesforce features such as **Custom Objects, Record Types, Screen Flows, Apex, Record-Triggered Flows, Validation Rules, Approval Processes, and Lightning Web Components** work together to support policy creation, information validation, premium processing, claim registration, claim assignment, and claim monitoring. The platform also provides controlled access to insurance information and enables users to monitor activities through reports and dashboards.



4.2 Proposed Solution

Functional Requirements

Parameter	Description
Problem	Separate handling of customer information, insurance policies, quotations, and claims can make insurance operations difficult to coordinate.
Solution	A centralized Salesforce application for managing Vehicle, Property, and Life Insurance policy and claim activities.
Innovation	Guided policy processing, automated premium calculation, claim assignment, validation, and approval-based claim handling.
Impact	Reduces manual processing, improves information accuracy, simplifies claim tracking, and supports faster operational activities.
Business Model	Provides a unified platform for managing different insurance lines while maintaining their individual policy requirements.
Scalability	The configurable Salesforce data model can accommodate additional insurance categories, fields, workflows, and business processes.

Visibility	Centralized Customer, Policy, Quotation, and Claim records with Salesforce Reports, Dashboards, and Claims Dashboard components.
Security	User access is controlled through Profiles, Permission Sets, Roles, and Sharing Rules according to organizational responsibilities.

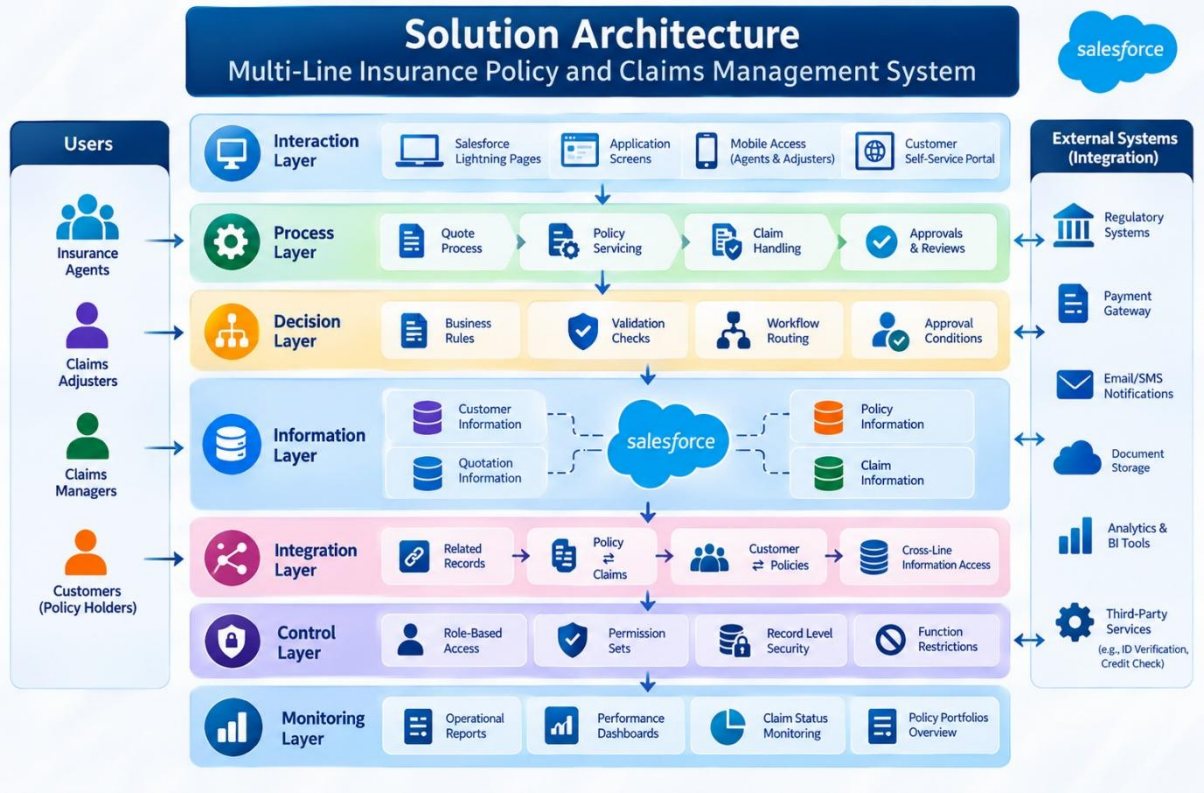
The proposed solution provides a **centralized Salesforce environment** for managing the insurance lifecycle from customer information and policy creation through quotation, policy servicing, claim processing, approval, and closure. The combination of Salesforce configuration, automation, Apex processing, and reporting provides a structured platform for managing multiple insurance products while maintaining controlled access to business information.

4.3 Solution Architecture

The proposed system is organized around the **complete movement of insurance information** rather than repeating the functional features described in Section 4.2. The architecture connects customer interaction, policy administration, claim handling, data management, and management monitoring through Salesforce.

Key Architecture Components

Architecture Component	Project Implementation
Interaction Layer	Agents and claims personnel access insurance records through Salesforce Lightning pages and application screens.
Process Layer	Guided processes control the sequence of activities involved in policy and claim transactions.
Decision Layer	Business conditions determine the next processing stage for applications and claims.
Information Layer	Salesforce stores related customer, policy, quotation, and claim information in connected records.
Integration Layer	Relationships between insurance records allow information to be accessed across related business activities.
Control Layer	Salesforce access controls protect records and restrict functions according to user responsibilities.
Monitoring Layer	Operational information is presented through dashboards and reports for supervisory review.



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

● **Agile Framework** – The project is developed incrementally so that insurance policy and claim features can be implemented and reviewed in manageable stages.

● **Feature-Based Planning** – The complete system is divided into major areas such as policy configuration, quotation, premium processing, claim management, approval, dashboard, and security.

● **User Story Definition** – Each functional requirement is converted into a user story describing the expected functionality from the perspective of an insurance user.

● **Effort Estimation** – Story points are assigned based on the complexity and development effort required for each feature.

● **Sprint Execution** – Related activities are grouped into short sprints, allowing the development team to complete and verify specific portions of the application.

● **Progress Tracking** – Completed story points are compared with planned work to monitor sprint progress and overall project completion.

5.2 Product Backlog, Sprint Schedule, and Estimation

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint -1	Data Modeling	US-01	As an admin, I want to create Policy and Claim objects so that insurance policy and claim information can be stored and managed systematically.	5	High	Member 1
Sprint -1	Data Modeling	US-02	As an admin, I want to configure Vehicle, Property, and Life Record Types so that different insurance products can be managed using a unified Policy structure.	3	High	Member 2
Sprint -1	Data Modeling	US-03	As an admin, I want to configure policy Field Sets and product-specific fields so that diverse insurance information can be captured efficiently.	5	High	Member 3
Sprint -1	Quoting & Automation	US-04	As an insurance agent, I want a guided Auto Quoting Screen Flow so that I can capture policy-specific information through a standardized quoting process.	8	High	Member 4
Sprint -2	Validation	US-05	As an insurance agent, I want VIN validation during vehicle quoting so that incorrect	3	High	Member 1

			vehicle information can be prevented.			
Sprint -2	Apex	US-06	As an insurance system, I want PremiumCalculator Apex to calculate premiums so that premium values are generated accurately.	8	High	Member 2
Sprint -2	Automation	US-07	As an insurance agent, I want the premium calculation integrated with the quoting Flow so that the calculated premium is available during the quote process.	5	High	Member 3
Sprint -2	Claims Automation	US-08	As a claims manager, I want claims to be routed based on policy type so that claims are assigned to the appropriate support queue.	8	High	Member 4
Sprint -3	Apex	US-09	As a claims adjuster, I want a ClaimsAdjusterControl ler to retrieve my assigned claims so that I can efficiently manage my claim workload.	8	High	Member 1
Sprint -3	LWC Dashboard	US-10	As a claims adjuster, I want a claims dashboard and reusable claim tile so that I can view and	8	High	Member 2

			manage assigned claims efficiently.			
Sprint -4	Approval	US-11	As a claims manager, I want high-value claims to go through an approval process and approver Flow so that high-value claims receive appropriate review.	8	High	Member 3
Sprint -4	Security & Testing	US-12	As a system administrator, I want to implement tests, sharing rules, and Permission Sets so that the solution is validated and users receive appropriate access.	8	High	Member 4

5.3 Project Tracker, Velocity & Burndown Chart (4 Marks)

The project tracker records the planned duration, completed story points, and release date for each sprint. Since the project was completed from **15 September 2026 to 3 October 2026**, the sprint schedule is arranged within this period.

Sprint	**Total Story Points**	**Duration**	**Sprint Start Date**	**Sprint End Date (Planned)**	**Story Points Completed (as on Planned End Date)**	**Sprint Release Date (Actual)**
Sprint-1	20	5 Days	15 Sep 2026	19 Sep 2026	20	19 Sep 2026
Sprint-2	20	5 Days	20 Sep 2026	24 Sep 2026	20	24 Sep 2026
Sprint-3	20	5 Days	25 Sep 2026	29 Sep 2026	20	29 Sep 2026

Sprint-4	20	4 Days	30 Sep 2026	03 Oct 2026	20	03 Oct 2026

Sprint-1

Sprint-1 focuses on establishing the basic Salesforce structure required for the insurance application. Policy and Claim objects are created, followed by the configuration of Vehicle, Property, and Life Record Types and their product-specific fields. The Auto Quoting Screen Flow is also developed as part of the initial application functionality.

Sprint-2

Sprint-2 concentrates on policy validation, premium processing, and claim automation. VIN validation is implemented to verify vehicle information, while the PremiumCalculator Apex class performs premium calculations. The calculated premium is connected with the quotation Flow, and claims are routed according to their associated policy type.

Sprint-3

Sprint-3 focuses on claim handling for adjusters. The ClaimsAdjusterController Apex class is developed to retrieve assigned claims. The Lightning Web Component dashboard and reusable claim tile are then implemented to provide adjusters with an effective interface for viewing their claim information.

Sprint-4

Sprint-4 completes the advanced claim and administrative functionality. High-value claim approval and the approver Flow are configured, followed by sharing rules and Permission Sets for controlled access. Apex tests and functional validation are performed to verify the completed Salesforce solution.

Velocity Calculation

Velocity represents the number of story points completed during a sprint. In this project, the planned sprint workload is **20 story points per sprint**. Across four sprints, the project completes a total of **80 story points**.

Burndown Chart

The burndown chart is used to monitor the remaining work throughout the project. At the beginning of each sprint, the planned story points represent the work to be completed. As the user stories are finished, the remaining points decrease. The chart therefore provides a visual representation of progress from the initial data-modeling activities through the final security and testing activities.

6. PROJECT DEVELOPMENT PHASE

What You'll Learn

1. Salesforce Data Modeling
2. Custom Objects and Relationships
3. Custom Fields and Record Types
4. Salesforce Lightning App Configuration
5. Screen Flow Development
6. Record-Triggered Flow Automation
7. Apex Classes and Business Logic

Milestones

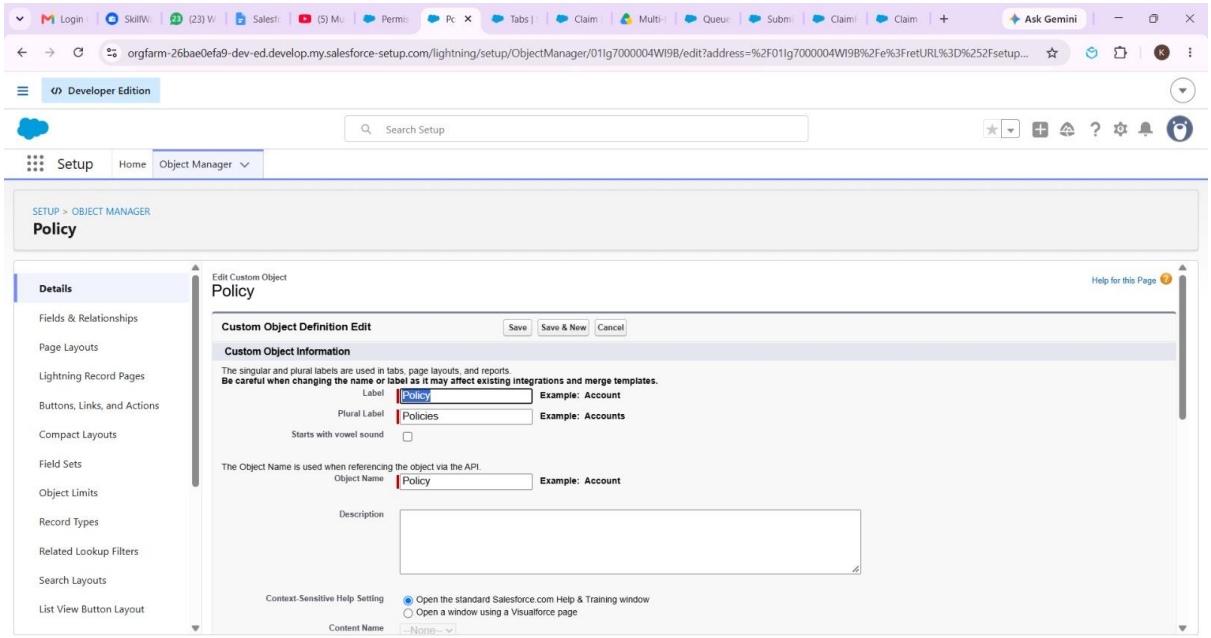
Milestone 1: Insurance Data Model & Policy Setup

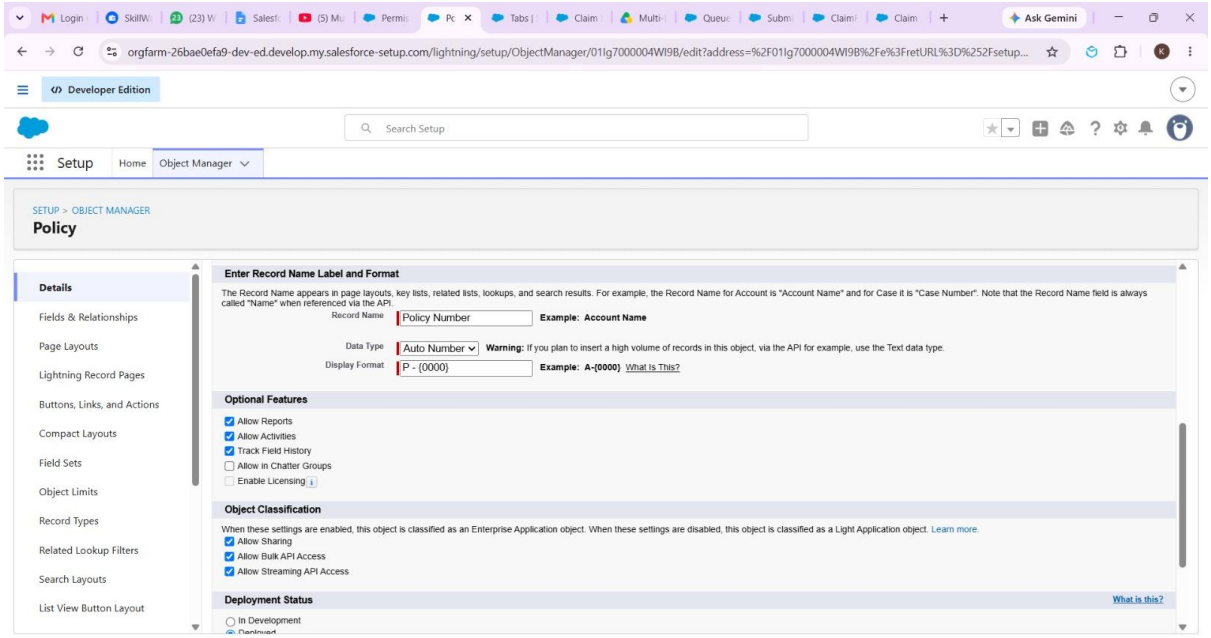
User Story 1: Create the Policy Object

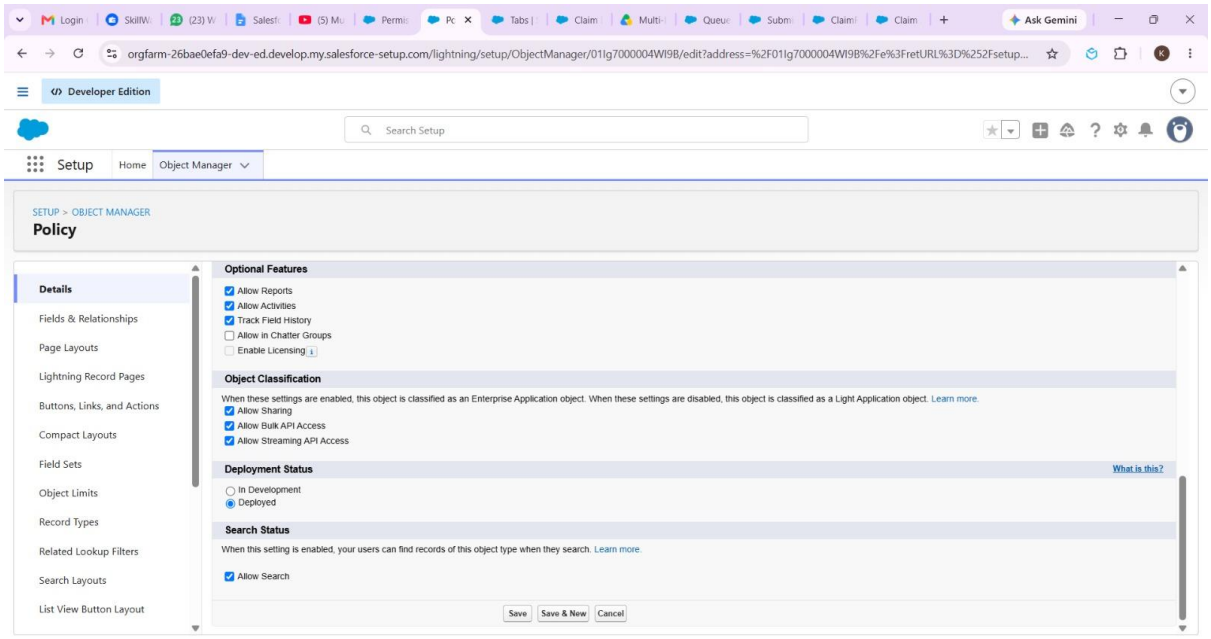
Steps:

- Open **Salesforce Setup**.
- Navigate to **Object Manager**.
- Select **Create → Custom Object**.
- Enter the following details:
    - **Label:** Policy
    - **Plural Label:** Policies
    - **Object Name:** Automatically populated
    - **Record Name:** Policy Number
    - **Data Type:** Auto Number
    - **Display Format:** POL-{0000}
    - **Starting Number:** 1
- Under **Optional Features**, enable:
    - Allow Reports
    - Allow Activities
    - Track Field History

- Under **Deployment Status**, select:
    - **Deployed**
- Under **Search Status**, enable:
    - **Allow Search**
- Click **Save**.





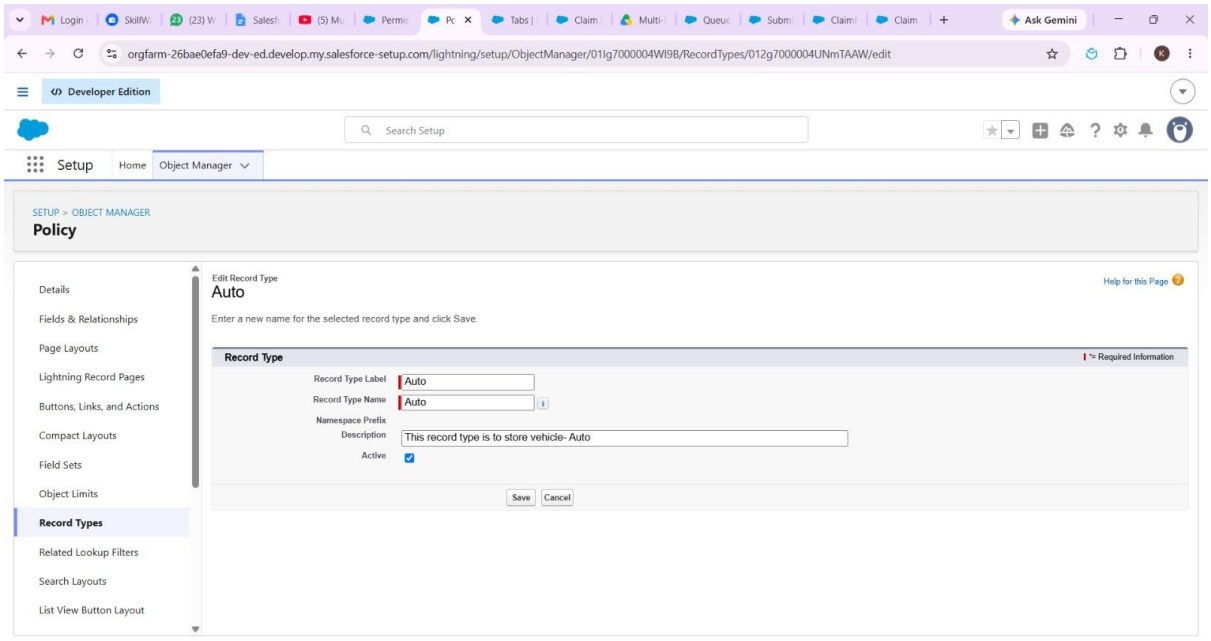


User Story 2: Define Record Types on the Policy Object

Steps:

- Open **Setup** in Salesforce.
- Go to **Object Manager**.
- Search for and select the **Policy** object (Policy__c).
- From the left-side menu, select **Record Types**.
- Click **New**.
- Configure the record type with the following details:
    - **Existing Record Type:** Master
    - **Record Type Label:** Auto
    - **Record Type Name:** Automatically populated
    - **Description:** Record type for managing automobile insurance policies.
    - **Active:** Checked
    - **Make Available:** Select the required user profiles.
- Click **Next**.
- On the **Assign Page Layouts** screen, retain the default page-layout assignments.
- Click **Save**.

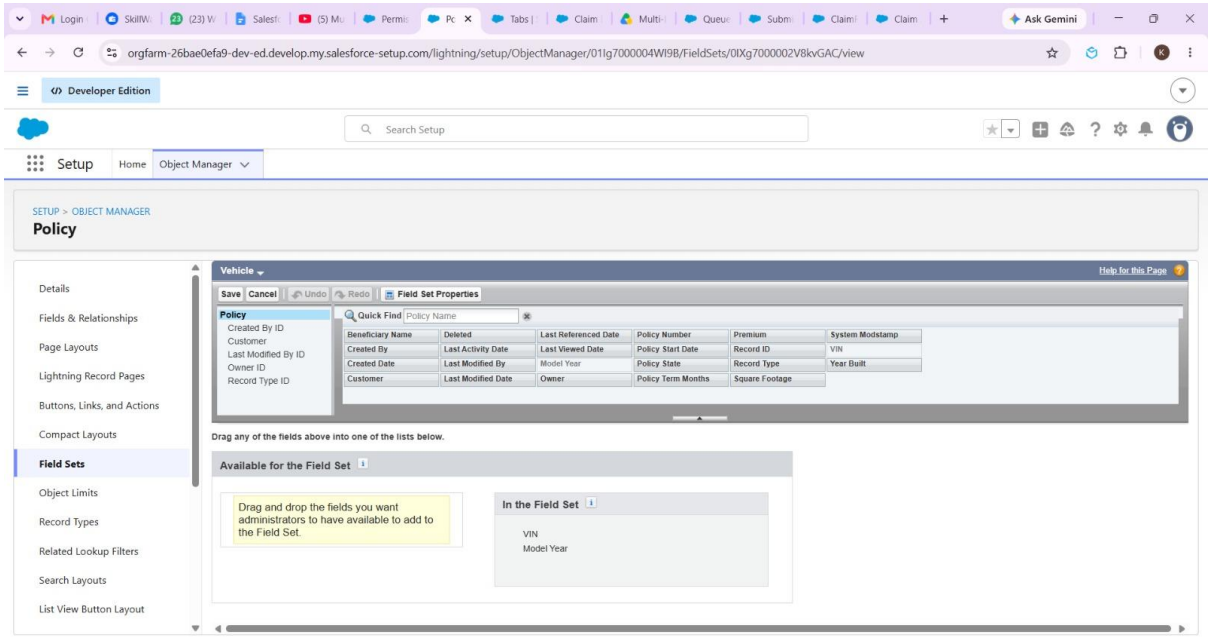
This creates the **Auto** record type under the Policy object, allowing automobile insurance policies to be handled separately from other policy categories.



3. Create Field Sets for Policy Specific Data:

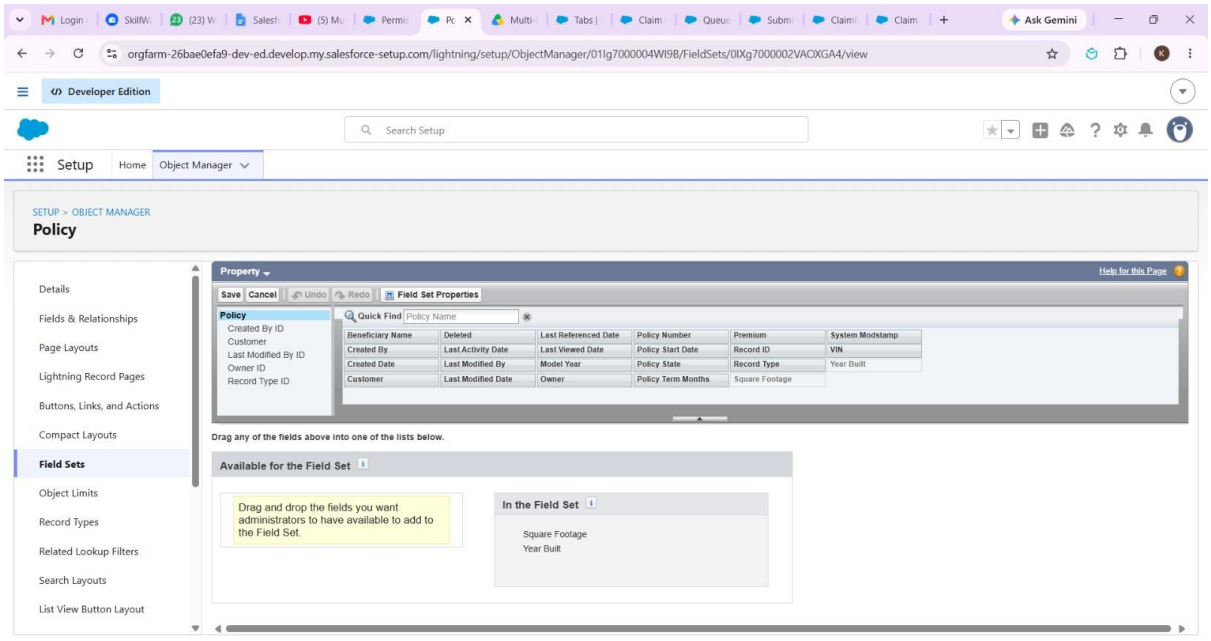
- Open **Setup → Object Manager → Policy (Policy__c)**.
- Select **Field Sets** from the left navigation menu.
- Click **New Field Set**.
- Enter the following details:
    - **Field Set Label:** Vehicle
    - **Field Set Name:** Automatically generated
    - **Purpose:** Vehicle-specific policy information
- Click **Save**.
- Open the **Vehicle** field set from the Field Sets list.
- Add the required **VIN** and **Model Year** fields from the available fields section.
- Arrange the fields in the required order.

- Click **Save.**



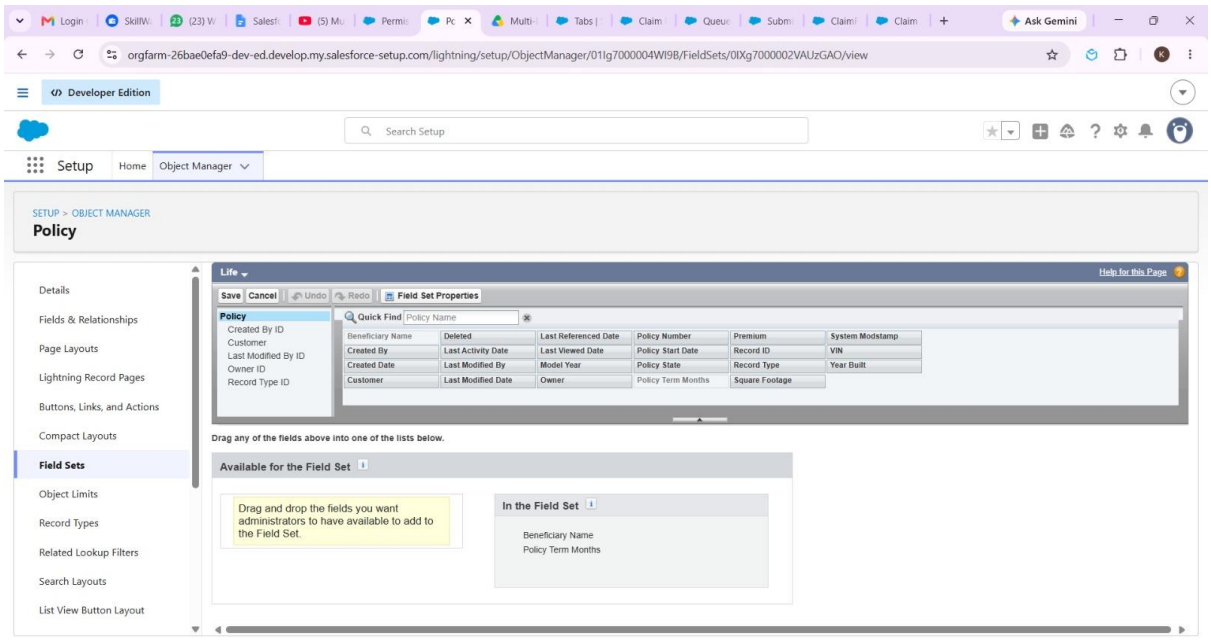
3.1 Property Field Set

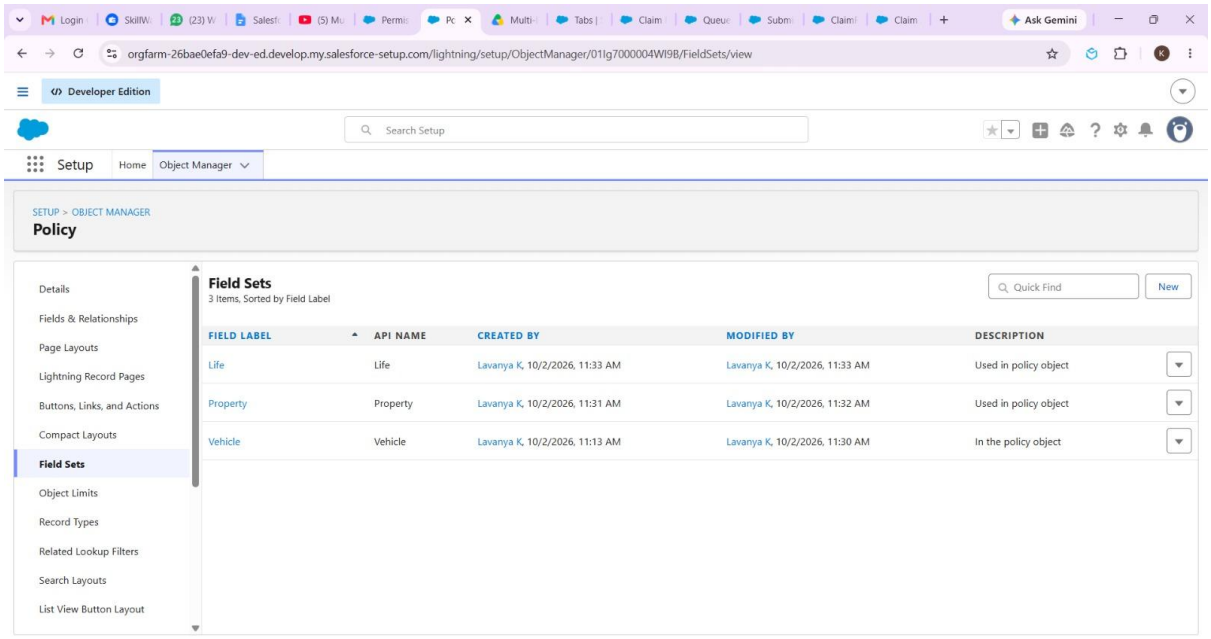
- **Field Set Label:** Property
- **Field Set Name:** Automatically generated
- **Purpose:** Property-specific policy information
- **Add:** Square Footage, Year Built
- Click **Save.**



3.2 Life Field Set

- **Field Set Label**: Life
- **Field Set Name:** Automatically generated
- **Purpose:** Life insurance policy information
- **Add:** Beneficiary Name, Policy Term Months
- Click **Save.**





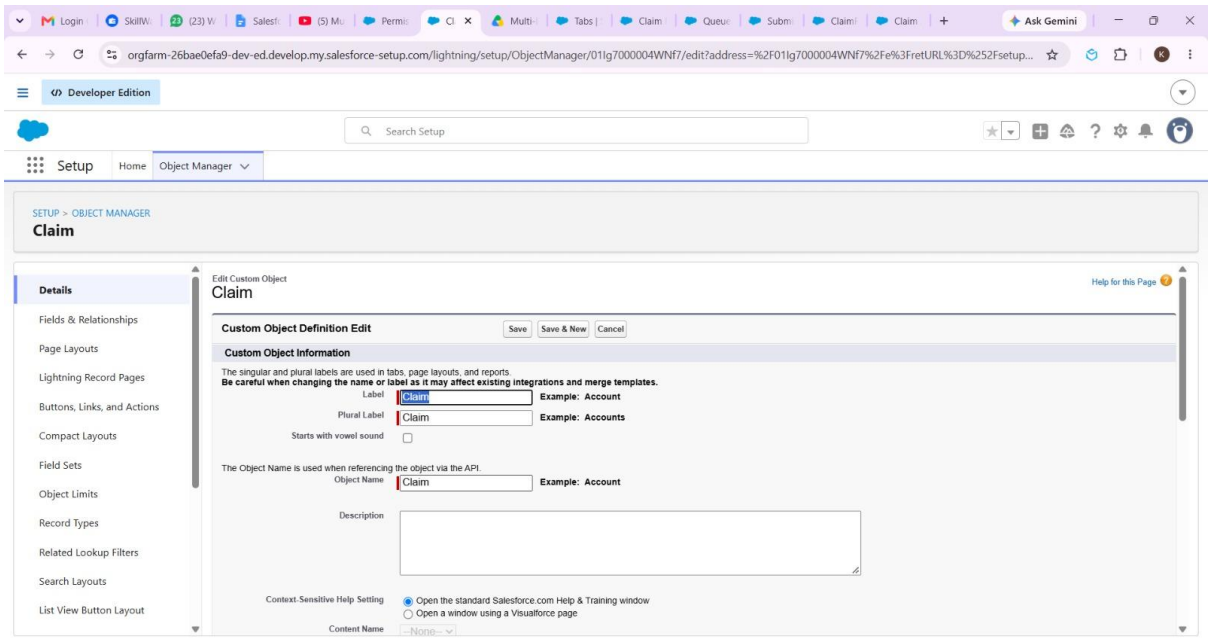
4. Create Fields in the Policy Object

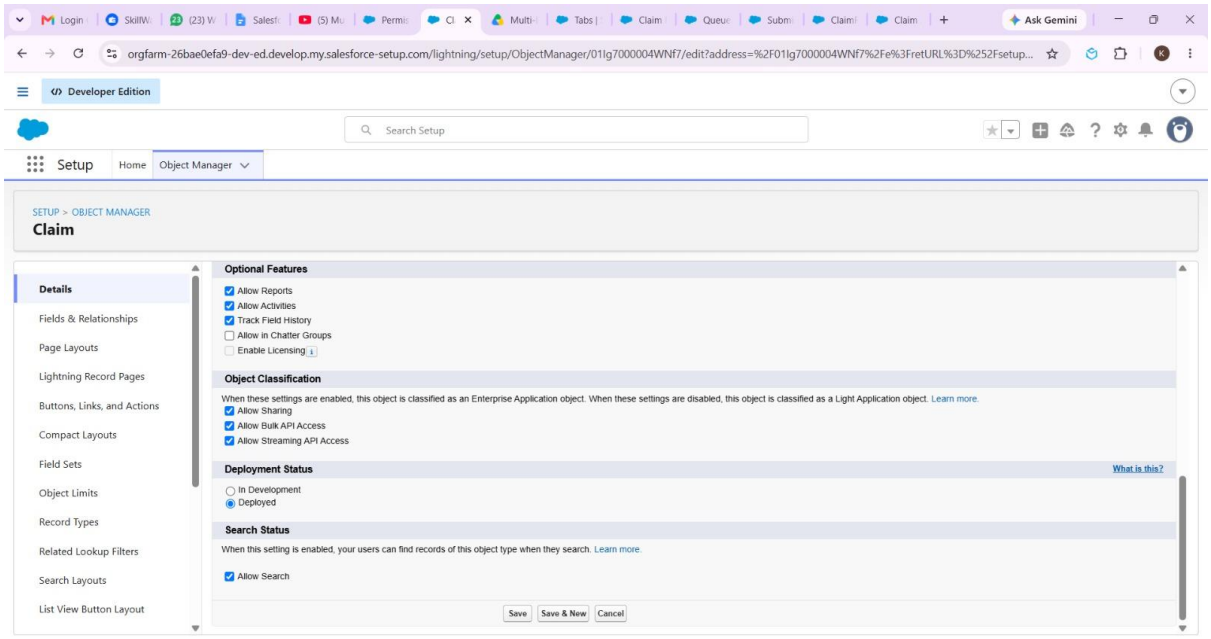
S.No	Name of the Field	Data Type	Required
1	Beneficiary Name	Text (50)	Yes
2	Customer	Lookup (Contact)	Yes
3	Model Year	Text (5)	Yes
4	Policy Start Date	Date	Yes
5	Policy Term Months	Number (18, 0)	Yes
6	Premium	Currency (16, 2)	Yes
7	Square Footage	Number (18, 0)	Yes
8	VIN	Text (20)	Yes
9	Year Built	Text (5)	Yes

5. Create Claim Custom Object:

- Go to **Setup** and click on **Object Manager**.
- Click on **Create** and select **Custom Object**.
- Fill in the following details:

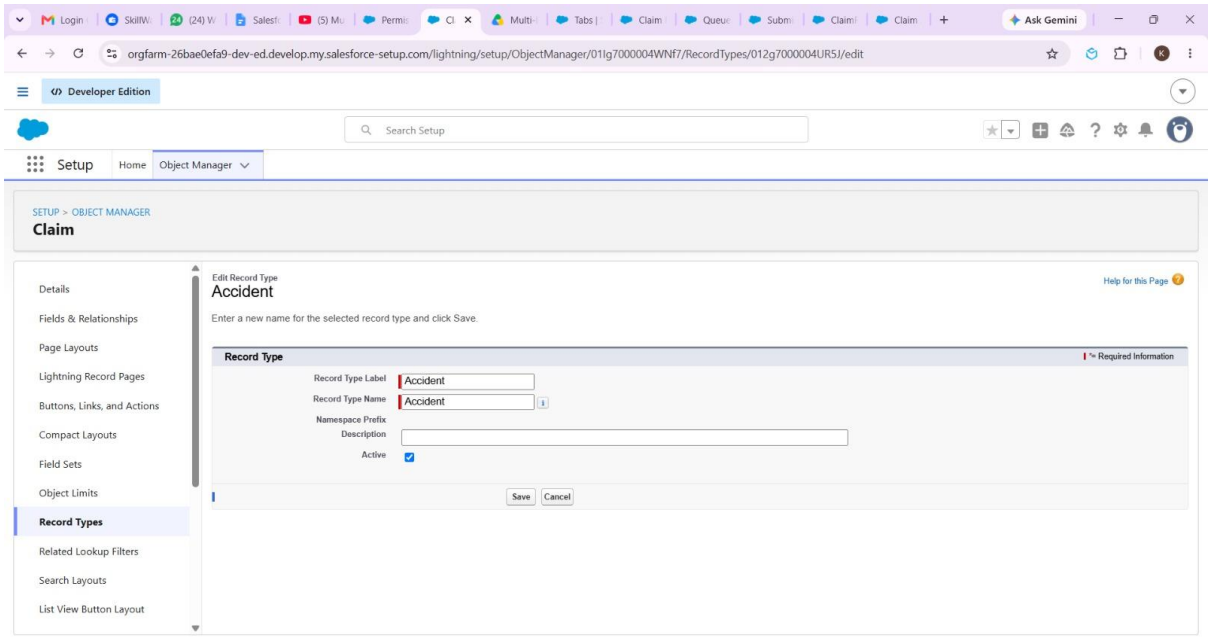
- **Label:** Claim
- **Plural Label:** Claims
- **Object Name:** Automatically Populated
- **Record Name:** Claim Number
- **Data Type:** Auto Number
- **Format:** C - {0000}
- **Starting Number:** 1
- Under **Optional Features**, enable **Allow Reports, Allow Activities, and Track Field History**.
- Under **Deployment Status**, select the **Deployed** radio button.
- Under **Search Status**, check the **Allow Search** checkbox.
- Click **Save**.

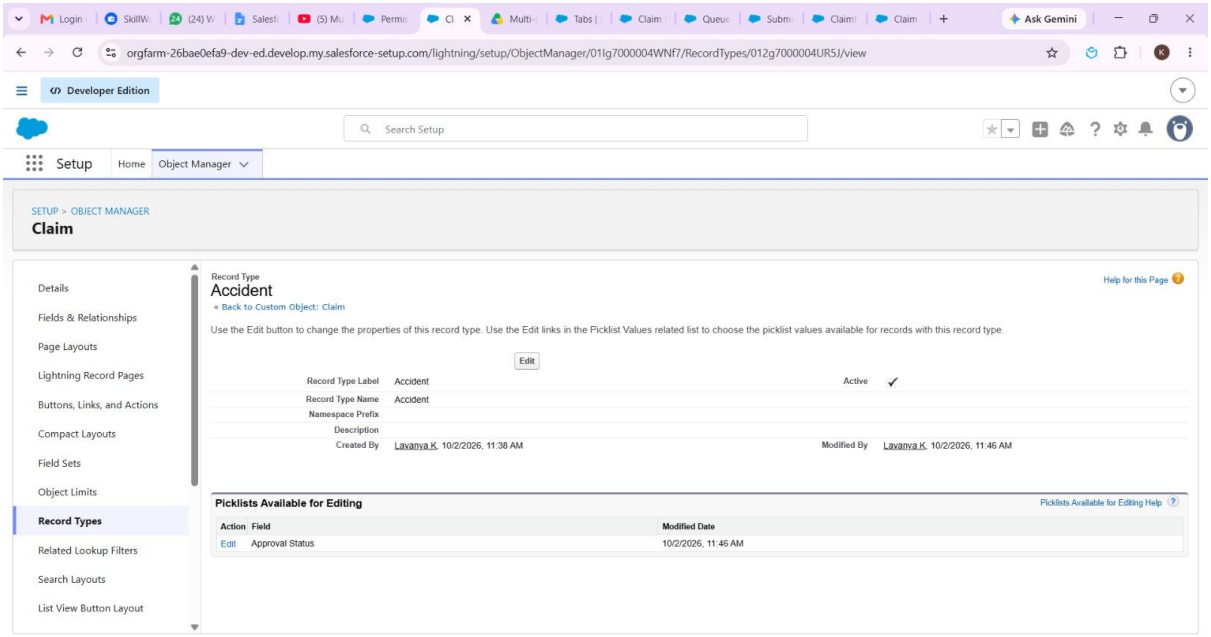




6. Define Record Types on Claim Object:

- Open **Setup** and access **Object Manager**.
- Locate the **Claim** object and open **Claim__c**.
- Select **Record Types** from the left navigation panel.
- Click **New** to add a record type.
- Choose **Master** as the existing record type.
- Enter **Accident** in the **Record Type Label** field.
- Allow Salesforce to automatically populate the **Record Type Name**.
- Provide a suitable description for the Accident claim category.
- Enable the **Active** option.
- Make the record type accessible to the required user profiles.
- Select **Next**.
- On the **Page Layout Assignment** screen, retain the existing layout settings.
- Select **Save** to complete the record type creation.





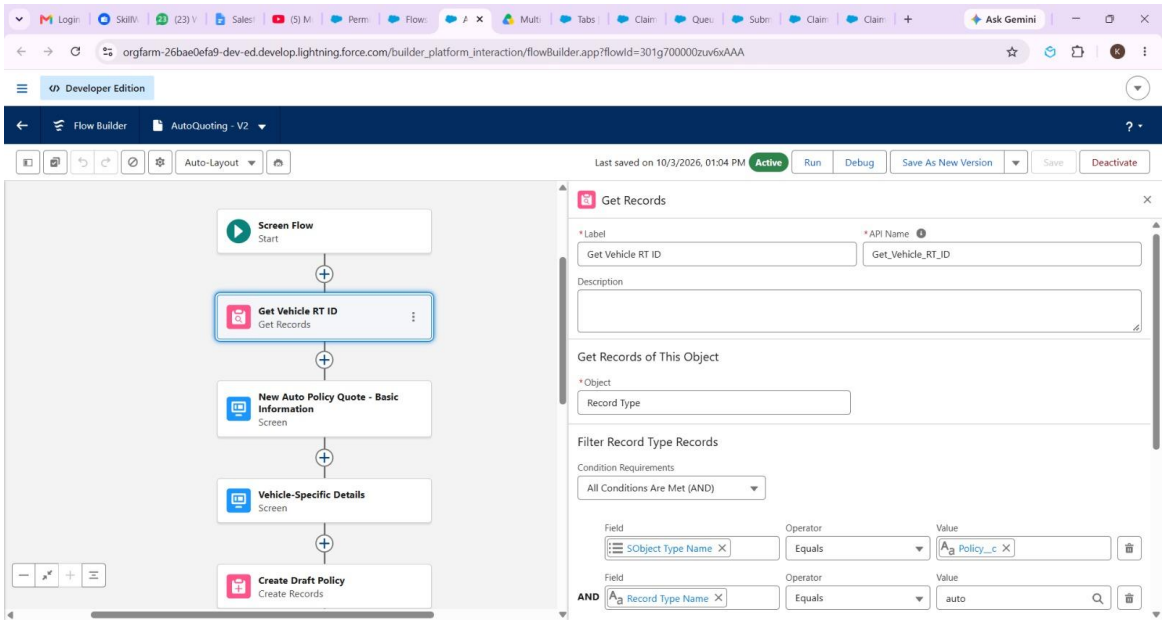
7. Create Fields in the Claim Object:

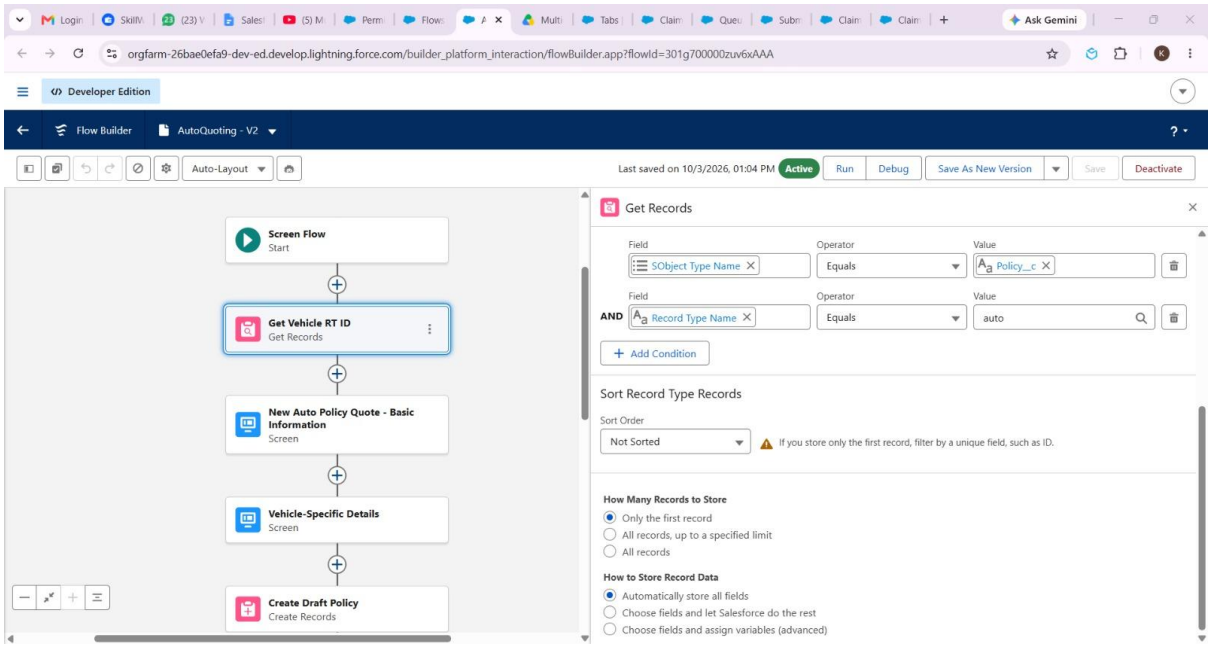
S.No	Field Name	Data Type	Required
1	Adjuster	Lookup (User)	Yes
2	Approval Status	Picklist (New, Submitted for Approval, Approved, Rejected)	Yes
3	Claim Amount	Currency (16, 2)	Yes

4	Date of Loss	Date/Time	Yes
5	Description	Long Text Area	No
6	Policy	Lookup (Policy)	Yes

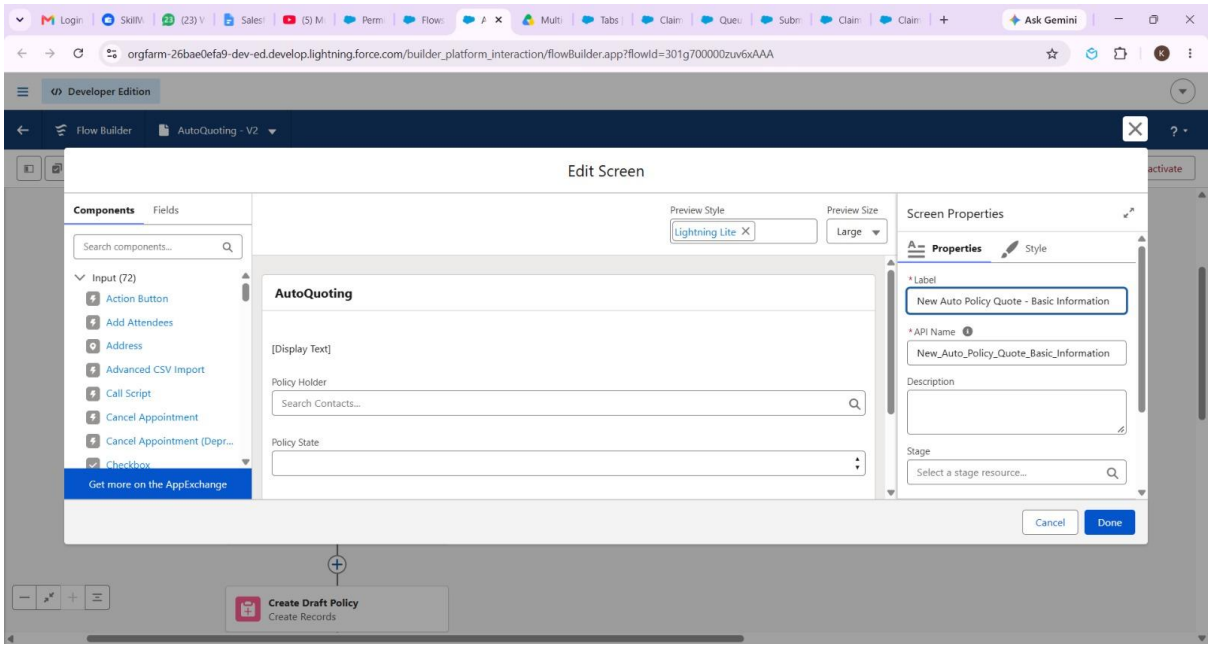
8. Create an AutoQuoting Screen Flow

- Open **Setup → Flows → New Flow**.
- Select **Screen Flow → Create**.
- Add **Get Records** after **Start**.
- Set **Label:** Get Vehicle RT ID.
- Keep the **API Name** automatic.
- Select **Record Type** as the object.
- Choose **All Conditions Are Met**.
- Set **SObject Type → Equals → Policy__c**.
- Add **And**.
- Set **Record Type Name → Equals → Auto**.

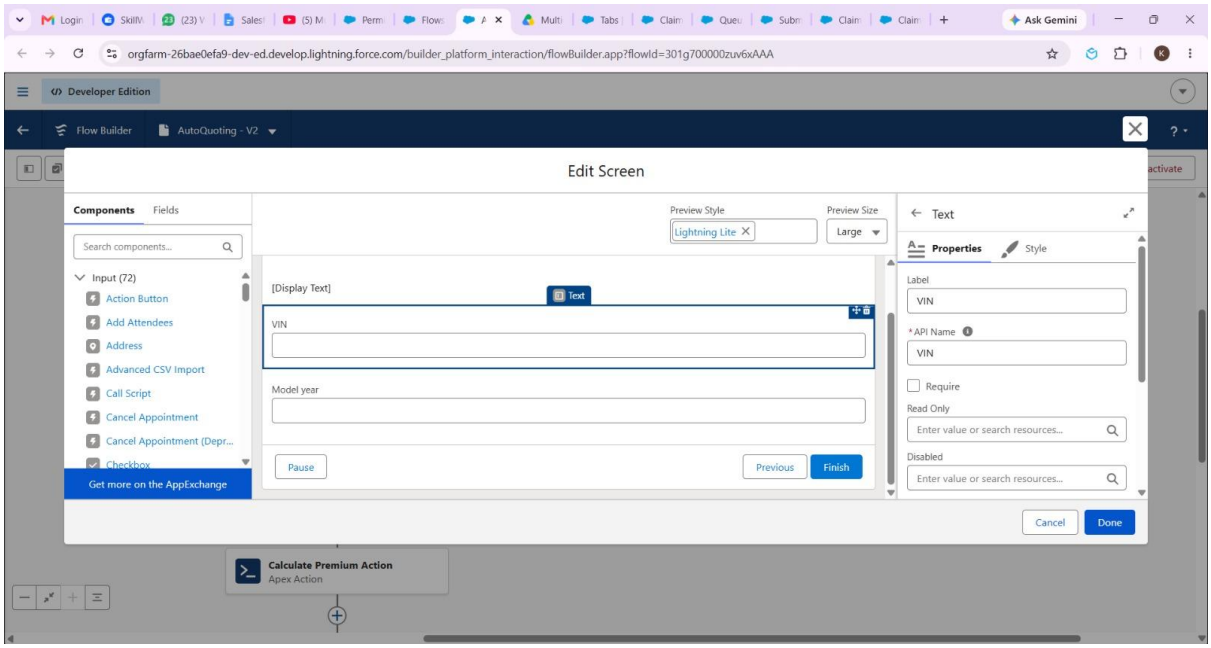




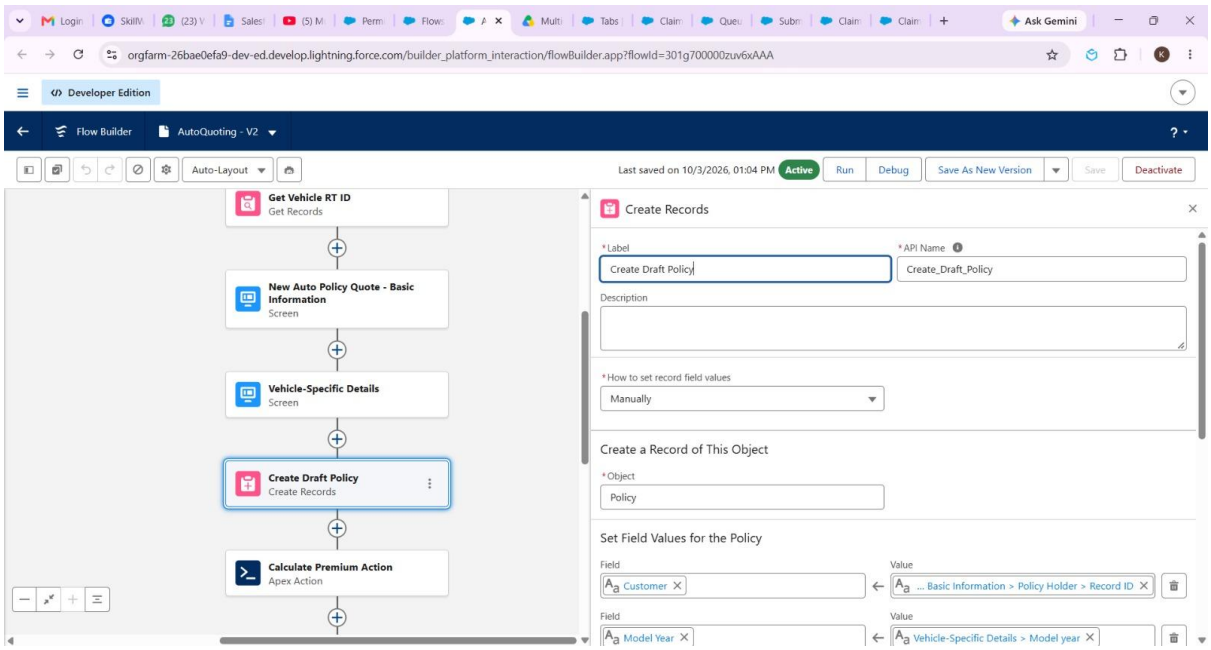
- Add a **Screen** element after **Get Records**.
- Label: **New Auto Policy Quote - Basic Information**.
- Add **Display Text** → API Name: Screen_1_Policy_Basics.
- Add **Lookup**:
    - API Name: Customer Lookup
    - Field API Name: Customer__c
    - Label: **Policy Holder**
    - Object API Name: Policy__c
- Add **Date** → Label: **Policy Start Date**.
- Add **Picklist** → Label: **Policy State**.
- Create a **Picklist Choice Set**:
    - API Name: required name
    - Object: **Policy**
    - Data Type: **Picklist**
    - Field: **Policy State**
- Click **Save**, then **Done**.

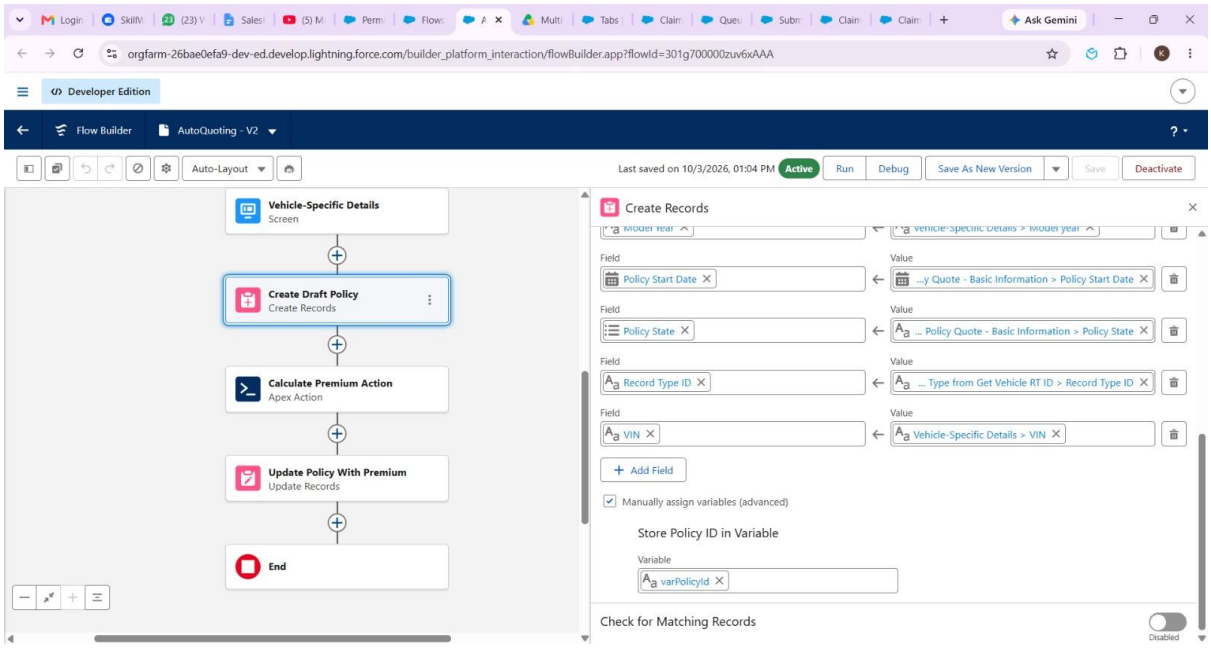


- Add another **Screen** element after the **Auto Policy Quote - Basic Information** screen.
- Set the **Label** as **Vehicle-Specific Details**.
- Allow the **API Name** to be populated automatically.
- Drag the **Display Text** component onto the flow canvas.
- Set the **API Name** as Screen_2_Vehicle_Details.
- Drag the **Text** component onto the canvas and set the **Label** as **VIN**. The API Name will be populated automatically.
- Drag another **Text** component onto the canvas and set the **Label** as **Model Year**. The API Name will be populated automatically.
- Click **Done**.

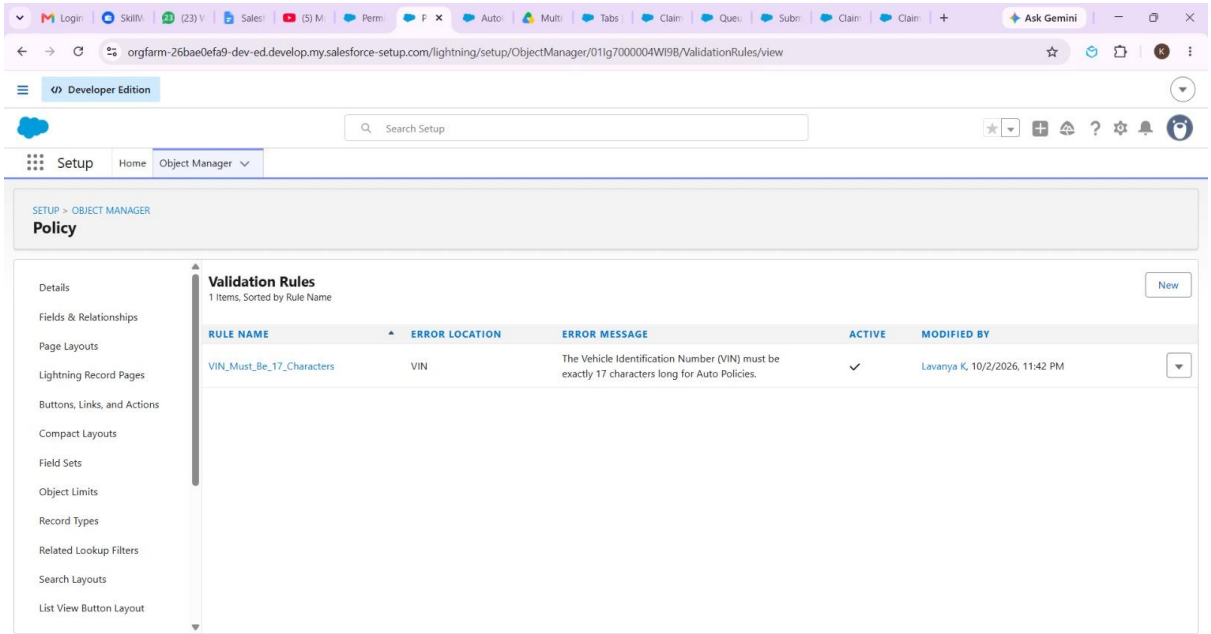


- Add a **Create Records** element after the **Screen** element.
- Set the **Label** as **Create Draft Policy**; API Name is auto-populated.
- Select **Store field values manually**.
- Select the **Policy** object.
- Set the Policy fields:
    - **Customer** → Basic Information Screen → Policy Holder → Record ID
    - **Model Year** → Vehicle Information Screen → Model Year
    - **Policy Start Date** → Basic Information Screen → Policy Start Date
    - **Policy State** → Basic Information Screen → Policy State
    - **RecordType ID** → Get Vehicle RT ID → Record ID
    - **VIN** → Vehicle Information Screen → VIN
- Select **Manually assign variables**.
- Create a new **Variable** resource → API Name: varPolicyId → Data Type: **Text**.
- Click **Done**.

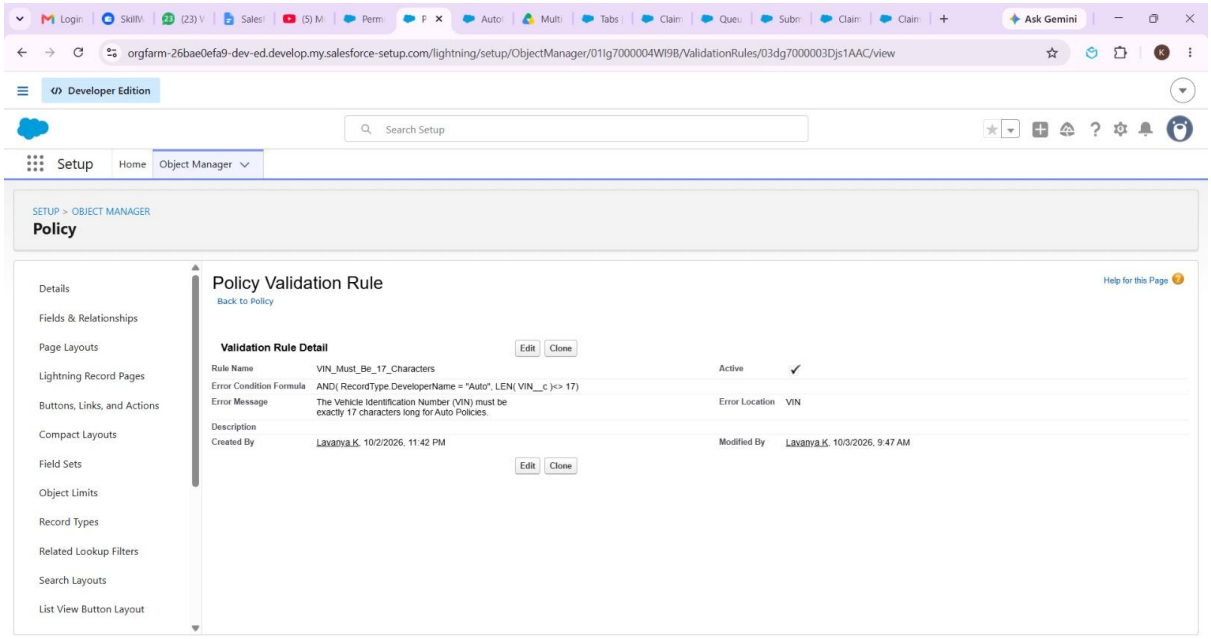




- Save the flow as **AutoQuotingFlow**.
- Activate the flow.
- **Create Validation Rule on Policy Object**
- Go to **Object Manager** and search for the **Policy** object.
- Open the **Policy** object.
- From the left panel, scroll to **Validation Rules** and select it.
- Click **New**.



- Set the **Rule Name** as VIN_Must_Be_17_Characters.
- Select the **Active** checkbox.
- Enter the following formula: AND(RecordType.DeveloperName = "Auto", LEN(VIN__c)<>17)
- Enter the **Error Message**: **The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.**
- Set **Error Location** to **Field** and select the **VIN** field.
- Click **Save**.



Milestone 2: Complex Policy Issuance & Claim Routing Automation

1. Create an Apex Class (PremiumCalculator) with a method that accepts policy data

- Open **Setup**, click the **Developer Console**, and open it in a new tab.
- Click **File** → **New** → **Apex Class**.
- Enter **PremiumCalculator** as the class name and click **Create**.
- Paste the provided **PremiumCalculator** Apex code into the class.

```
public class PremiumCalculator {
    public class PremiumInputWrapper {
        @InvocableVariable(required=true)
        public Id policyId;
```

```
    }
    @InvocableMethod(
        label='Calculate Policy Premium'
        description='Calculates premium based on policy fields (VIN, Model Year, State).'
    )
    public static List<Decimal> calculatePremium(
        List<PremiumInputWrapper> inputs
    ) {
        List<Decimal> premiumResults = new List<Decimal>();
        Set<Id> policyIds = new Set<Id>();
        for (PremiumInputWrapper input : inputs) {
            if (input.policyId != null) {
                policyIds.add(input.policyId);
            }
        }
        List<Policy__c> policiesToRate = [
            SELECT Id, VIN__c, Model_Year__c, Policy_State__c
            FROM Policy__c
            WHERE Id IN :policyIds
        ];
        for (Policy__c policy : policiesToRate) {
            Decimal basePremium = 1000.00;
            if (policy.Policy_State__c == 'CA') {
                basePremium *= 1.15;
            } else if (policy.Policy_State__c == 'TX') {
                basePremium *= 1.05;
            }
```

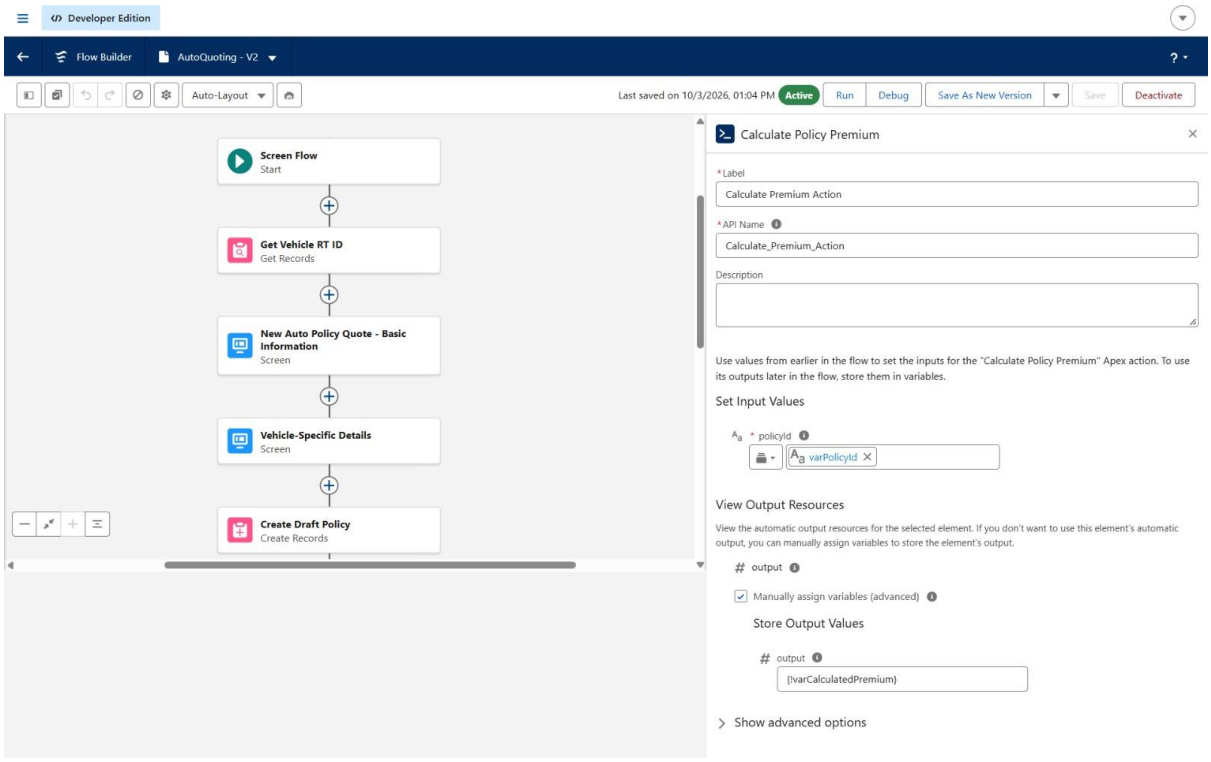
```
            if (
               policy.Model_Year__c != null &&
               Integer.valueOf(policy.Model_Year__c) < 2018
            ) {
               basePremium += 100;
            }
            premiumResults.add(basePremium);

        return premiumResults;
      }
    }
```

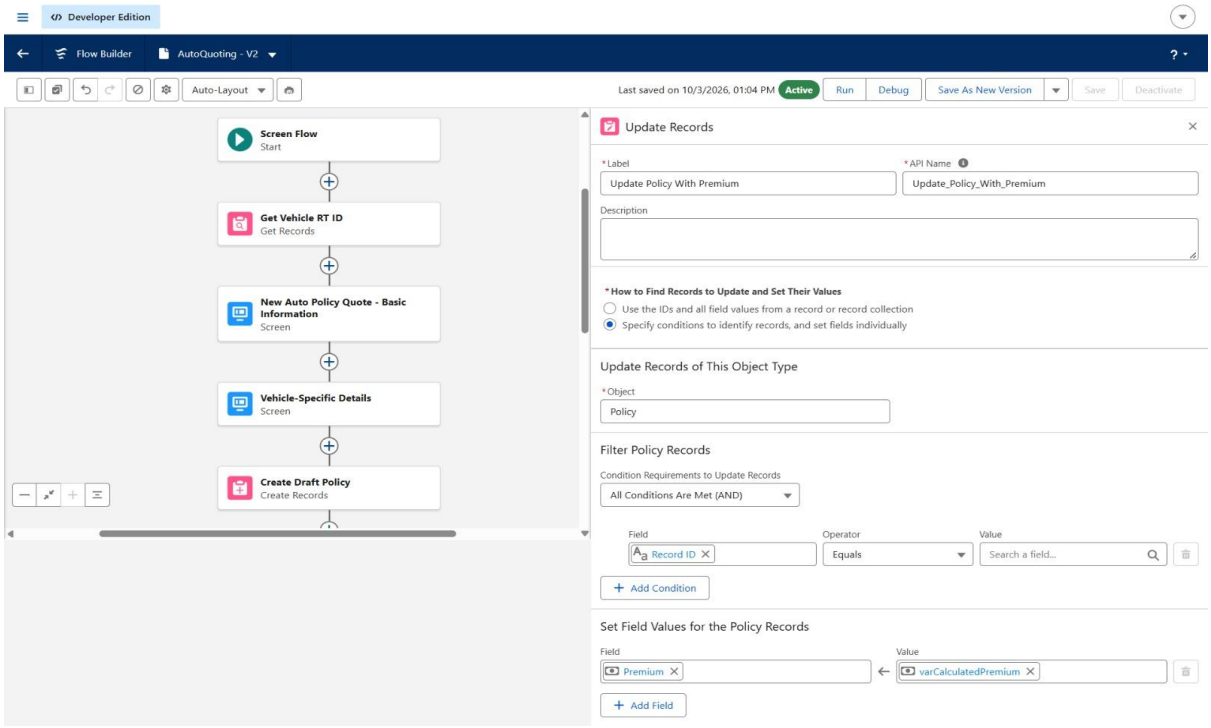
- Press **Ctrl + S** to save the Apex class.
- Invoke the **PremiumCalculator** Apex class in the **AutoQuotingFlow**.

2. Integrate Apex Action into AutoQuotingFlow

- Go to **Setup**, search **Flows** in Quick Find, and select **Flows**.
- Open **AutoQuotingFlow** in **Flow Builder**.
- Locate the **Create Records** element.
- Add an **Action** element after it.
- Label: **Calculate Premium Action**.
- Allow the **API Name** to populate automatically.
- Under **Set Input Values**, set **Policy Id** to varPolicyId.
- Under **View Output Resources**, select **Manually assign variables**.
- Create the output resource:
    1. **Resource Type:** Variable
    2. **API Name:** varCalculatedPremium
    3. **Data Type:** Currency
    4. **Decimal Places:** 2
- Click **Done**.



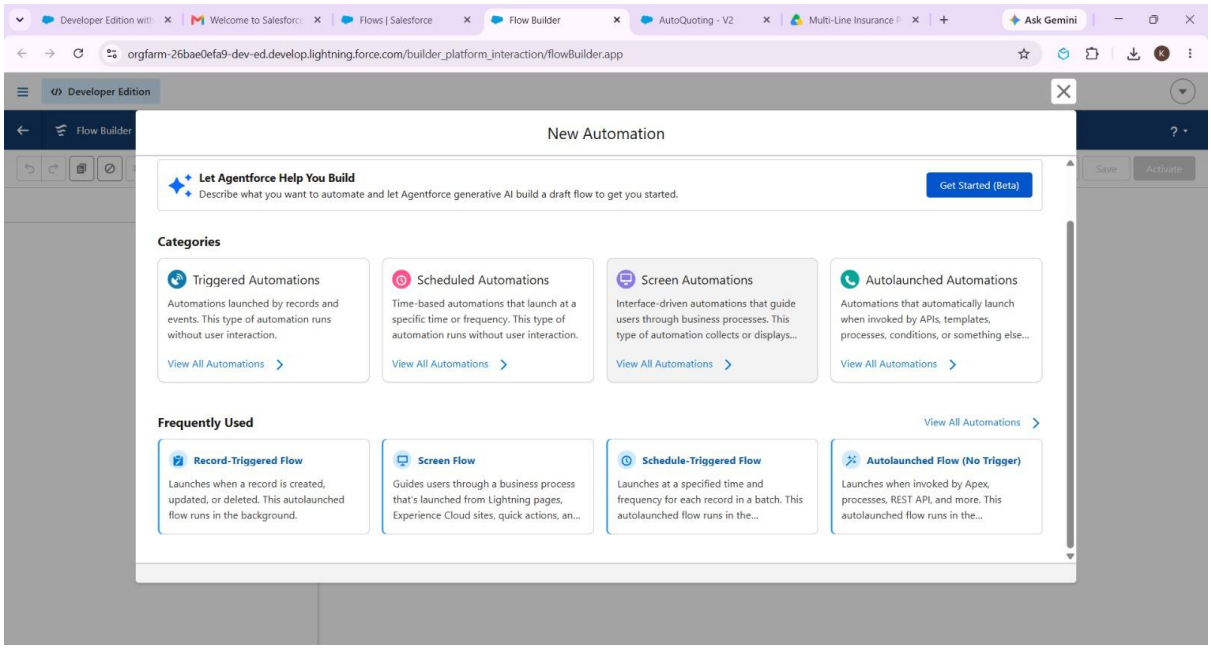
- After the **Action** element, add an **Update Records** element.
- Set the **Label** as **Update Policy With Premium**.
- Select **Specify conditions to identify records, and set fields individually**.
- Select **Policy** as the object.
- Select **All Conditions Are Met** for the filter criteria.
- Set the condition: **Record Id** → **Equals** → varPolicyId.
- Under **Set Field Values for Policy Record**, assign varCalculatedPremium to the **Premium** field.



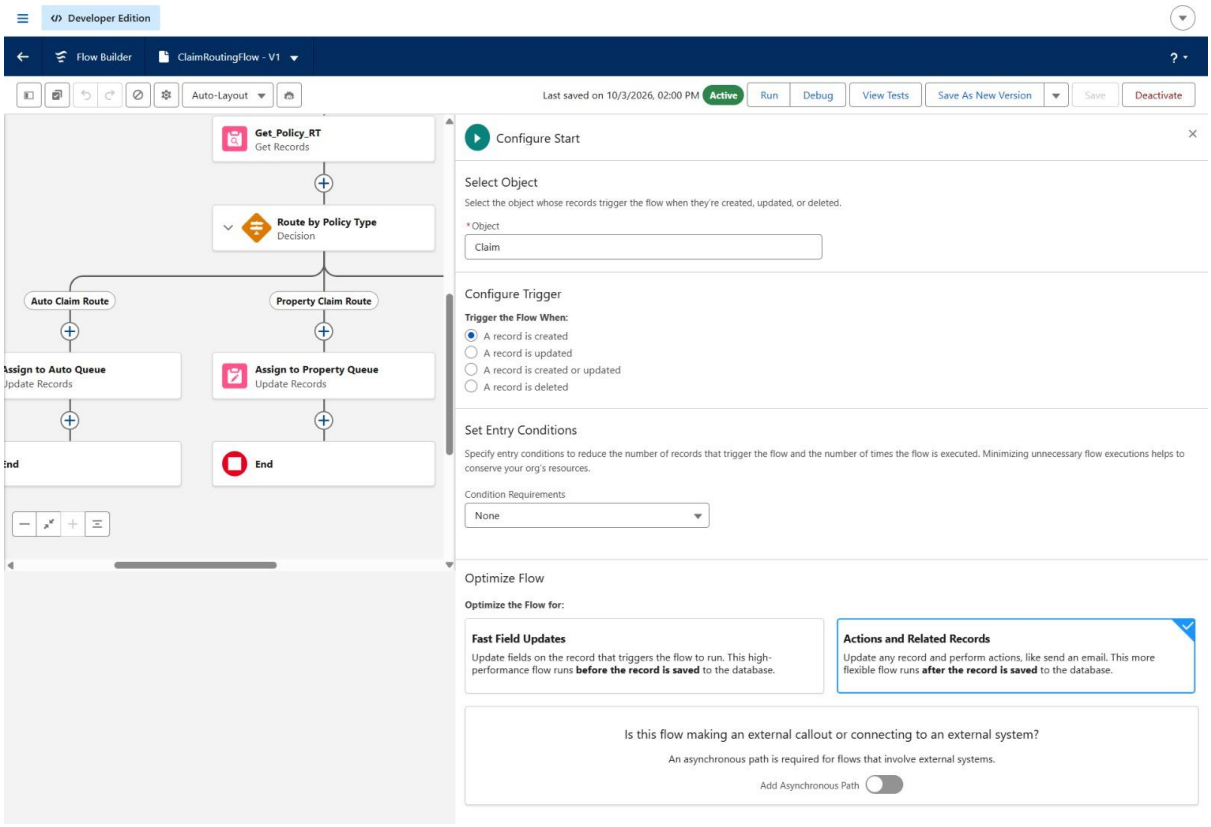
- Save the flow as a **new version**.
- **Activate** the flow.

3. Create a Record-Triggered Flow (After Save) on the Claim Object

- Go to **Setup**, search for **Flows** in the Quick Find box, and select **Flows**.
- Click **New Flow**.
- Select **Record-Triggered Flow** and click **Create**.

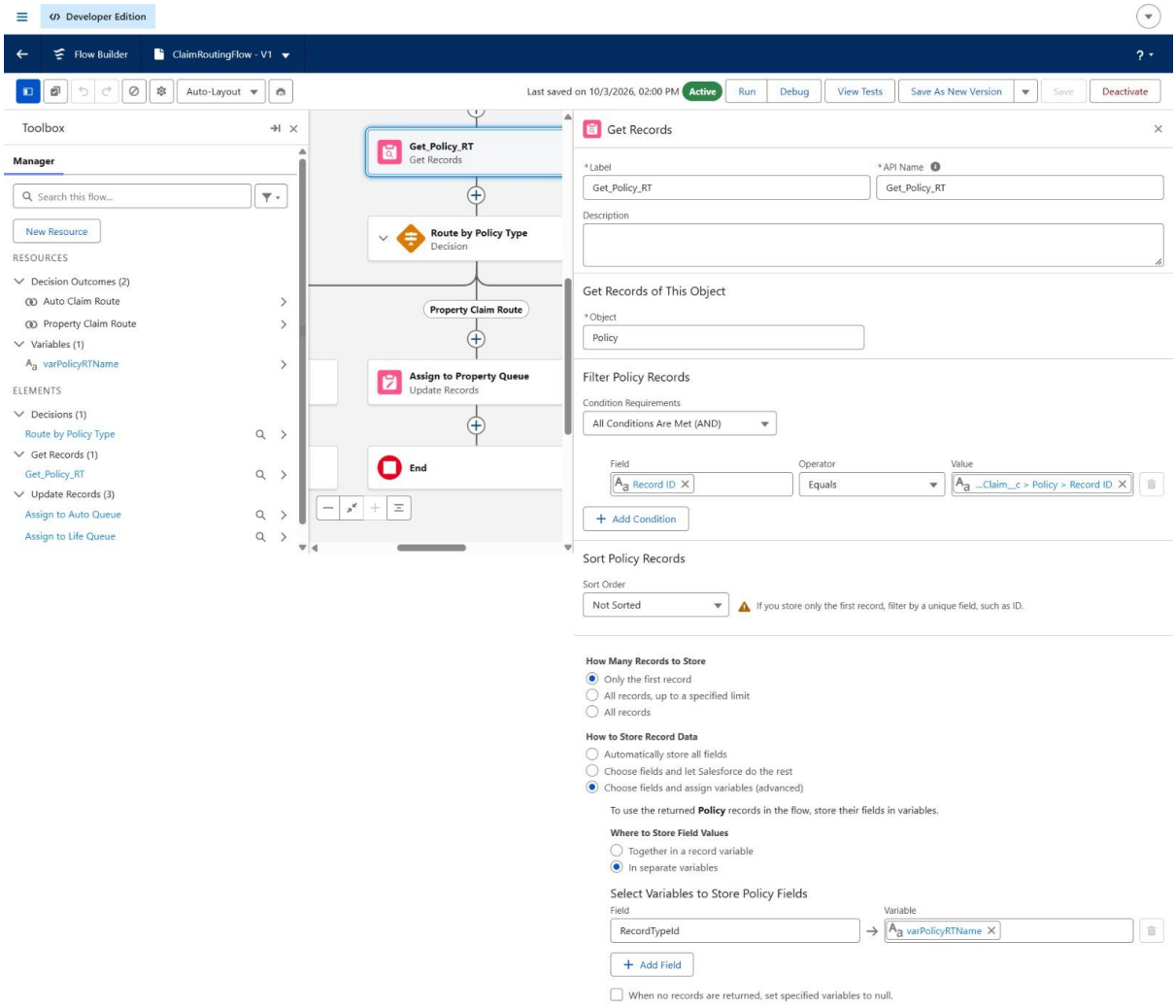


- Select **Claim** as the object.
- Configure the trigger to run when the **record is created**.
- Select **Actions and Related Records** under **Optimize the Flow For**.



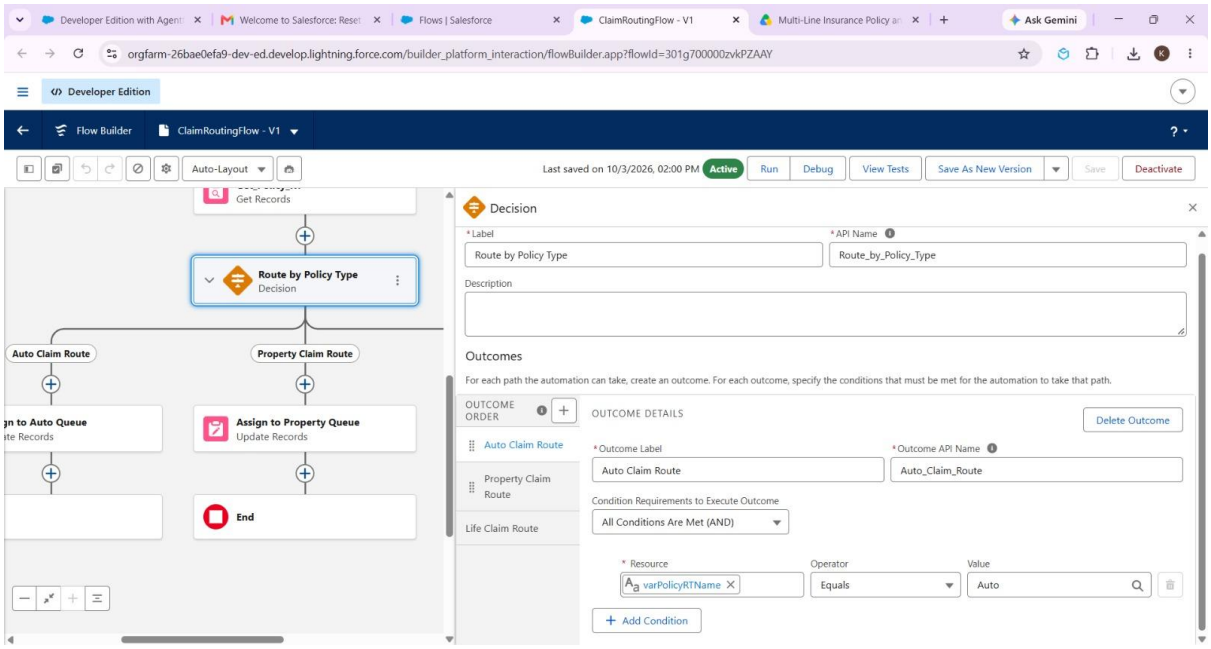
- Add a **Get Records** element after **Start** and set the **Label** as **Get_Policy_RT**.
- Select **Policy** as the object.
- Select **All Conditions Are Met** for the conditional requirements.
- Set the condition: **Record Id → Equals → Triggering Claim → Policy**.
- Select **Store only the first record**.
- Choose **Fields and assign variables for storing record data**.
- Select **Separate variables** for storing field values.
- Under **Select variables to store Policy fields**, for **RecordTypeID**, create a resource:
    1. **Resource Type:** Variable
    2. **API Name:** varPolicyRTName
    3. **Data Type:** Text

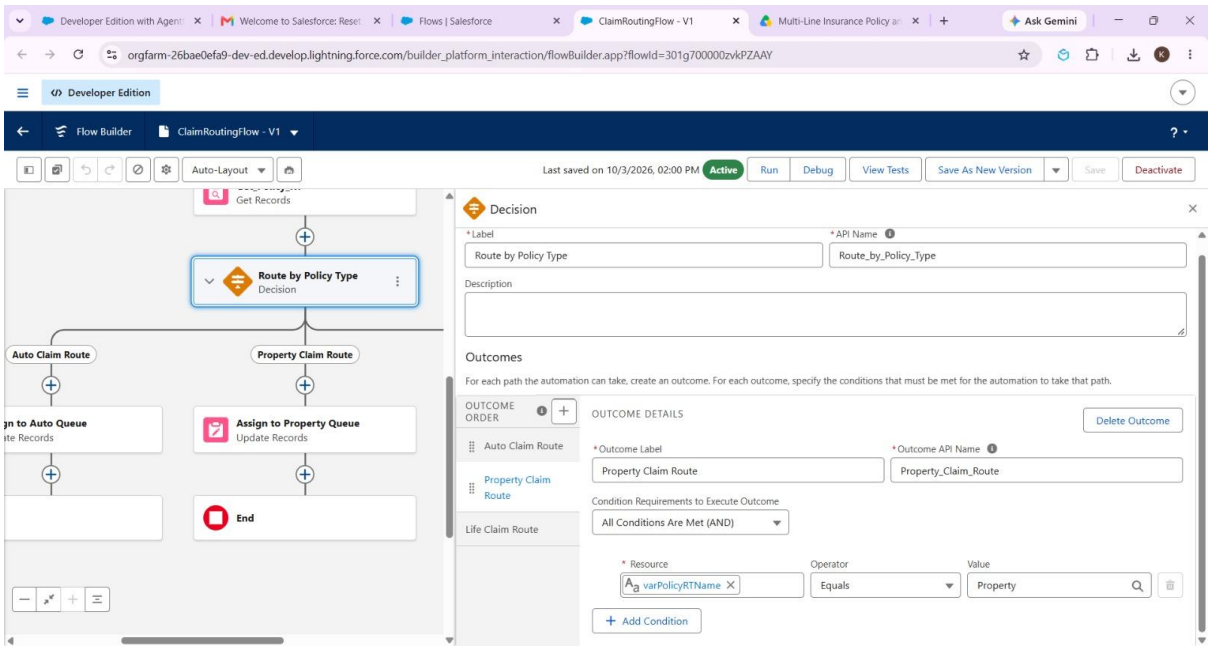
- Click **Done**.



- Under the **Get Records** element, add a **Decision** element.
- Set the **Label** as **Route by Policy Type**; the API Name will be auto-populated.
- Select **Manually set conditions** for the Decision Logic.
- Configure **Outcome 1**:
    1. **Label:** Auto Claim Route
    2. **Conditional Requirements:** All Conditions Are Met
    3. **Condition:** varPolicyRTName → **Equals** → Auto
- Configure **Outcome 2**:
    1. **Label:** Property Claim Route
    2. **Conditional Requirements:** All Conditions Are Met
    3. **Condition:** varPolicyRTName → **Equals** → Property

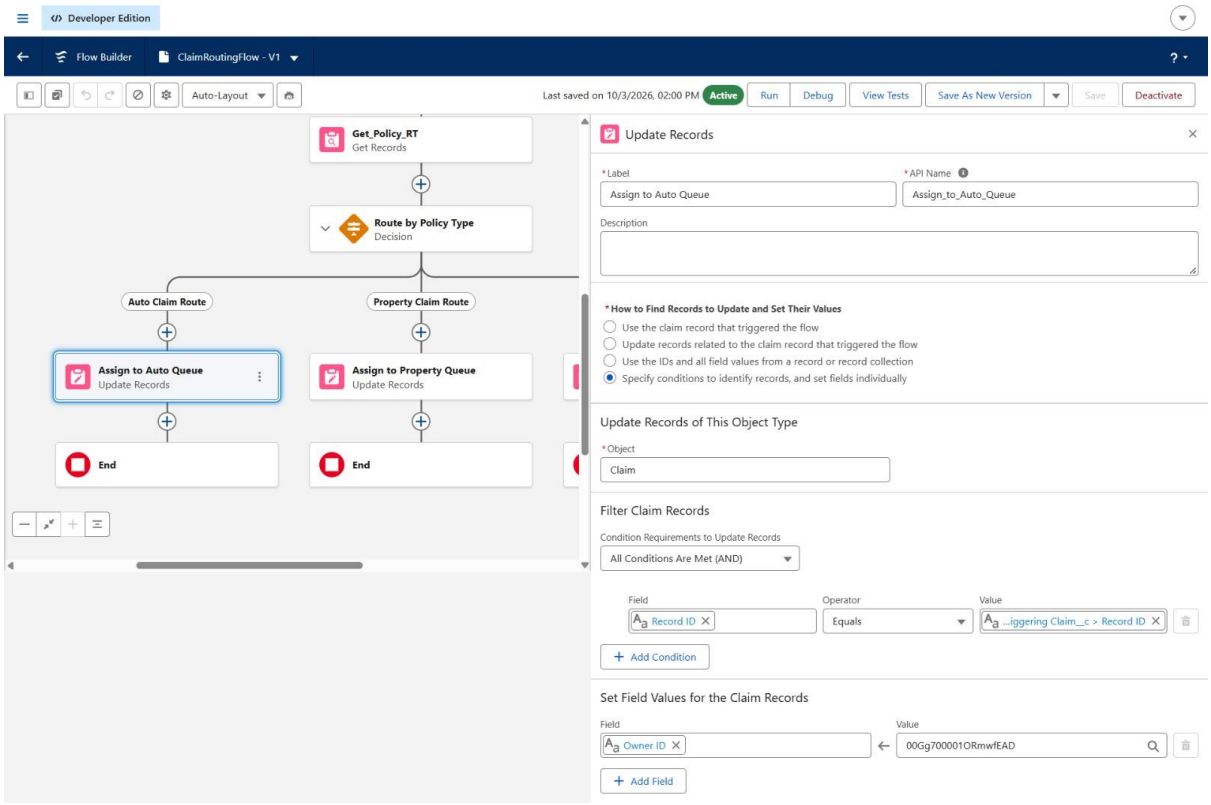
- Set the **Default Outcome** label as **Life Claim Route**.



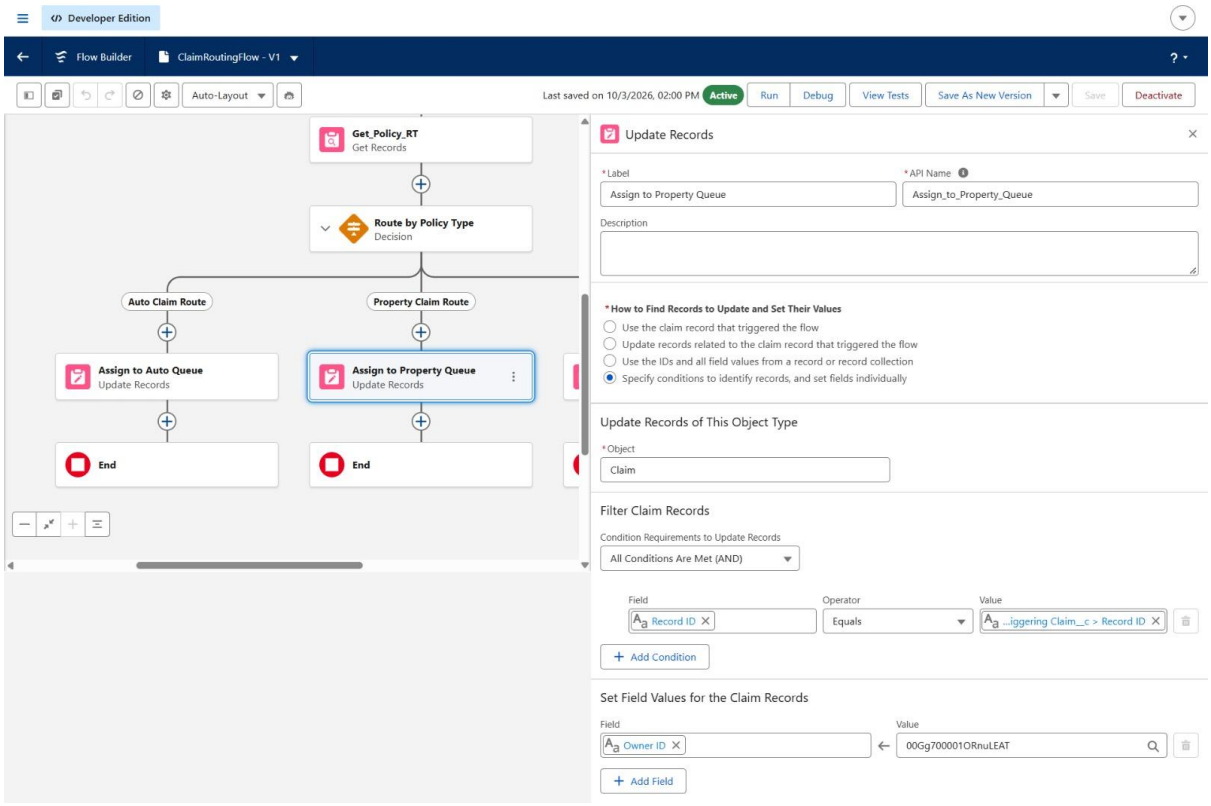


- Under the **Auto Claim Route** path, add an **Update Records** element.
- Set the **Label** as **Assign to Auto Queue**; the API Name will be auto-populated.
- Select **Specify conditions to identify records**.
- Select **Claim** as the object.
- Select **All Conditions Are Met**.
- Set **Record Id → Equals → Triggering Claim → Record ID**.

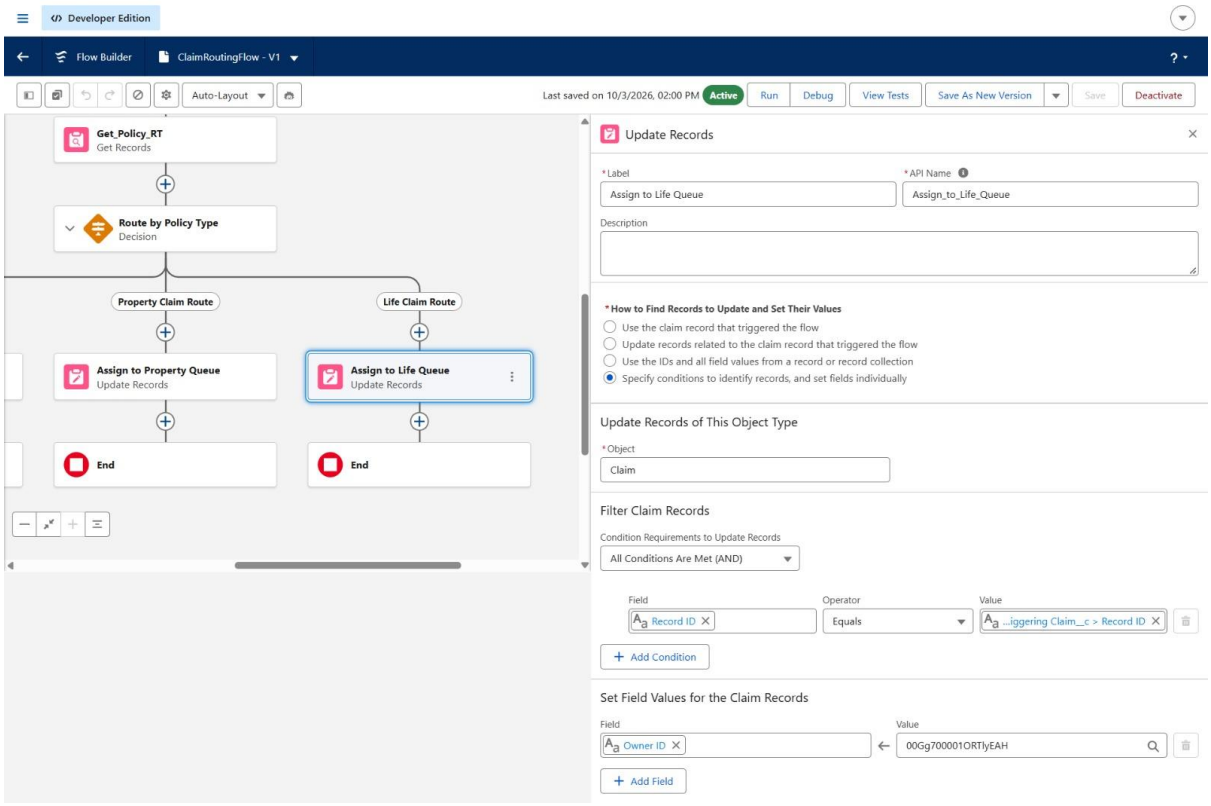
- Under **Set Fields for the Claim Records**, set the **Owner ID** field to the hardcoded **18-digit Auto Queue ID**.

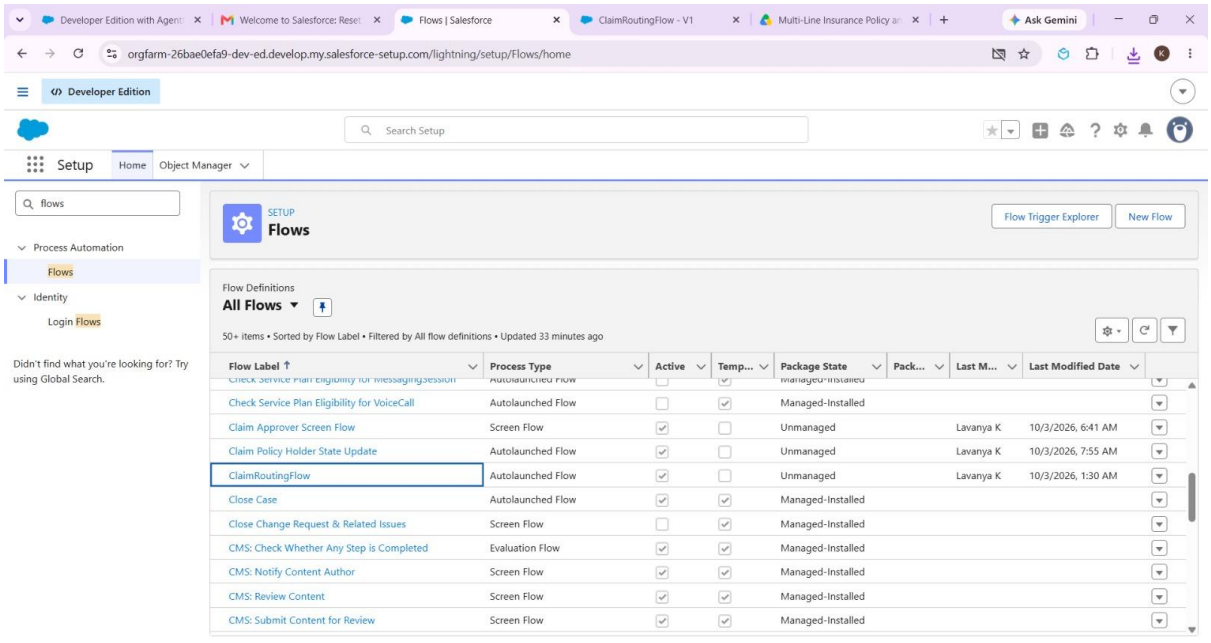


- Under the **Property Claim Route** path, add an **Update Records** element.
- Set the **Label** as **Assign to Property Queue**; the API Name will be auto-populated.
- Select **Specify conditions to identify records**.
- Select **Claim** as the object.
- Select **All Conditions Are Met**.
- Set **Record Id → Equals → Triggering Claim → Record ID**.
- Under **Set Fields for the Claim Records**, set the **Owner ID** field to the hardcoded **18-digit Property Queue ID**.



- Under the **Life Claim Route** path, add an **Update Records** element.
- Set the **Label** as **Assign to Life Queue**; the API Name will be auto-populated.
- Select **Specify conditions to identify records**.
- Select **Claim** as the object.
- Select **All Conditions Are Met**.
- Set **Record Id → Equals → Triggering Claim → Record ID**.
- Under **Set Fields for the Claim Records**, set the **Owner ID** field to the hardcoded **18-digit Life Queue ID**.





Milestone 3: Claims Adjuster LWC Dashboard Development

1. Create an Apex Class (ClaimsAdjusterController) with a SOQL-based method to retrieve assigned Claims

- Open **Setup**, right-click **Developer Console**, and open it in a new tab.
- In **Developer Console**, select **File → New → Apex Class**.

- Enter **ClaimsAdjusterController** as the class name and click **Create**.
- Paste the following modified code into the Apex class:

```
public class ClaimsAdjusterController {
  public class ClaimWrapper {
    @AuraEnabled public Id claimId;
    @AuraEnabled public String claimNumber;
    @AuraEnabled public Decimal claimAmount;
    @AuraEnabled public String status;
    @AuraEnabled public String policyType;
    @AuraEnabled public String policyHolderName;
    @AuraEnabled public Integer daysOpen;
    public ClaimWrapper(Claim__c claimRecord, Policy__c policyRecord, Contact customerRecord) {
      this.claimId = claimRecord.Id;
      this.claimNumber = claimRecord.Name;
      this.claimAmount = claimRecord.Claim_Amount__c;
      this.status = claimRecord.Approval_Status__c;
      Policy__c relatedPolicy = (Policy__c)policyRecord;
      if (relatedPolicy != null && relatedPolicy.RecordType != null) {
        this.policyType = relatedPolicy.RecordType.Name;
      } else {
        this.policyType = 'Unknown';
      }
      this.policyHolderName = (customerRecord != null) ? customerRecord.Name : '';

      this.daysOpen = (claimRecord.Date_of_Loss__c != null)
        ? (Date.today().daysBetween(claimRecord.Date_of_Loss__c.date()))
```

```
            : 0;
        }
    }
    @AuraEnabled(cacheable=true)
    public static List<ClaimWrapper> getAssignedClaims() {
        List<ClaimWrapper> claimResults = new List<ClaimWrapper>();
        List<Claim__c> assignedClaims = [
            SELECT
                Id, Name, Claim_Amount__c, Approval_Status__c, Date_of_Loss__c,
                Policy__r.Name,
                Policy__r.RecordType.DeveloperName,
                Policy__r.Customer__r.FirstName,
                Policy__r.Customer__r.LastName
            FROM Claim__c
            WHERE OwnerId = :UserInfo.getUserId()
            ORDER BY Date_of_Loss__c DESC
        ];
        for (Claim__c claimRecord : assignedClaims) {
            Policy__c relatedPolicy = claimRecord.Policy__r;
            Contact customerRecord = (relatedPolicy != null &&
relatedPolicy.Customer__r != null)
                ? (Contact)relatedPolicy.Customer__r
                : new Contact();
            claimResults.add(
                new ClaimWrapper(claimRecord, relatedPolicy, customerRecord)
            );
        }
        return claimResults;
```

```
    }
}
```

- Press **Ctrl + S** to save the Apex class.
- Use this Apex class in the **Claims Adjuster LWC component**.

2. Create a Component (claimsDashboardLwc) Using Apex and Client-Side Filtering

- Open Chrome, search for the **Lightning Studio Extension**, open the first result, click **Add to Chrome**, and confirm with **Add Extension**.
- Open the **Lightning Studio Extension** from your Salesforce org.
- Click **Create New** → **Lightning Web Component**.
- Enter the component name as claimsDashboardLwc.
- Select the **Is Exposed** checkbox.
- Under **Target Configurations**, select **Lightning App Page**, **Home Page**, and **Record Page**.
- Click **Deploy**.
- Open the **HTML** file and replace its contents with:

```
<template>
  <lightning-card title="My Assigned Claims Dashboard"
          icon-name="standard:claim">
    <div class="slds-m-around_medium">
      <lightning-combobox
        name="policyFilter"
        label="Filter by Policy Type"
        value="All"
        options={policyTypeOptions}
        onchange={handleFilterChange}
        class="slds-m-bottom_medium">
      </lightning-combobox>
      <template if:true={visibleClaims}>
```

```
            <div class="slds-grid slds-wrap slds-grid_pull-padded">
              <template for:each={visibleClaims} for:item="claim">
                <div key={claim.claimId}
                  class="slds-col slds-size_1-of-1
                  slds-medium-size_1-of-2
                  slds-large-size_1-of-3
                  slds-p-horizontal_small">
                  <c-claim-tile-lwc
                    claim-data={claim}>
                  </c-claim-tile-lwc>
                </div>
              </template>
            </div>
            <p class="slds-m-top_medium slds-text-heading_small">
              Total Claims Assigned: {visibleClaims.length}
            </p>
          </template>
          <template if:false={visibleClaims}>
            <p>No claims currently assigned to you.</p>
          </template>
        </div>
      </lightning-card>
    </template>
```

- Open the **JS** file and add the following code:

```
import { LightningElement, wire, track } from 'lwc';
import getAssignedClaims from
  '@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';
export default class ClaimsDashboardLwc extends LightningElement {
```

```
    @track allClaims = [];
    @track visibleClaims = [];
    error;
    @wire(getAssignedClaims)
    wiredClaims({ error, data }) {
      if (data) {
        this.allClaims = data;
        this.visibleClaims = data;
        this.error = undefined;
      } else if (error) {
        this.error = error;
        this.allClaims = undefined;
        this.visibleClaims = undefined;
        console.error(
          'Error retrieving claims:',
          JSON.stringify(error)
        );
      }
  }
    handleFilterChange(event) {
      const selectedPolicyType = event.target.value;
      if (selectedPolicyType === 'All') {
        this.visibleClaims = this.allClaims;
      } else {
        this.visibleClaims = this.allClaims.filter(
          claim => claim.policyType === selectedPolicyType
        );
      }
```

```
    }
    get policyTypeOptions() {
      return [
        { label: 'All Policy Types', value: 'All' },
        { label: 'Auto', value: 'Auto' },
        { label: 'Property', value: 'Property' },
        { label: 'Life', value: 'Life' }
      ];
    }
}
```

- Open the **meta-xml** file and add:

```
<?xml version="1.0"?>
<LightningComponentBundle
xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>63.0</apiVersion>
  <isExposed>true</isExposed>
  <targets>
    <target>lightning__AppPage</target>
    <target>lightning__HomePage</target>
  </targets>
</LightningComponentBundle>
```

3. Create a Reusable Component (claimTileLwc) to Display Essential Claim Information

1. Open the **Lightning Studio Extension** from your Salesforce org.
2. Click **Create New** → **Lightning Web Component**.
3. Enter the component name as claimTileLwc.
4. Select the **Is Exposed** checkbox.

5. Under **Target Configurations**, select **Lightning App Page**, **Home Page**, and **Record Page**.
6. Click **Deploy**.
7. Open the **HTML** file and paste the following code:

```
<template>
  <lightning-layout-item size="12" padding="around-small">
    <div class="tile slds-box slds-p-around_medium slds-theme_default">
      <header class="slds-media slds-media_center slds-m-bottom_small">
        <div class="slds-media__figure">
          <lightning-icon
            icon-name={policyIcon}
            alternative-text={claimData.policyType}
            size="small">
          </lightning-icon>
        </div>
        <div class="slds-media__body slds-m-left_small">
          <h2 class="slds-text-heading_medium slds-m-bottom_none">
            Claim: {claimData.claimNumber}
          </h2>
          <p class="slds-text-body_small">
            Policy Type: <strong>{claimData.policyType}</strong>
          </p>
        </div>
      </header>
      <div class="slds-border_top slds-p-top_small">
        <p>
          Policy Holder:
          <strong>{claimData.policyHolderName}</strong>
```

```
            </p>
            <lightning-formatted-number
              value={claimData.claimAmount}
              format-style="currency"
              currency-code="USD"
              label="Claim Amount">
            </lightning-formatted-number>
            <p>Status: {claimData.status}</p>
            <p class="slds-text-body_regular">
              Days Open:
              <strong class="slds-text-color_error">
                {claimData.daysOpen}
              </strong>
            </p>
          </div>
        </div>
      </lightning-layout-item>
    </template>
```

- Open the **JS** file and paste:

```
    import { LightningElement, api } from 'lwc';

    export default class ClaimTileLwc extends LightningElement {
      @api claimData;

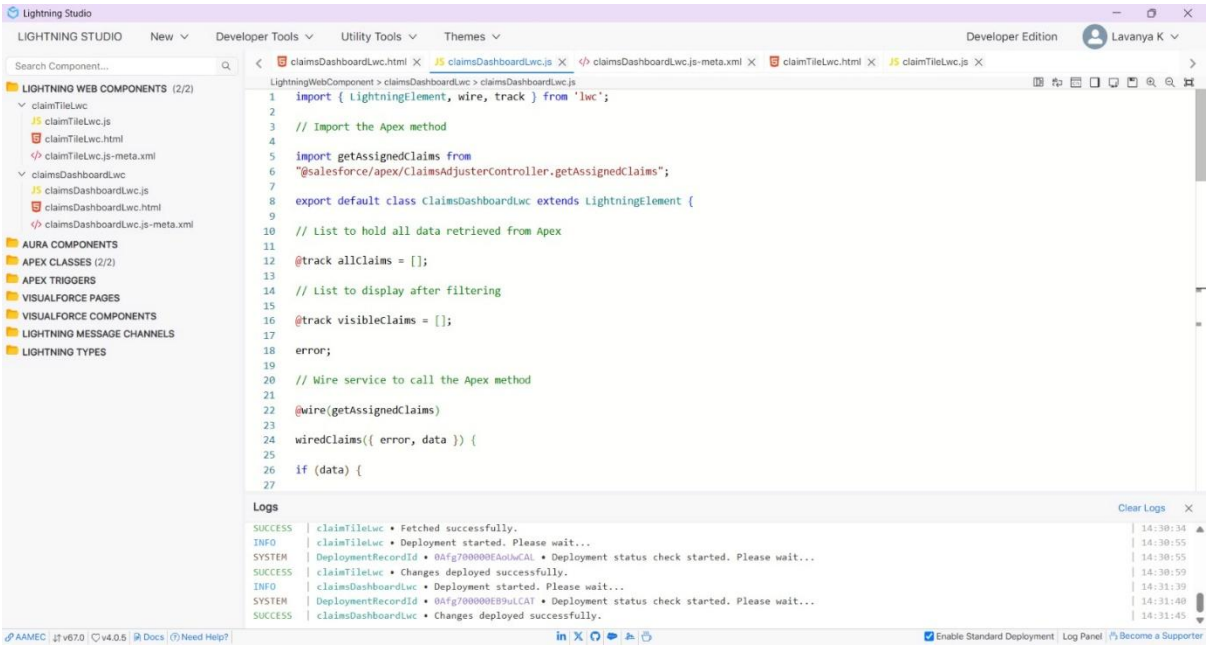
      get policyIcon() {
        if (this.claimData && this.claimData.policyType) {
          const type = this.claimData.policyType.toLowerCase();
```

```
        if (type === 'auto') return 'utility:truck';
        if (type === 'property') return 'utility:home';
        if (type === 'life') return 'utility:connected_apps';
      }

      return 'utility:help';
    }
  }
```

- Open the **meta-xml** file and paste:

```
<?xml version="1.0"?>
<LightningComponentBundle
xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>63.0</apiVersion>
  <isExposed>true</isExposed>
</LightningComponentBundle>
```



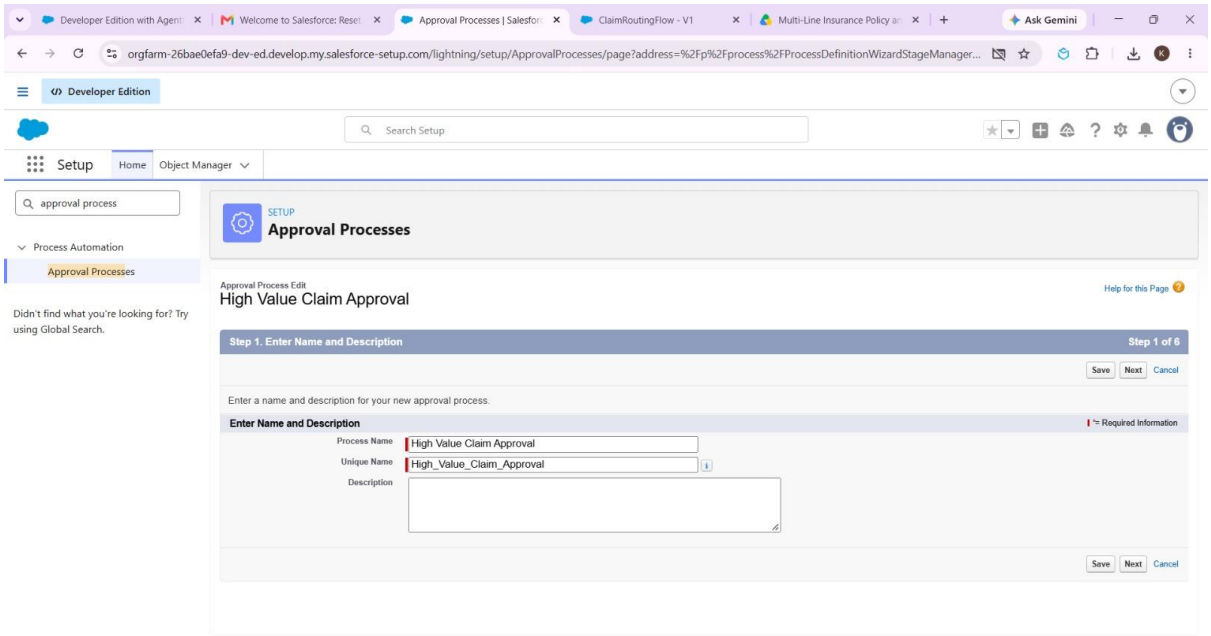
Milestone 4: Advanced Claim Processing, Security & Code Setup

1. Modify the Claim Object to Include an Approval Status Field

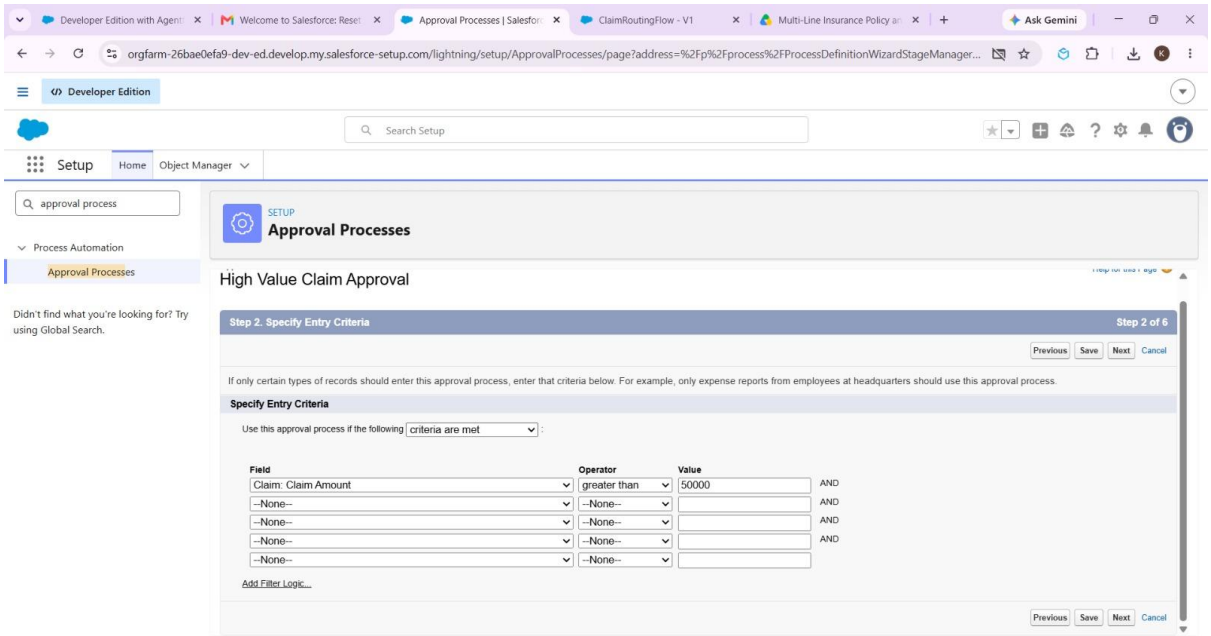
- Go to **Setup** → **Object Manager**, search for **Claim**, and open the **Claim** object.
- From the left panel, select **Fields and Relationships**.
- Click **New**.
- Select **Picklist** as the data type.
- Set the **Label** as **Approval Status**.
- Select **Enter values** and add:
    1. New
    2. Submitted for Approval
    3. Approved
    4. Rejected
- Allow the **Field Name** to populate automatically.
- Click **Next** → **Next** → **Save**.

2. Configure the Approval Process for Sequential Claim Review

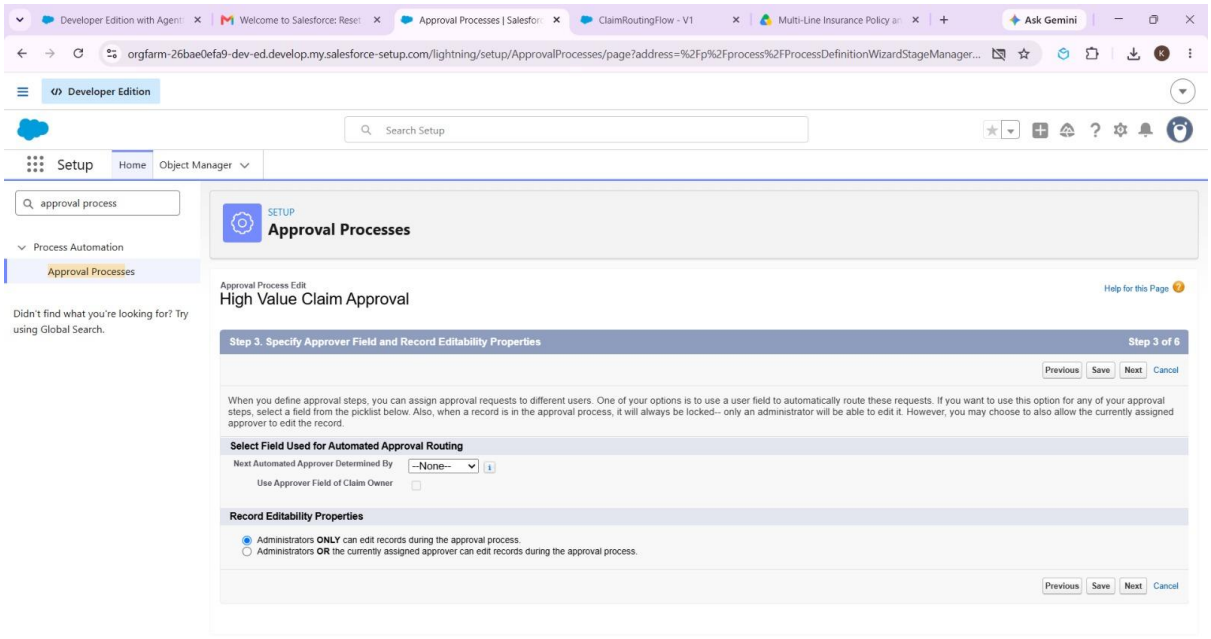
- Go to **Setup** and search for **Approval Processes** in Quick Find.
- Select **Claim** for **Manage approval processes for**.
- Click **Create New Approval Process** and select **Use Standard Setup Wizard**.
- Enter **High Value Claim Approval** as the process name.
- Allow the **Unique Name** to populate automatically.
- Click **Next**.



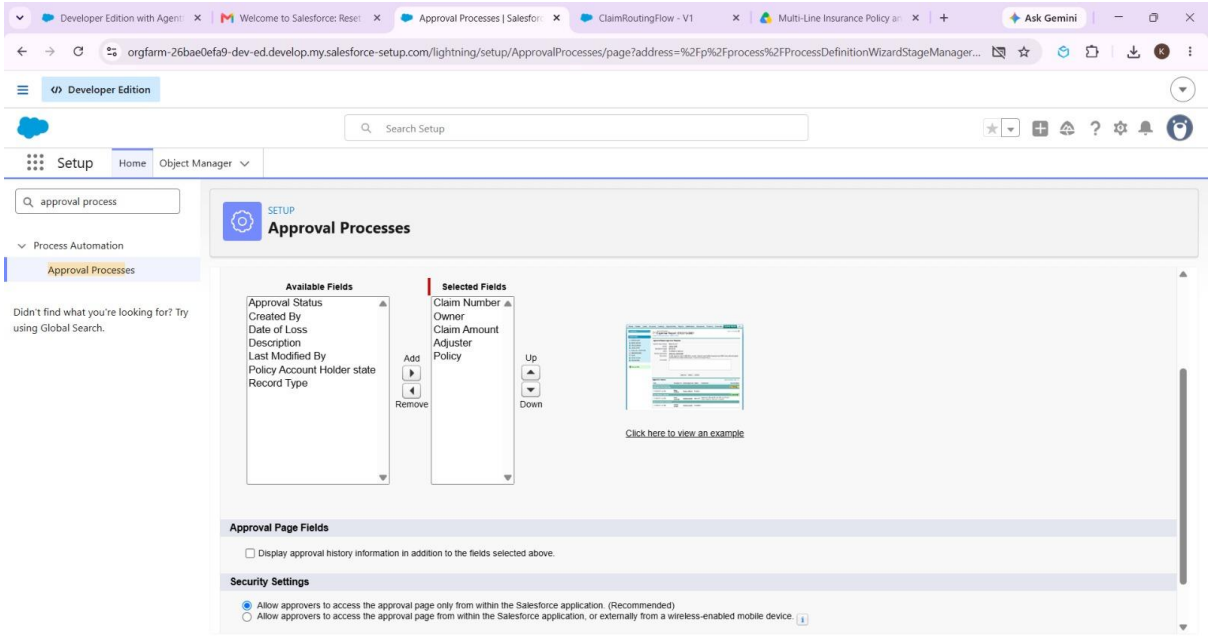
- Under **Specify Entry Criteria**, select **Criteria are met** for "Use this approval process if".
- Set the criteria to **Claim Amount greater than 50,000**.
- Click **Next**.



- Under **Record Editability Properties**, select **Administrators ONLY can edit the records during the approval process**.
- Click **Next**.

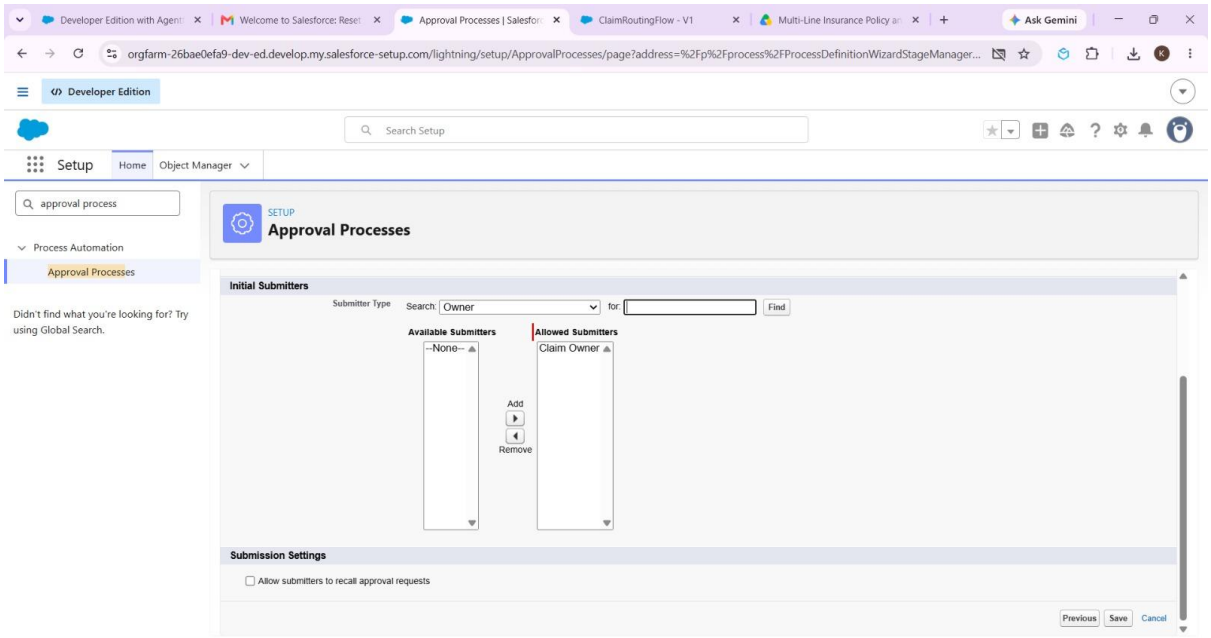


- Click **Next** again.
- Under **Approval Page Layout**, select **Claim Number, Owner, Claim Amount, Adjuster, and Policy** from the available fields and move them to the selected fields.
- Under **Security Settings**, select **Allow Approvers to access the page only from within the Salesforce application**.
- Click **Next**.



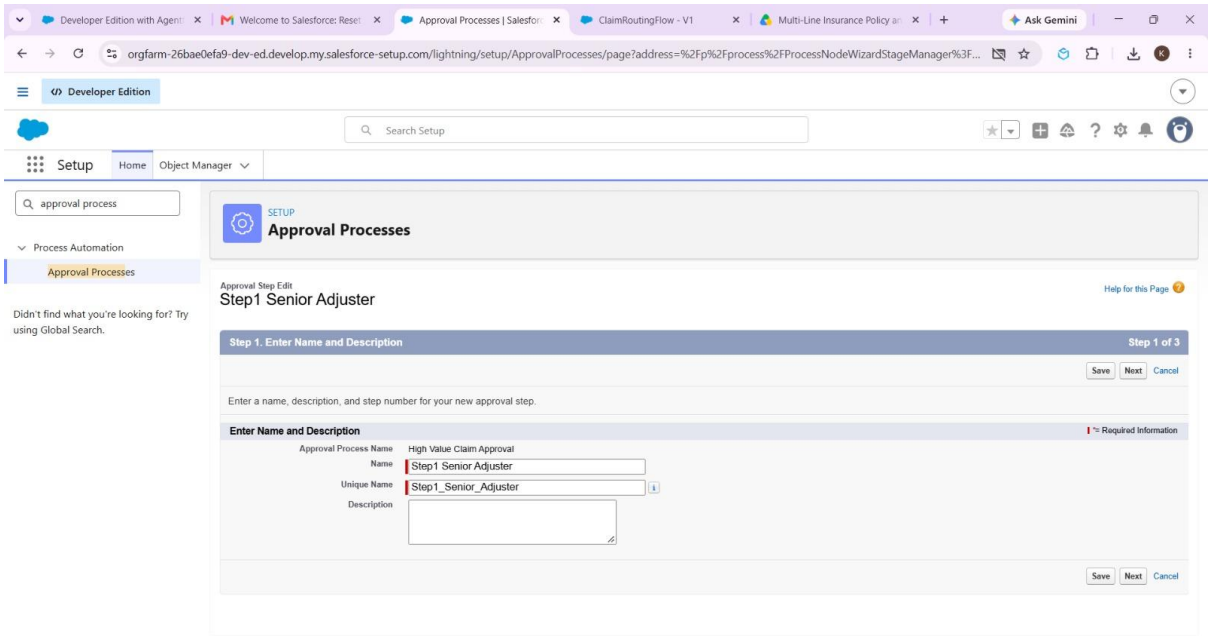
- Under **Initial Submitters**, search for **Owner** under **Submitter Type**.
- Select **Claim Owner** as an allowed submitter from the available submitters.

- Click **Save**.



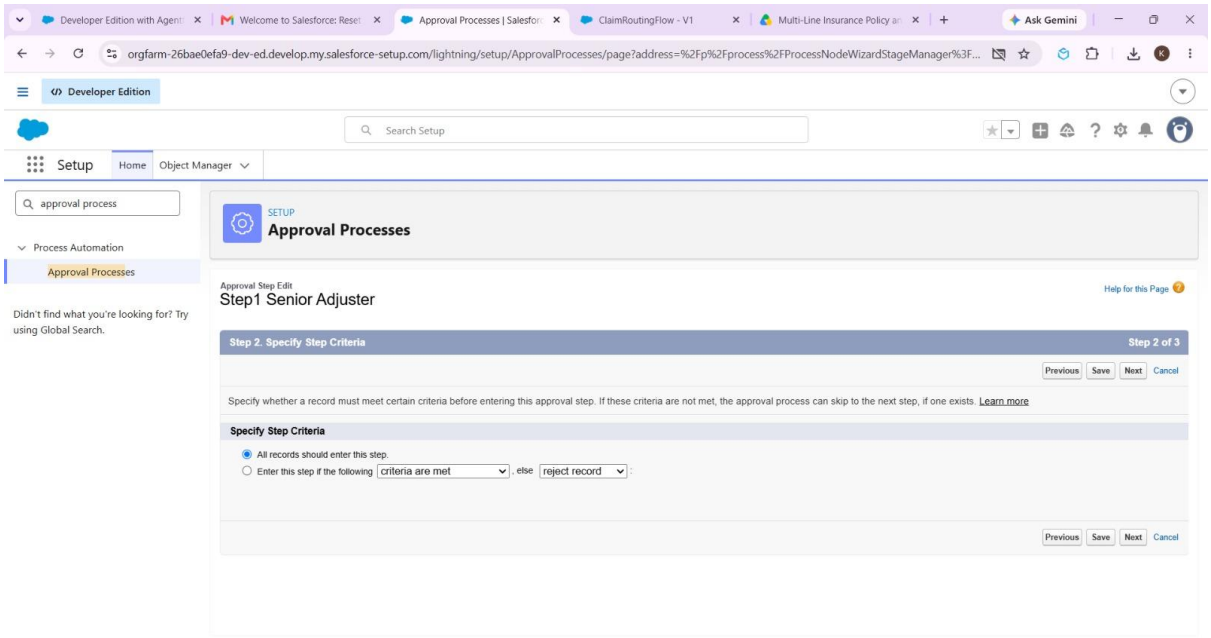
Step 1: Create Approval Step for Senior Adjuster

- Under **Initial Submission Actions**, locate **Approval Steps**.
- Under **Approval Steps**, click **New**.
- Enter the name **Step1 Senior Adjuster**.
- The **Unique Name** will be populated automatically.
- Click **Next**.

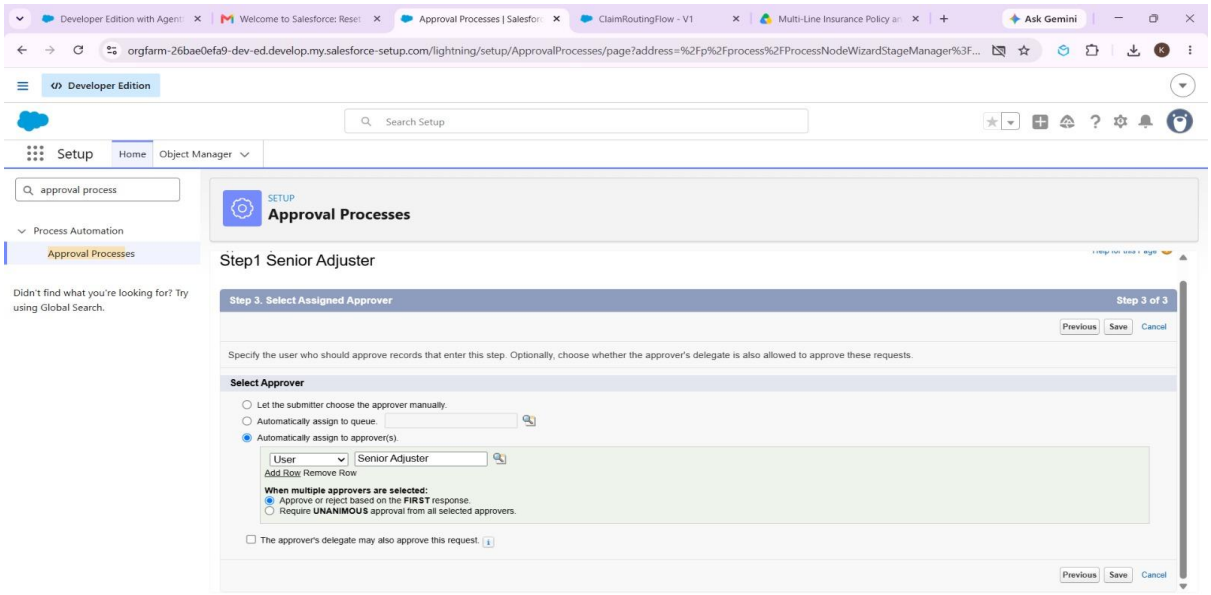


- Under **Specify Step Criteria**, select **All records should enter the step**.

- Click **Next**.

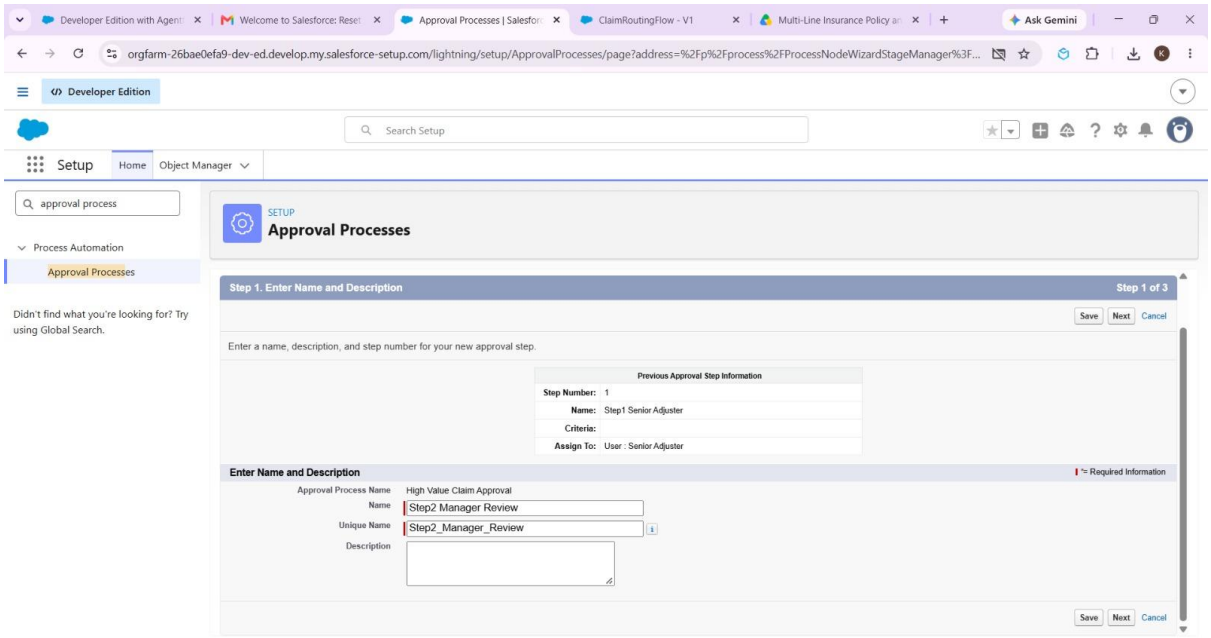


- Under **Select Approver**, choose **Automatically assign to approver**.
- Select the **Senior Adjuster User**.
- Under **When multiple approvers are selected**, choose **Approve or Reject based on the FIRST Response**.
- Click **Save**.

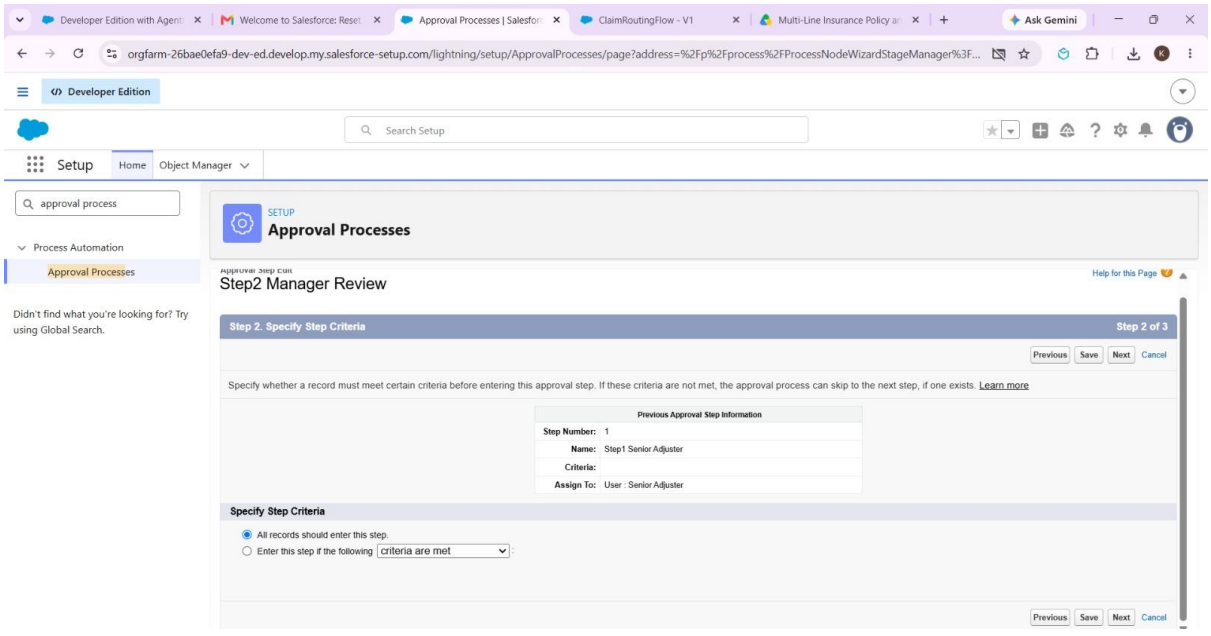


- Click **New** again to create another approval step.
- Enter the name **Step2 Manager Review**.
- The **Unique Name** will be populated automatically.

- Click **Next**.

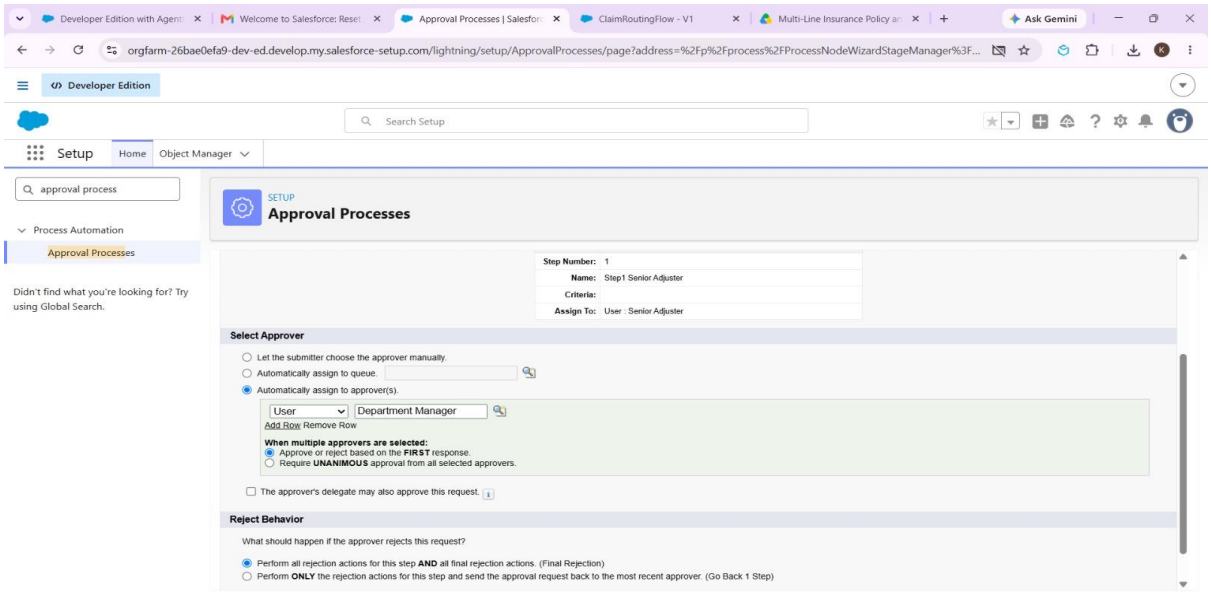


- Under **Specify Step Criteria**, select **All records should enter the step**.
- Click **Next**.

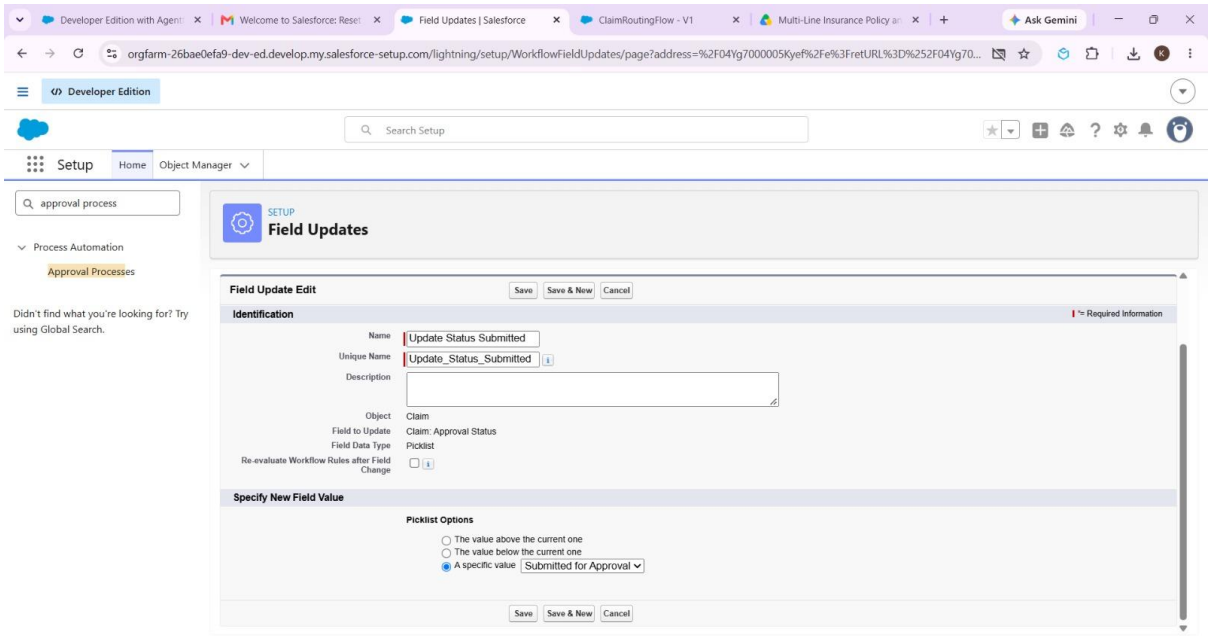


- Under **Select Approver**, choose **Automatically assign to approver**.
- Select the **Department Manager User**.
- Under **When multiple approvers are selected**, choose **Approve or Reject based on the FIRST Response**.

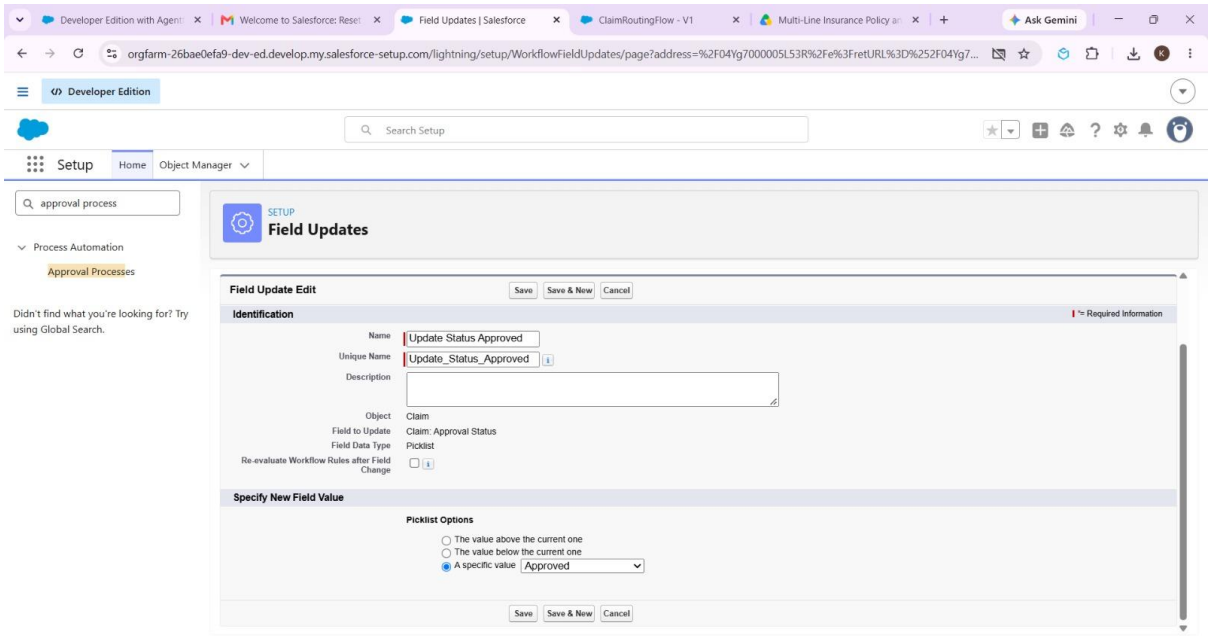
- Under **Reject Behaviour**, select **Perform all rejection actions for this step AND all final rejection actions**.
- Click **Save**.



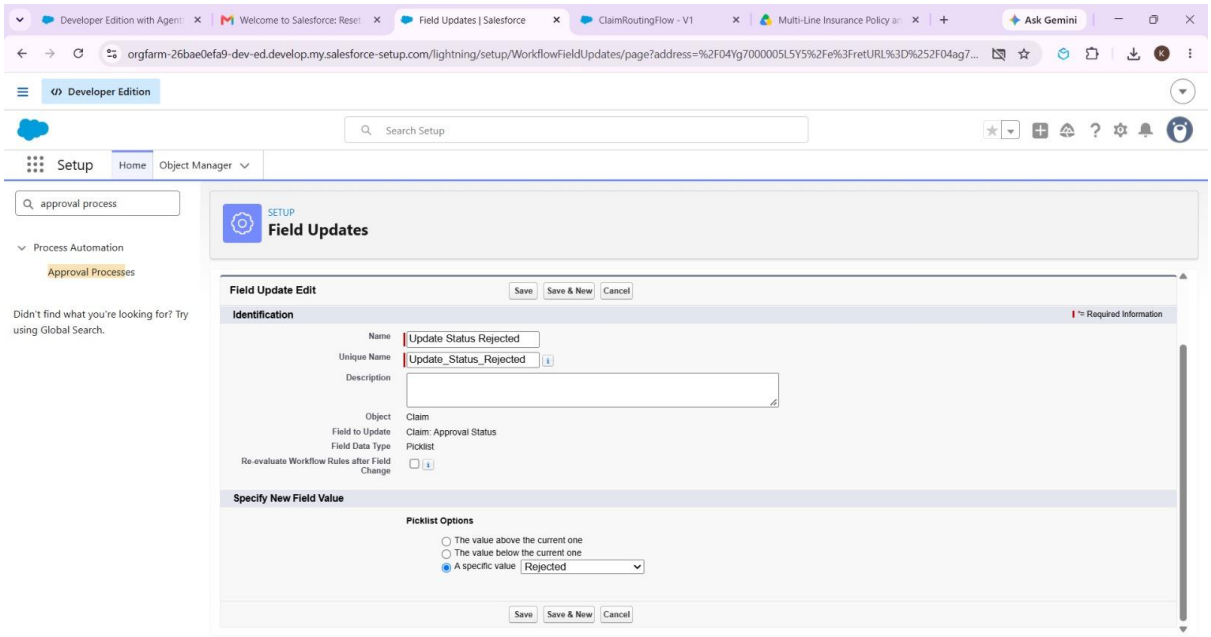
- Under **Initial Submission Actions**, click **Add New** and select **Field Update**.
- Enter the name **Update Status Submitted**.
- The **Unique Name** will be populated automatically.
- Select **Claim** as the object.
- Select **Approval Status** as the field to update.
- Select **Picklist** as the field data type.
- Under **Specify New Field Value**, select **A Specific Value** and choose **Submitted for Approval**.
- Click **Save**.



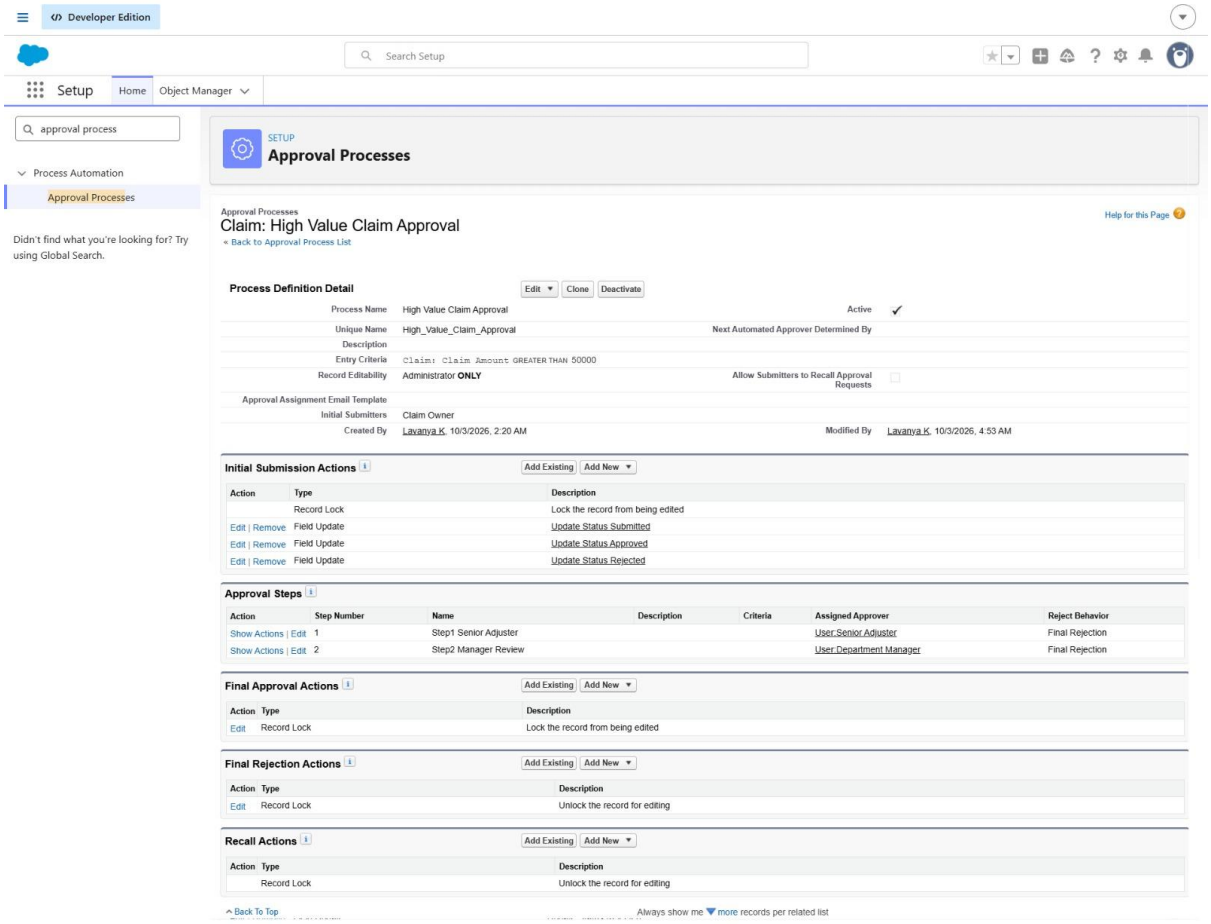
- Under **Final Approval Actions**, click **Add New** and select **Field Update**.
- Enter the name **Update Status Approved**.
- The **Unique Name** will be populated automatically.
- Select **Claim** as the object.
- Select **Approval Status** as the field to update.
- Select **Picklist** as the field data type.
- Under **Specify New Field Value**, select **A Specific Value** and choose **Approved**.
- Click **Save**.



- Under **Final Rejection Actions**, click **Add New** and select **Field Update**.
- Enter the name **Update Status Rejected**.
- The **Unique Name** will be populated automatically.
- Select **Claim** as the object.
- Select **Approval Status** as the field to update.
- Select **Picklist** as the field data type.
- Under **Specify New Field Value**, select **A Specific Value** and choose **Rejected**.
- Click **Save**.



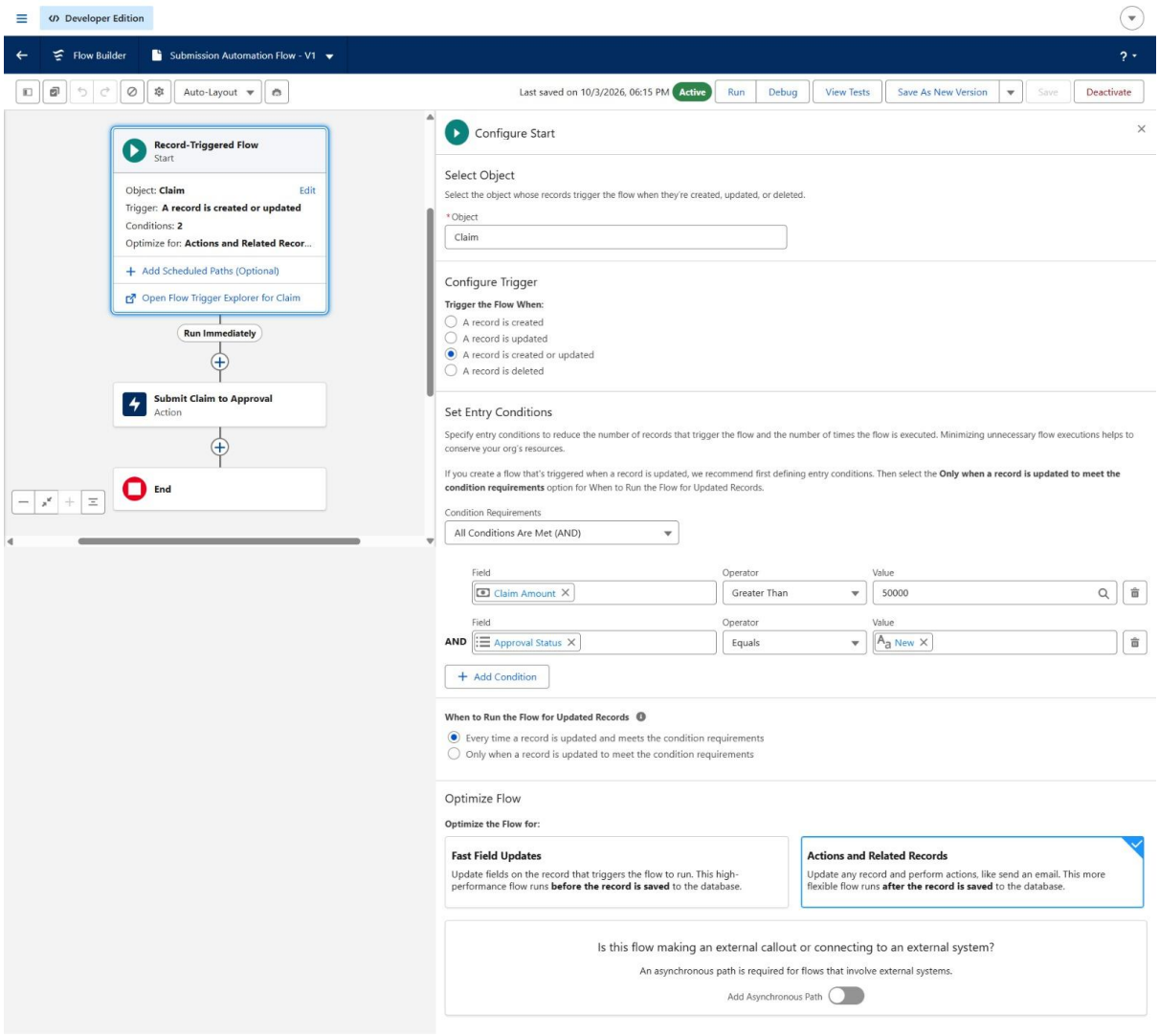
- Click **Activate** to enable the Approval Process.



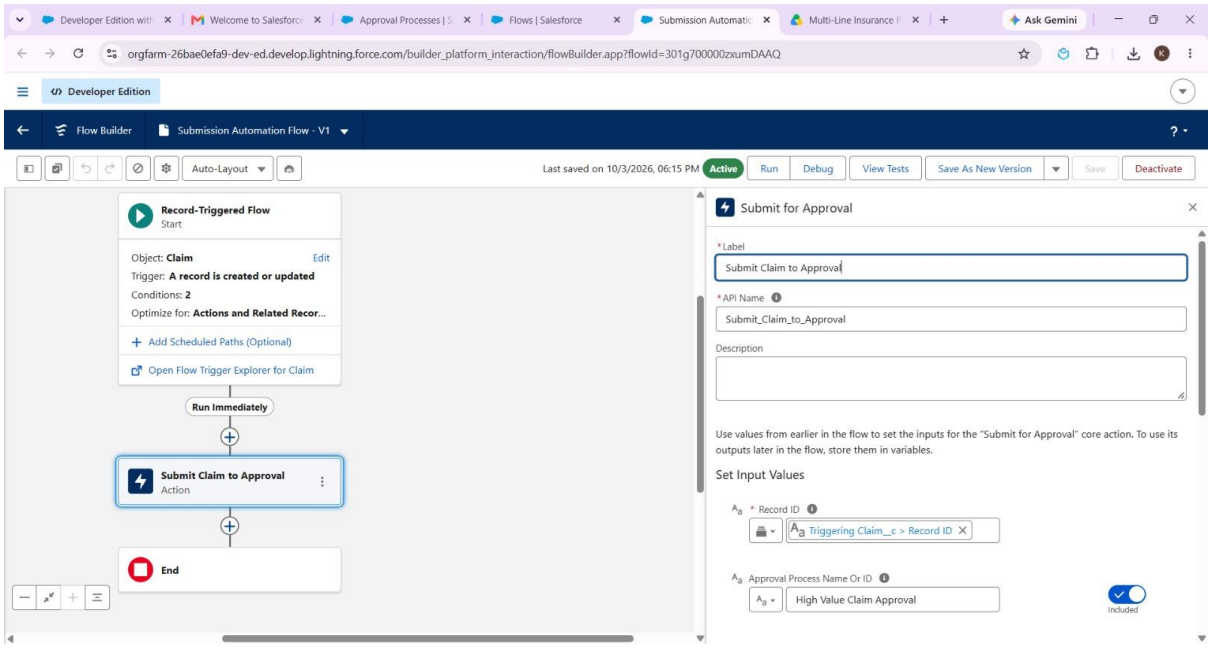
3. Create a Record-Triggered Flow (After Save) for High-Value Claims

- Go to **Setup**, search for **Flows** in the Quick Find box, and open **Flows**.
- Click **New Flow**.
- Select **Record-Triggered Flow**.
- Select **Claim** as the object.
- Configure the trigger to run when a **record is created or updated**.
- Under **Conditional Requirements**, select **All Conditions Are Met**.
- Set the following conditions:
    1. **Claim Amount** → Greater Than → **50,000**
    2. **Approval Status** → Equals → **New**
- Under **When to Run the Flow for Updated Records**, select **Every time a record is updated and meets the conditional requirements**.

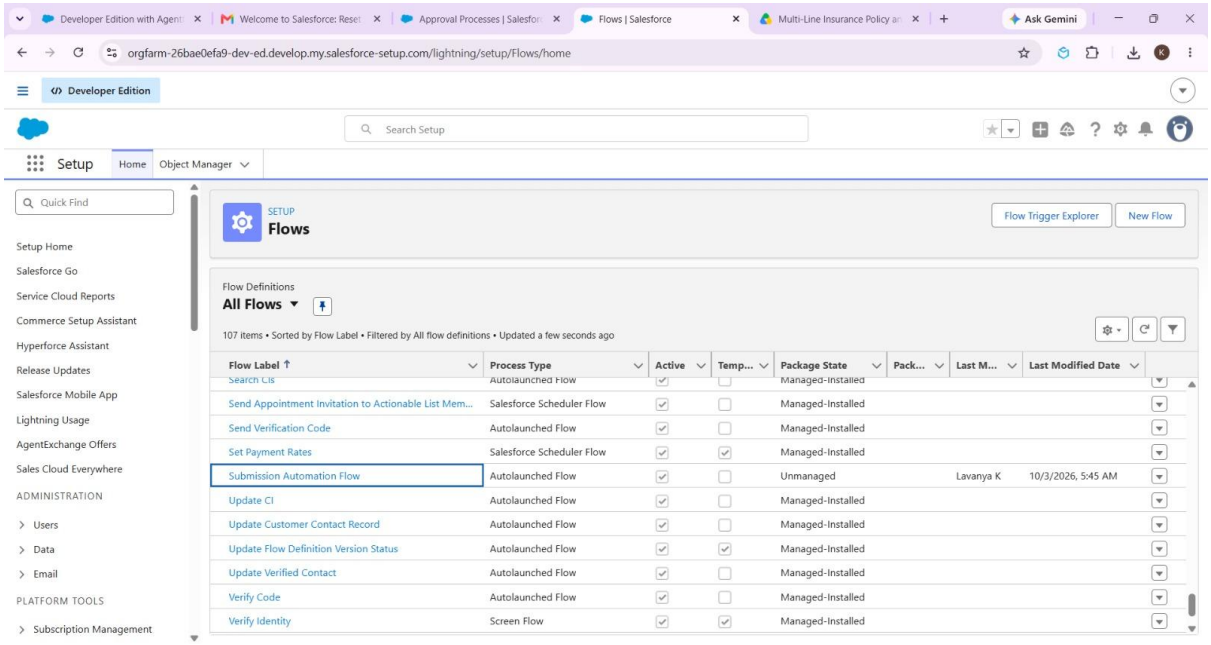
- Optimize the flow for **Actions and Related Records**.



- Add an **Action** element after the **Start** element.
- Set the label as **Submit Claim to Approval**.
- Under **Set Input Values**, for **Record Id**, select **Triggering Claim → Record ID**.
- For **Approval Process Name or Id**, enter **High Value Claim Approval**.



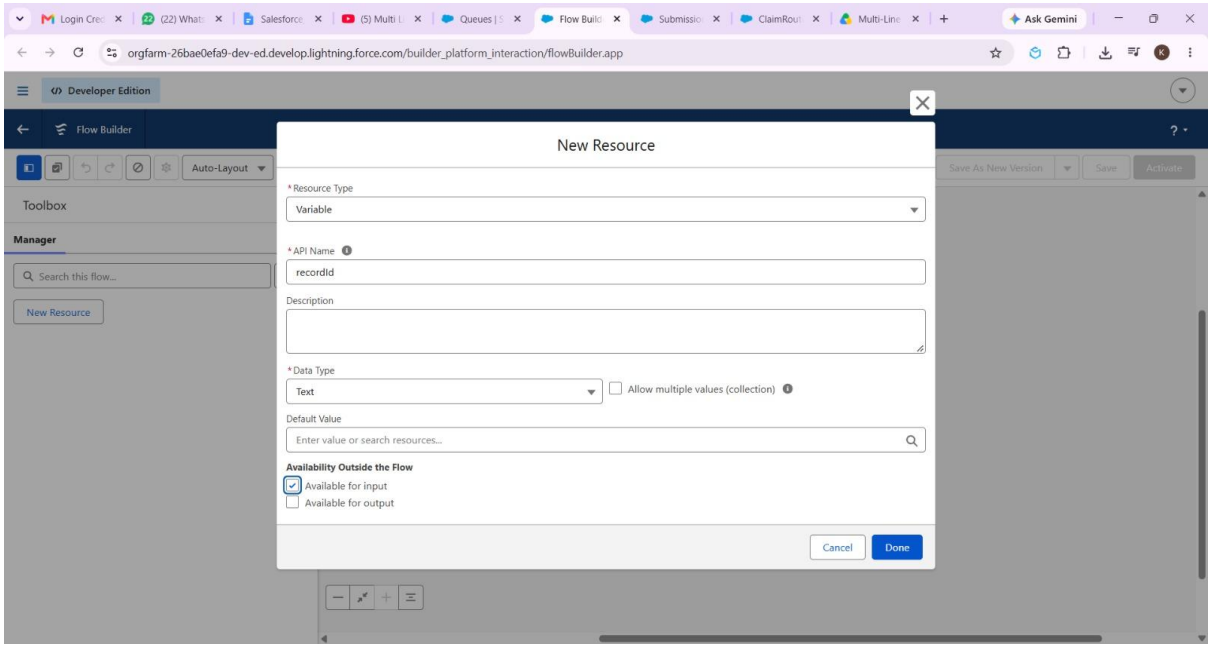
- Click **Save** and name the flow **Submission Automation Flow**.
- Click **Activate** to enable the flow.



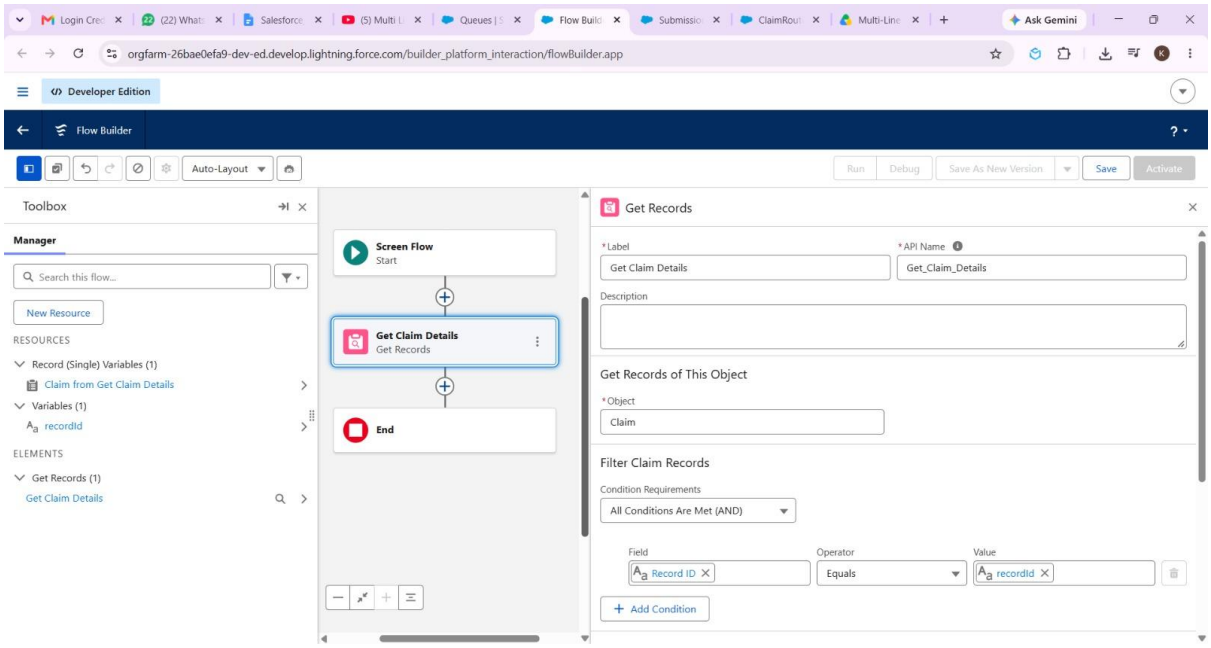
4. Implement a Screen Flow for Approvers to Quickly Review Claim Details and Approve or Reject Directly from the Claim Record

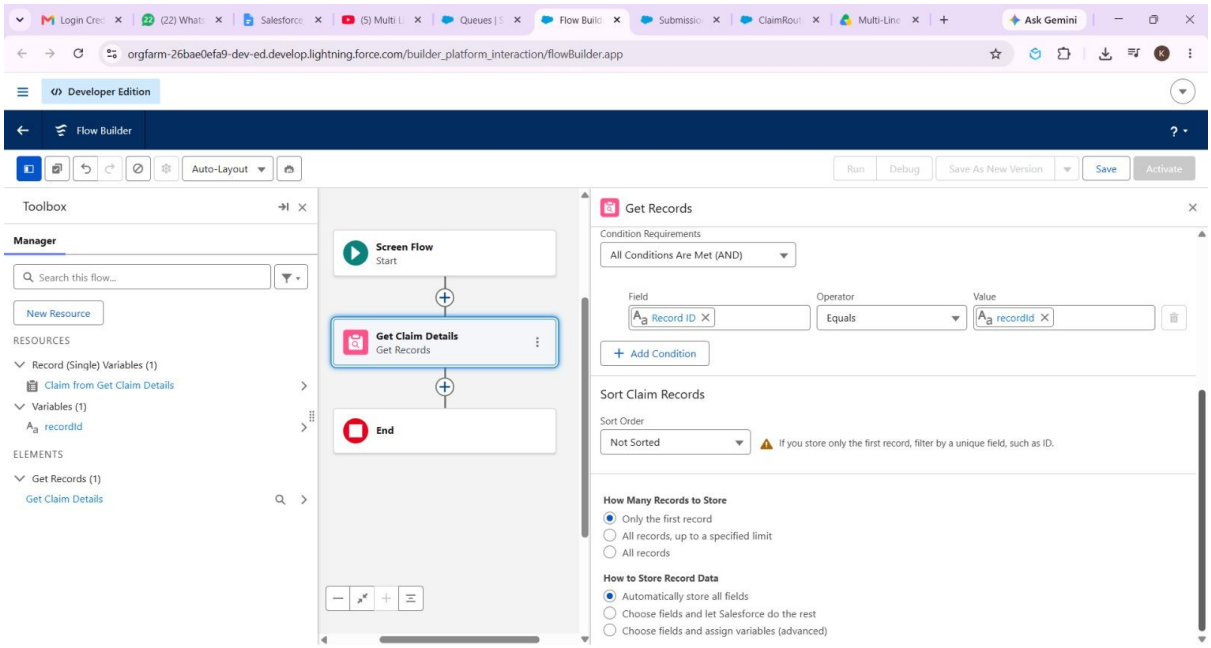
- Go to **Setup**, search for **Flows** in the Quick Find box, and open **Flows**.
- Click **New Flow** and choose **Screen Flow**.
- In the **Toolbox**, click **New Resource** to create a resource.

- Set the **Resource Type** to **Variable**.
- Enter **recordId** as the **API Name**.
- Select **Text** as the **Data Type**.
- Under **Availability Outside the Flow**, check **Available for Input**.
- Click **Done**.



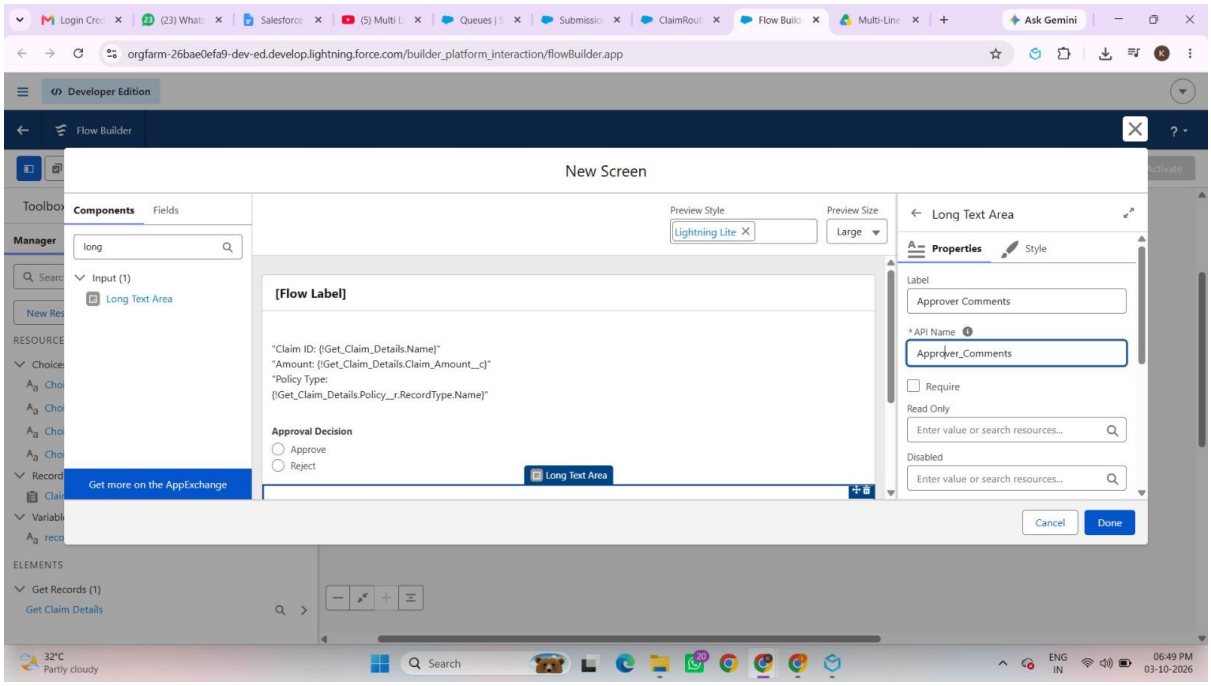
- Add the **Get Records** element below the **Screen** element.
- Set the label as **Get Claim Details**.
- Select **Claim** as the object.
- Under **Conditional Requirements**, select **All Conditions Are Met**.
- Set the condition as **Record ID** → **Equals** → **recordId** variable.
- Select **Store Only the First Record**.
- Select **Automatically Store All Fields**.

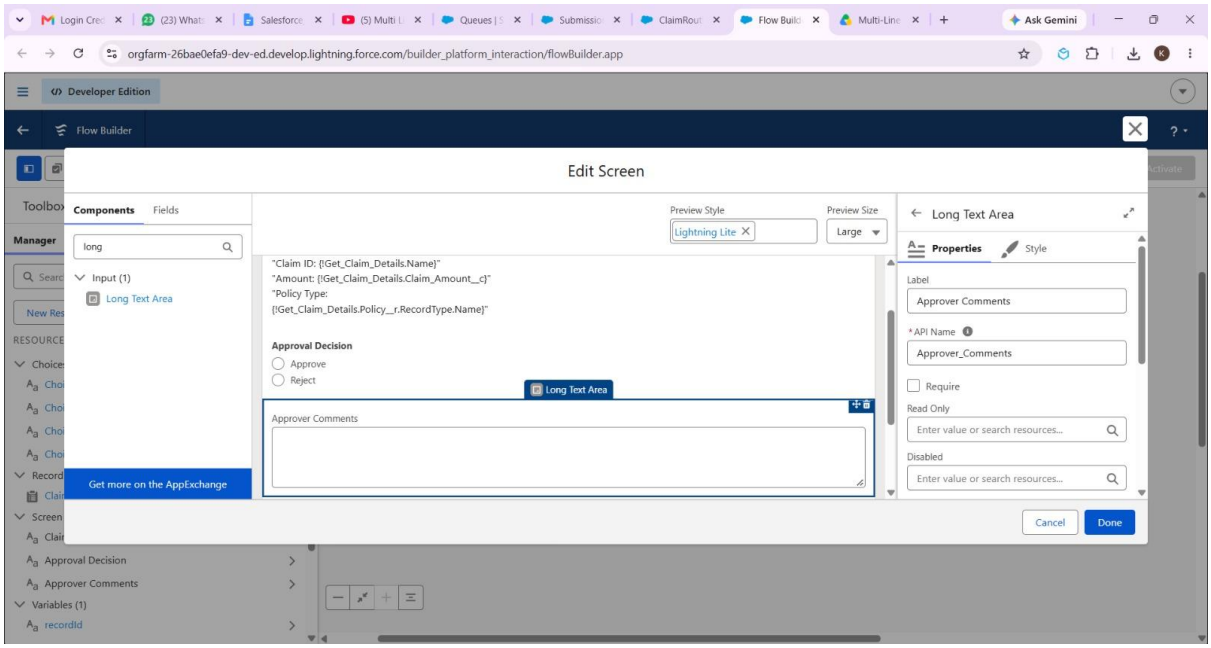




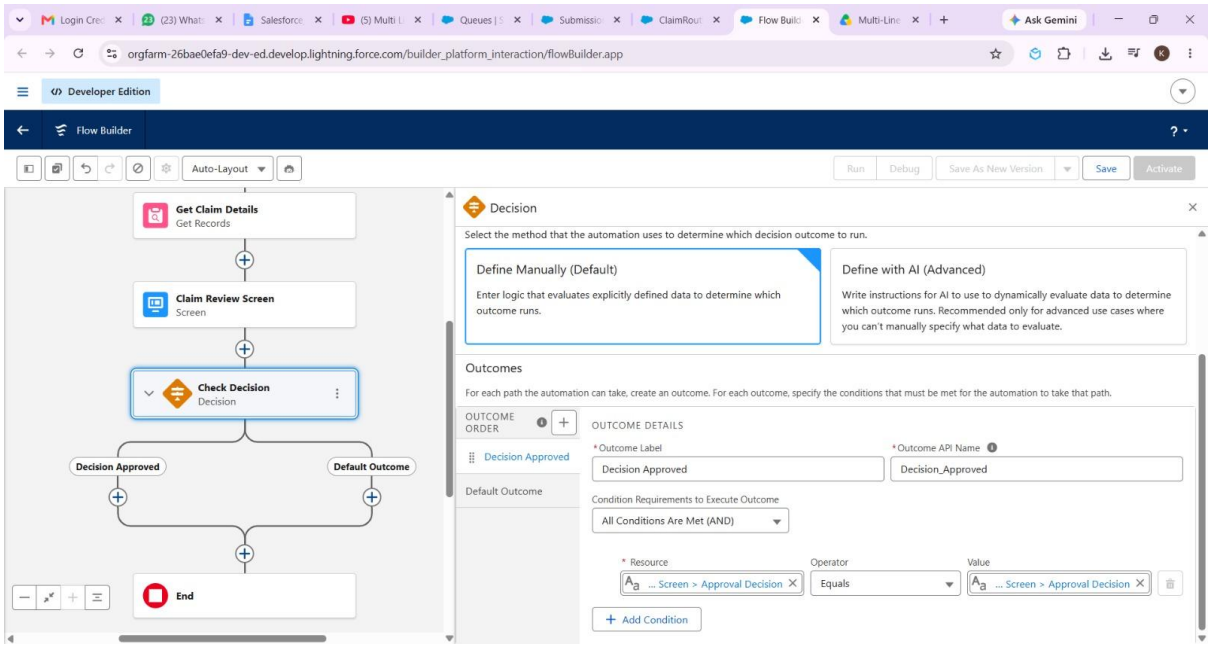
- Add a **Screen** element below the **Get Records** element.
- Set the label as **Claim Review Screen**.
- Add a **Display Text** component to the flow canvas and enter:
    1. **Claim ID:** {!Get_Claim_Details.Name}
    2. **Amount:** {!Get_Claim_Details.Claim_Amount__c}
    3. **Policy Type:** {!Get_Claim_Details.Policy__r.RecordType.Name}
- Add the **Radio Buttons** component and set the label as **Approval Decision**.
- The **API Name** will be populated automatically.

- Under **Choice**, create a new resource with **Resource Type** as **Choice**.
- Set the **Choice Label** as **Approve**.
- Select **Text** as the **Data Type**.
- Enter **Approve** as the **Choice Value**.
- Enter **Approve** as the **Input Label**.
- Click **Done**.
- Add another **Choice** and create a new resource with **Resource Type** as **Choice**.
- Set the **Choice Label** as **Reject**.
- Select **Text** as the **Data Type**.
- Enter **Reject** as the **Choice Value**.
- Enter **Reject** as the **Input Label**.
- Click **Done**.
- Add a **Long Text Area** component to the flow canvas and set the label as **Approver Comments**.
- The **API Name** will be populated automatically.
- Click **Done**.

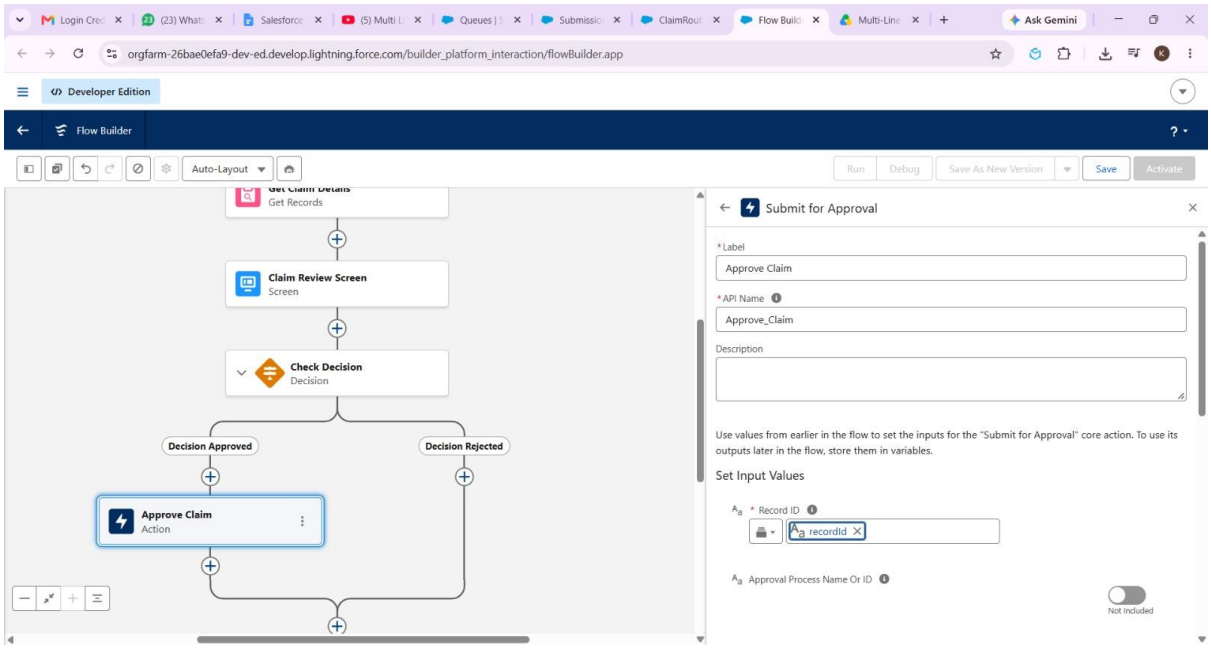


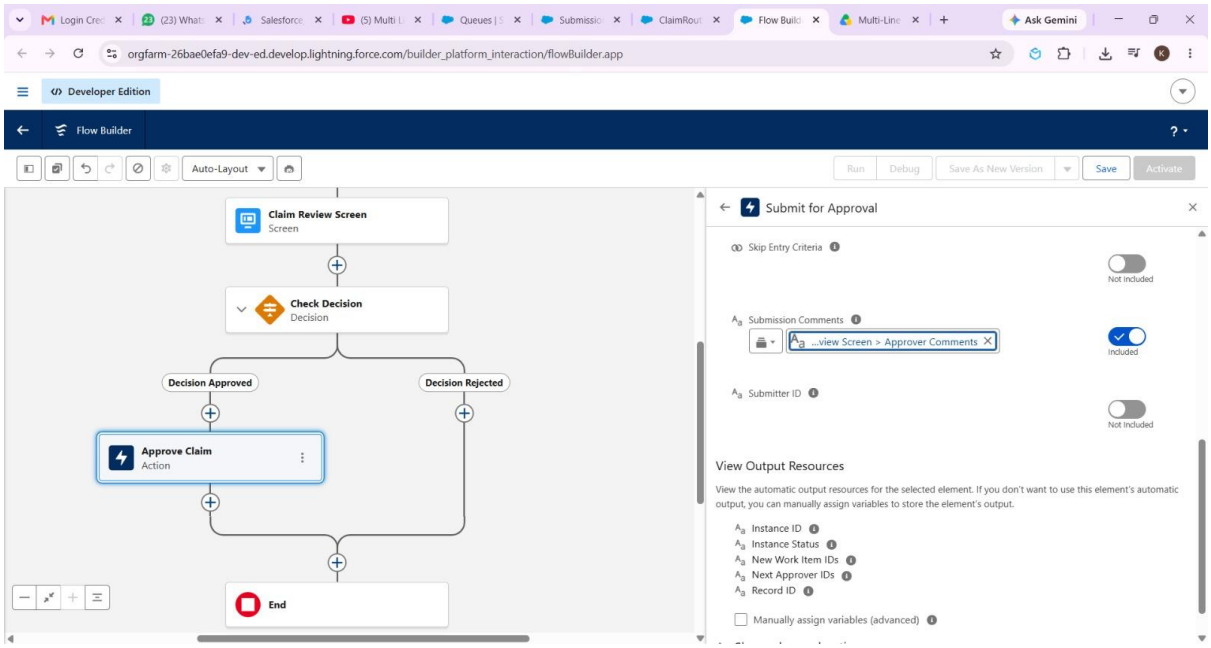


- Add a **Decision** element below the **Screen** element.
- Set the label as **Check Decision**.
- Select **Manually Select Conditions**.
- Set **Outcome 1**:
    1. Label: **Decision Approved**
    2. Under **Conditional Requirements**, select **All Conditions Are Met**.
    3. Set the condition as **Approve Decision From Claim Review Screen → Equals → Approval Decision**.
- Set the **Default Outcome**:
    1. Label: **Decision Rejected**.

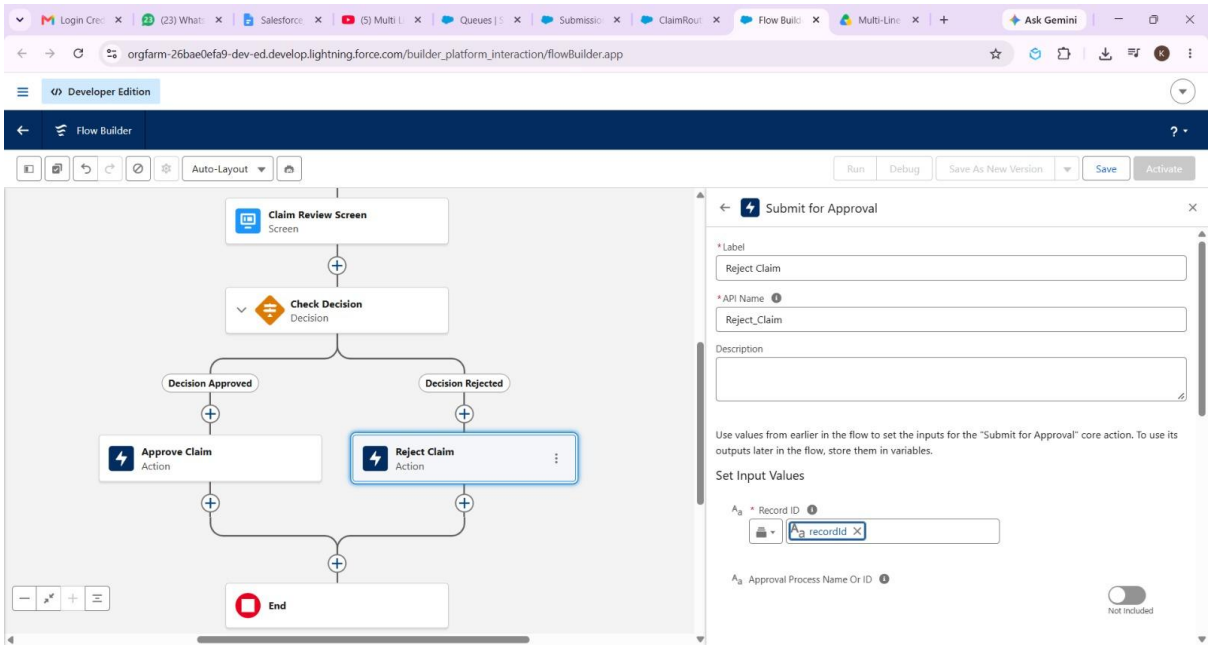


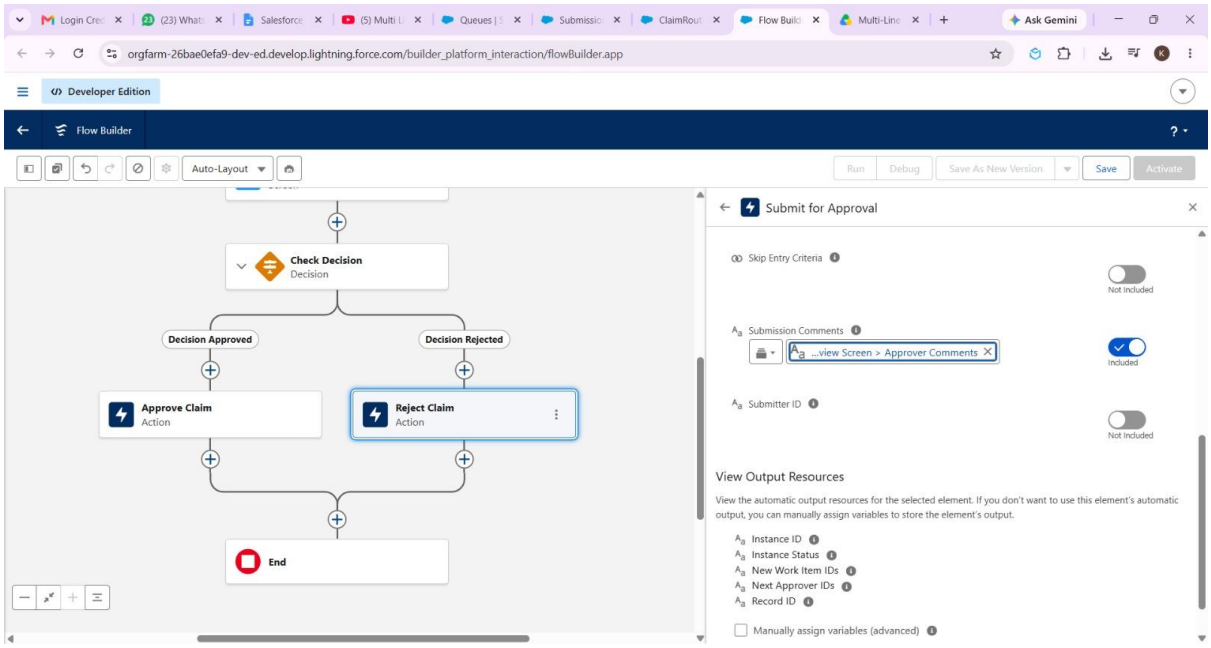
- Add an **Action** element under the **Decision Approved** path.
- Search for the **Submit for Approval** action and select it.
- Set the label as **Approve Claim**.
- Under **Set Input Values**, select the **recordId** variable for **Record ID**.
- For **Submission Comments**, select **Approver Comments** from the **Claim Review Screen**.





- Add an **Action** element under the **Decision Rejected** path.
- Search for the **Submit for Approval** action and select it.
- Set the label as **Reject Claim**.
- Under **Set Input Values**, select the **recordId** variable for **Record ID**.
- For **Submission Comments**, select **Approver Comments** from the **Claim Review Screen**.

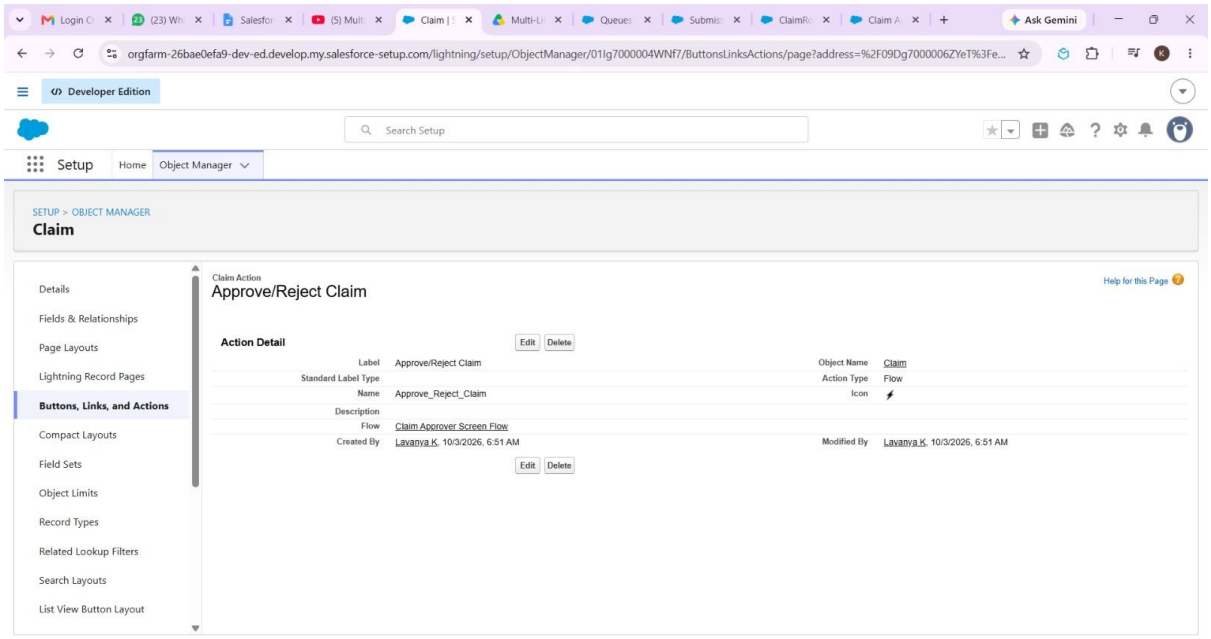




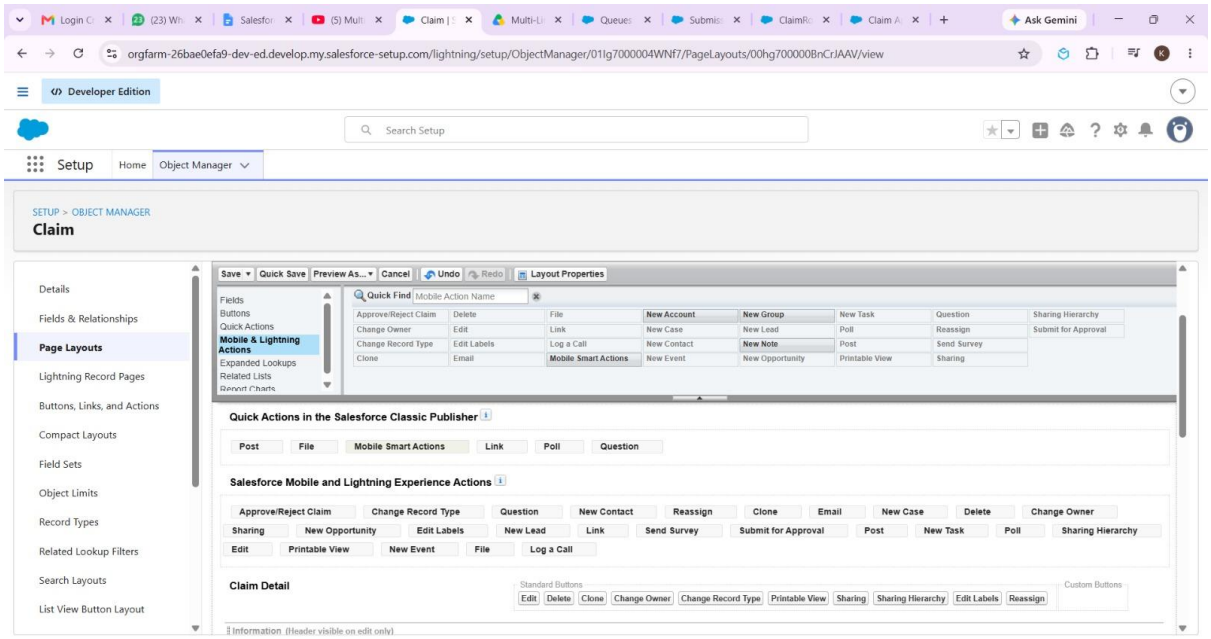
- Save and activate the flow with the name **Claim Approver Screen Flow**.

4.1 Create a Quick Action

- Go to **Setup**, click **Object Manager**, search for **Claim**, and select the **Claim** object.
- In the left-side panel, click **Buttons, Links, and Actions**.
- Click **New Action**.
- Select **Flow** as the **Action Type**.
- Select the **Claim Approver Screen Flow** that you created.
- Enter **Approve/Reject Claim** as the **Label**.
- The **Name** will be populated automatically.
- Click **Save**.



- In the left-side panel, click **Page Layouts** and open the **Claim Page Layout**.
- Go to **Mobile & Lightning Actions**.
- Find **Approve/Reject Claim** and drag it to **Salesforce Mobile and Lightning Experience Actions**.
- Click **Save**.



5. Write an Extensive Apex Test Class for the ClaimsAdjusterController with 95%+ Code Coverage

- Click on the **Setup** icon, right-click **Developer Console**, and click **Open link in new tab**.

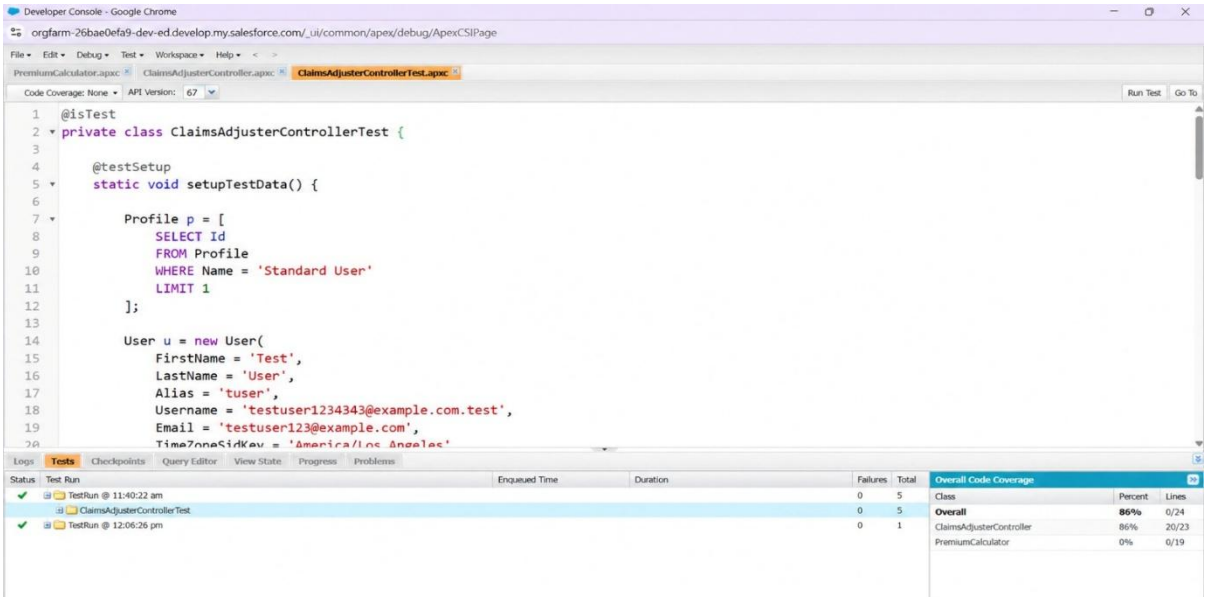
- In the top-left corner, click **File → New → Apex Class** and give it the name **ClaimsAdjusterControllerTest**.
- Paste the following code:

```
@isTest
private class ClaimsAdjusterControllerTest {
@testSetup
static void setupTestData() {
// Create a real active test user
Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
User u = new User(
FirstName = 'Test',
LastName = 'User',
Alias = 'tuser',
Username = 'testuser1234343@example.com.test',
Email = 'testuser123@example.com',
TimeZoneSidKey = 'America/Los_Angeles',
LocaleSidKey = 'en_US',
EmailEncodingKey = 'UTF-8',
LanguageLocaleKey = 'en_US',
ProfileId = p.Id
);
insert u;
// Query Policy__c RecordType
Id policyRT =
Schema.SObjectType.Policy__c.getRecordTypeInfosByDeveloperName()
.get('Auto')
.getRecordTypeId();
Id claimRT =
```

```
Schema.SObjectType.Claim__c.getRecordTypeInfosByDeveloperName()
.get('Accident')
.getRecordTypeId();
// Create Contact owned by same user
Contact con = new Contact(
FirstName = 'John',
LastName = 'Doe',
OwnerId = u.Id
);
insert con;
// Create Policy owned by same user
Policy__c policy = new Policy__c(
Customer__c = con.Id,
RecordTypeId = policyRT,
OwnerId = u.Id,
VIN__c = '1A903284375893532'
);
insert policy;
// Create Claim owned by same user
Claim__c claim = new Claim__c(
Policy__c = policy.Id,
RecordTypeId = claimRT,
Claim_Amount__c = 5000,
Date_of_Loss__c = Date.today().addDays(-10),
OwnerId = u.Id
);
insert claim;
System.debug('Claim inserted: '+claim.Id);
```

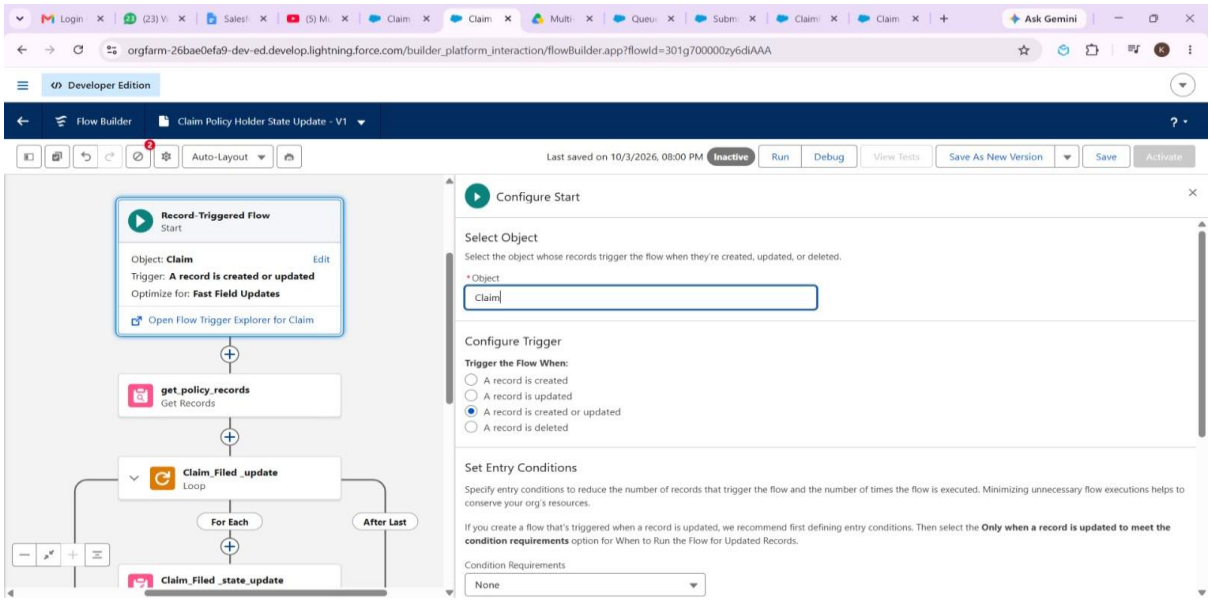
```
}
@isTest
static void testGetAssignedClaims() {
User u = [SELECT Id FROM User WHERE Alias = 'tuser' LIMIT 1];
System.runAs(u) {
Test.startTest();
List<ClaimsAdjusterController.ClaimWrapper> results =
ClaimsAdjusterController.getAssignedClaims();
System.debug('Result is: '+results);
Test.stopTest();
//System.assert(results.size() > 0,
//'Expected at least one wrapper returned');
ClaimsAdjusterController.ClaimWrapper wrap = results[0];
System.assertNotEquals(null, wrap.claimId);
System.assertNotEquals(null, wrap.claimNumber);
System.assertEquals('New', wrap.status);
System.assertEquals(5000, wrap.claimAmount);
System.assert(wrap.policyHolderName.contains('John'));
System.assertNotEquals('Unknown', wrap.policyType);
}
}
}
```

- Save the file using **Ctrl + S**.
- In the top-right corner, click **Run Test** to execute the test class.
- To check the test results, go to **Tests** and view the percentage of code covered by the ClaimsAdjusterController Apex class.

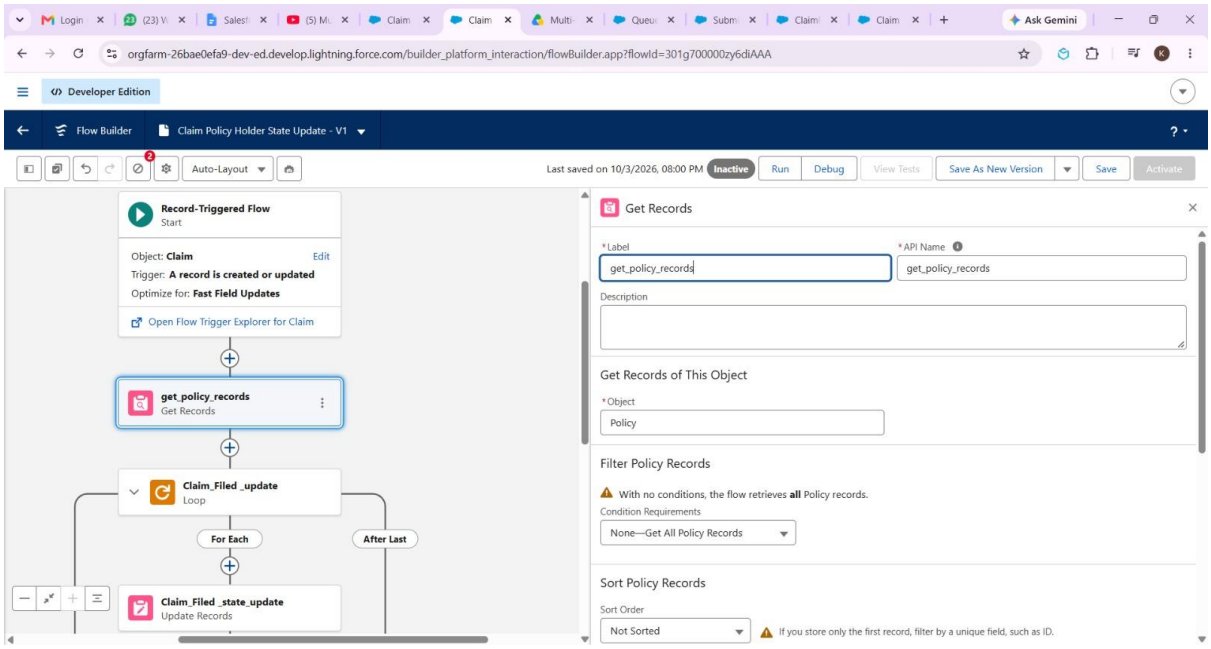


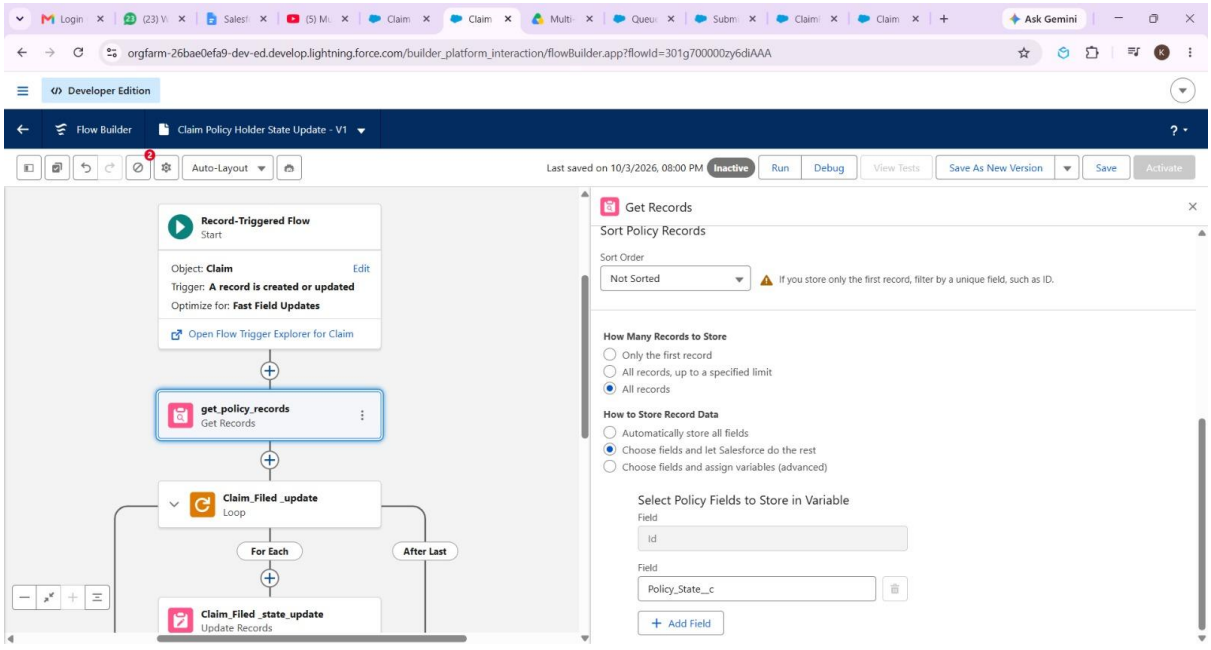
6. Refine the Sharing Rules to Ensure Adjusters Only See Claims in Their Assigned Territory/State

- Go to **Setup**, search for **Flows** in the Quick Find box, and click **Flows**.
- Click **New Flow**.
- Select **Record-Triggered Flow**.
- Select **Claim** as the object.
- Configure the trigger to run when a **record is created or updated**.
- Set the **Entry Conditions** to **None**.
- Optimize the flow for **Updates**.

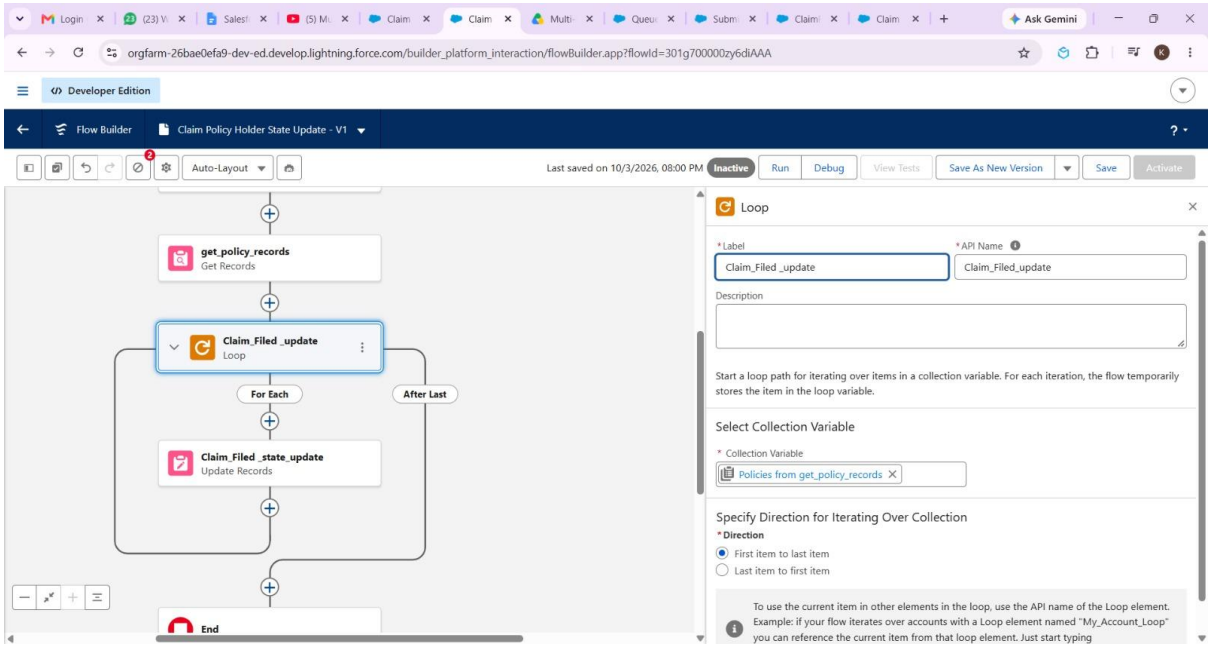


- Add a **Get Records** element to the flow canvas.
- Set the label as **get_policy_records**.
- The **API Name** will be populated automatically.
- Set the **Filter** to **None**.
- Select **Store All Records**.
- Under **How to Store Record Data**, select **Choose fields and let Salesforce do the rest**.
- Under **Select Policy fields to store in Variable**, select the **Policy State** field.

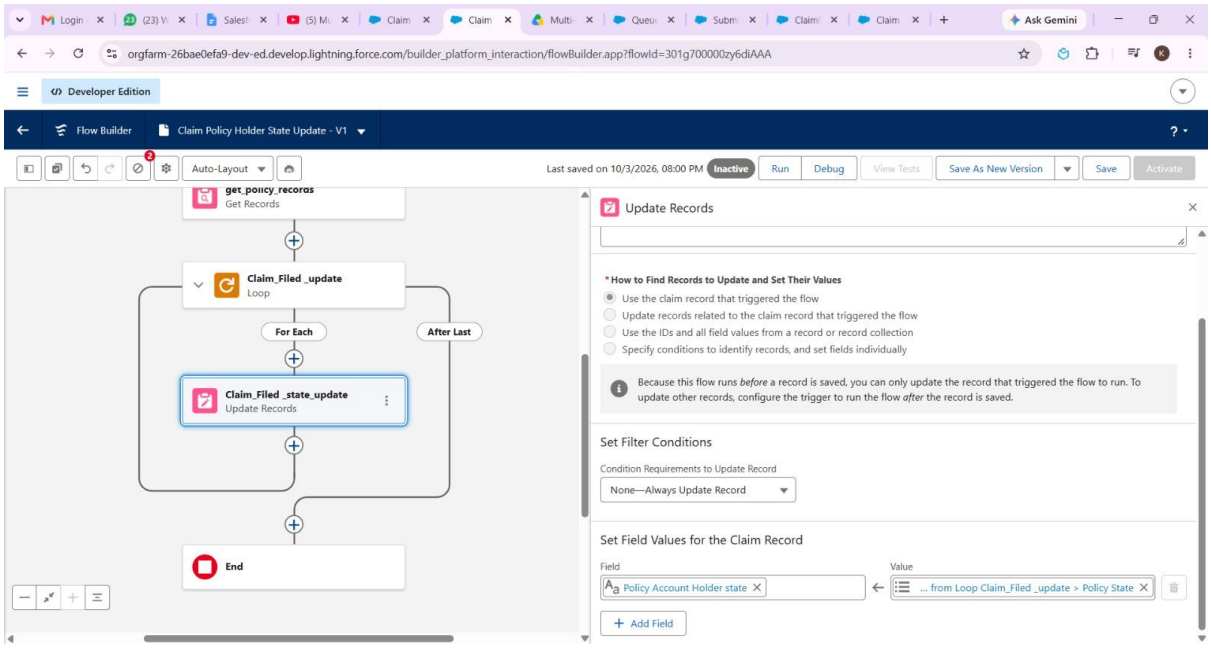




- Add the **Loop** element to the canvas after the **Get Records** element.
- Set the label as **Claim_Filed_Update**.
- The **API Name** will be populated automatically.
- Under **Select Collection Variable**, select **Policies** from **Get Policy Records**.
- Under **Specify Direction for Iterating Over Collection**, select **First item to last item**.



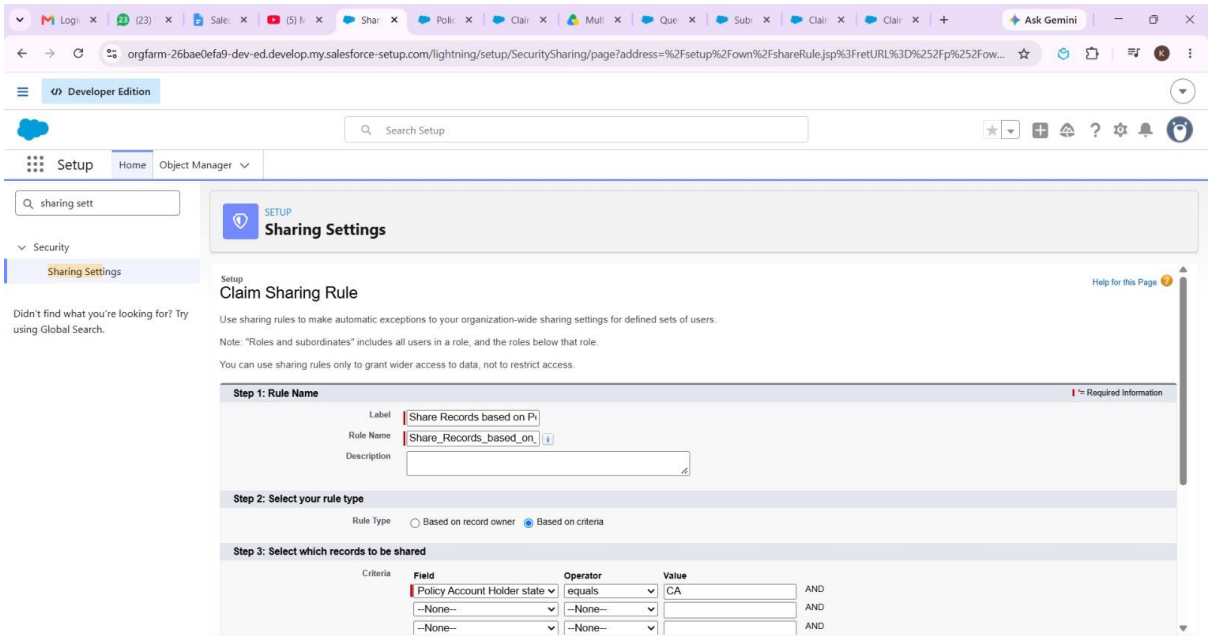
- Under **For Each**, add the **Update Records** element to the canvas.
- Set the label as **Claim_Filed_State_Update**.
- The **API Name** will be populated automatically.
- Set the **Filter Conditions** to **None**.
- Under **Set Field Values for the Claim Record**, select **Policy State** from the **Current Loop Item** and set it to the **Policy Account Holder State** field.

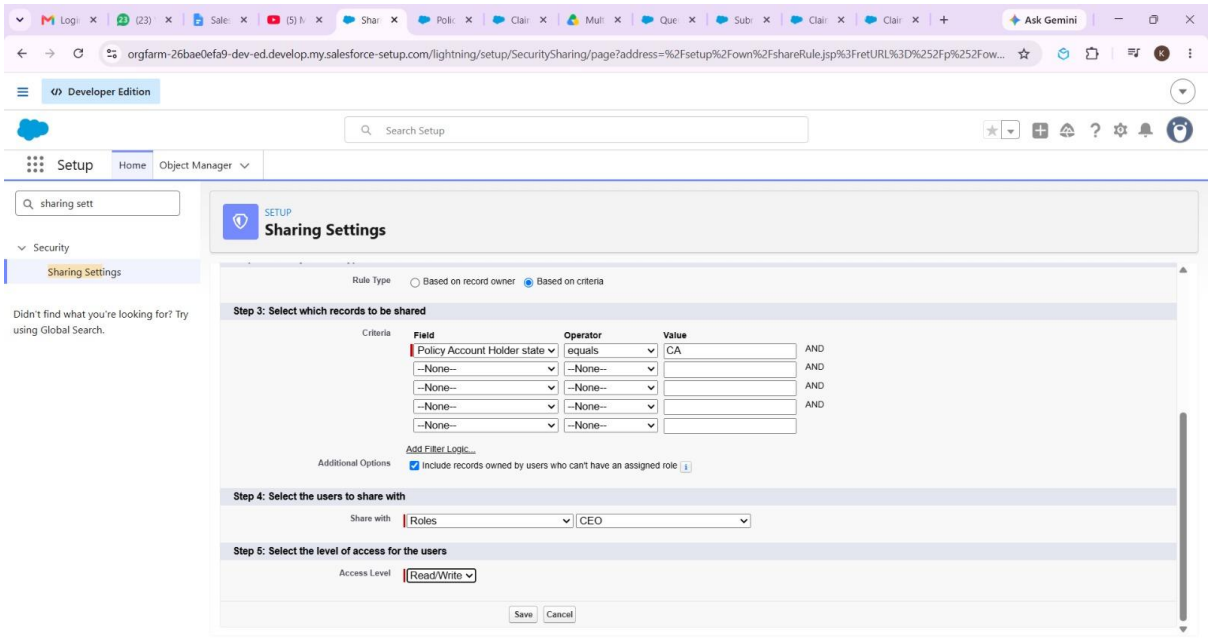


- Save and activate the flow with the name **Claim Policy Holder State Update**.

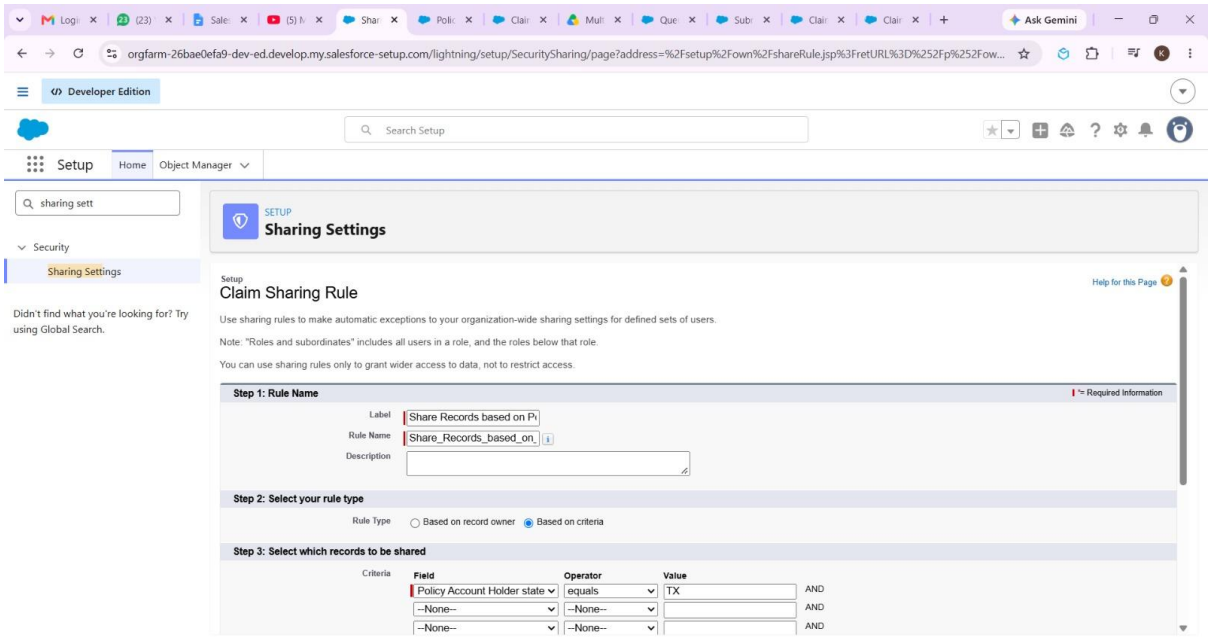
6.1 Define Sharing Rules for the Claim Object

- Go to **Setup** and search for **Sharing Settings** in the Quick Find box.
- Click **Sharing Settings**.
- Under **Manage Sharing Settings for**, select the **Claim** object.
- Scroll down to **Claim Sharing Rules**.
- Click **New**.
- Enter **Share Records Based on Policy State CA** as the label.
- The **Rule Name** will be populated automatically.
- Set the criteria to **Policy Account Holder State Equals CA**.
- Set the **Access Level** to **Read/Write**.
- Click **Save**.



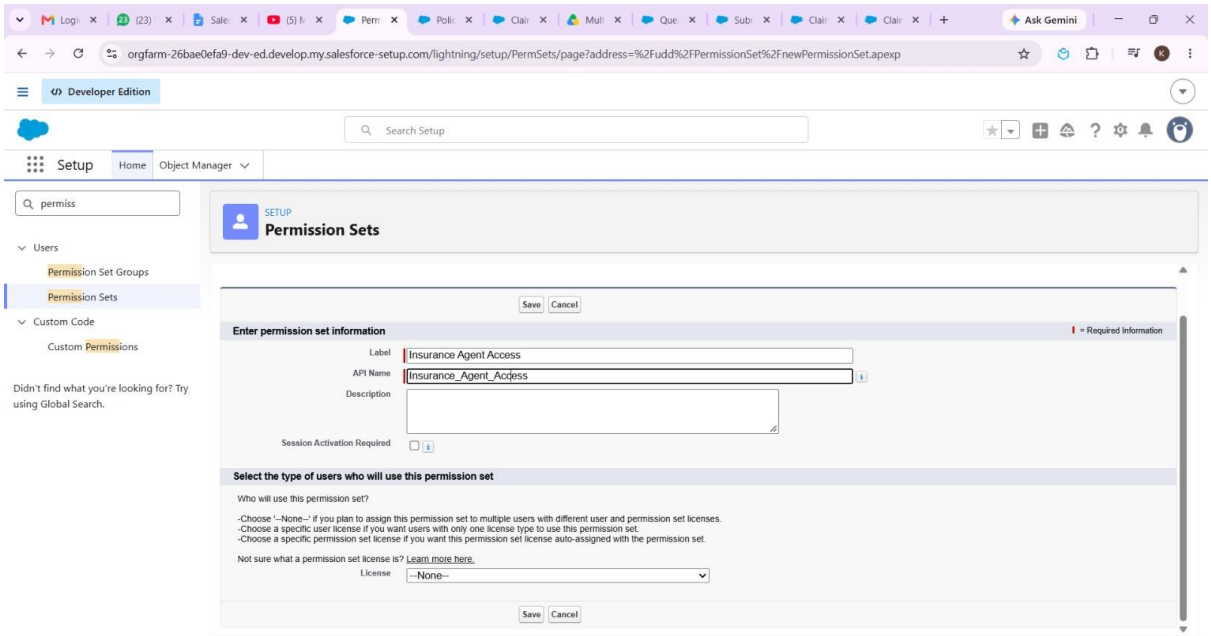


- Click **New** again.
- Enter **Share Records Based on Policy State TX** as the label.
- The **Rule Name** will be populated automatically.
- Set the criteria to **Policy Account Holder State Equals TX**.
- Set the **Access Level** to **Read/Write**.
- Click **Save**.



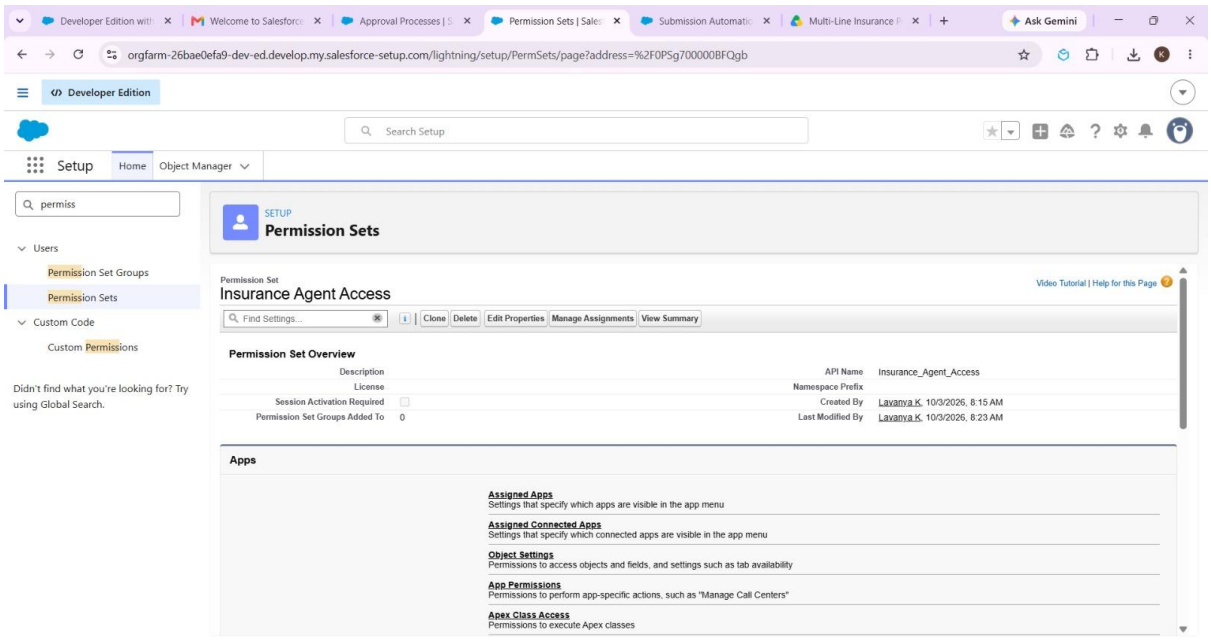
7. Finalize Permission Sets for Agents, Adjusters, and Managers

- Go to **Setup**, search for **Permission Sets** in the Quick Find box, and click **Permission Sets**.
- Click **New**.
- Enter **Insurance Agent Access** as the label.
- The **API Name** will be populated automatically.



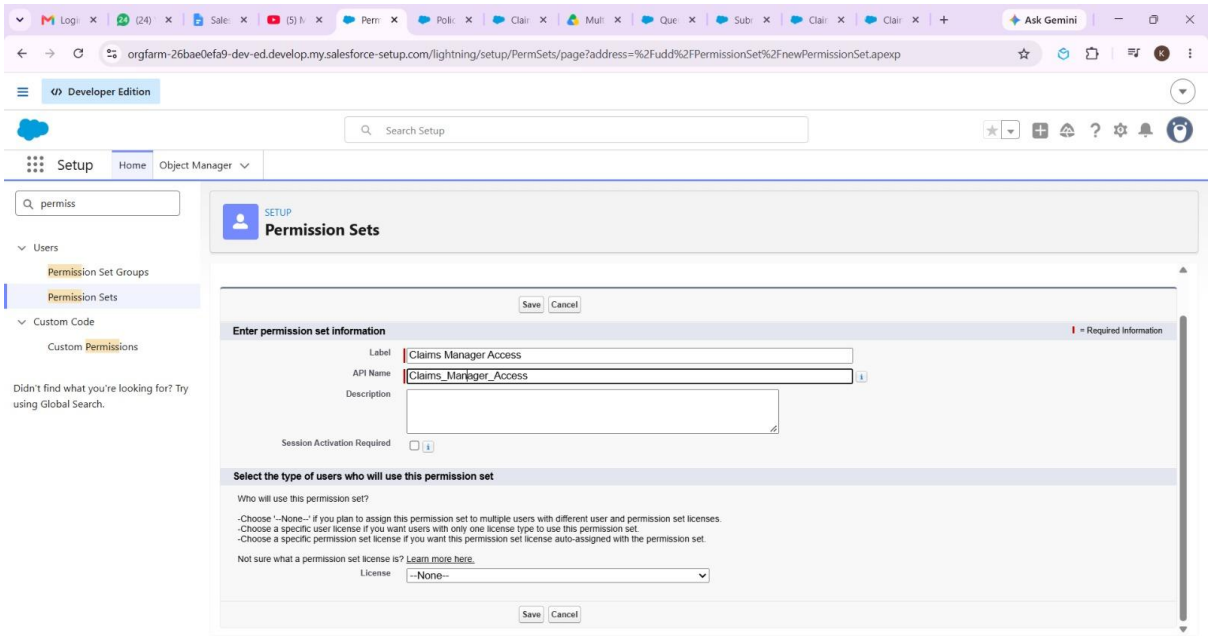
- Set the required **Object Permissions**.
- Give **Create** and **Read** access to the **Policy** object.

- Give **Read** and **Edit** access to the **Contact** object.
- Give **Read** access to the **Claim** object.
- Enable **Flow User** under **System Permissions**.
- Enable access to the **PremiumCalculator** Apex class.

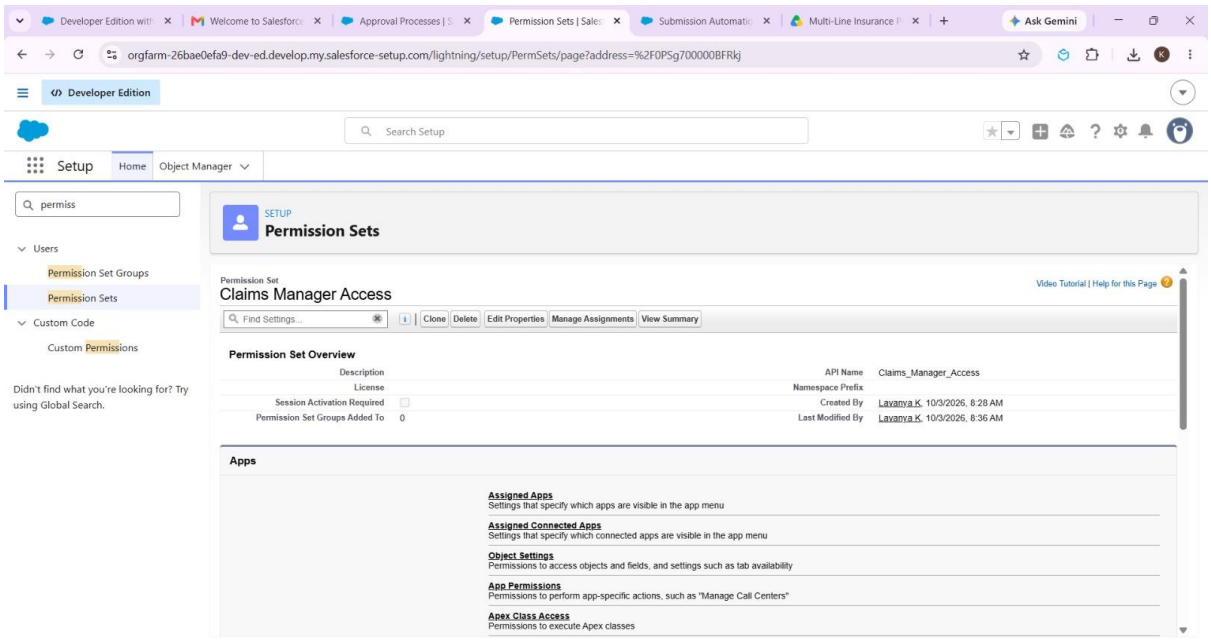


7.1 Create Claims Manager Access Permission Set

- Click **New**.
- Enter **Claims Manager Access** as the label.
- The **API Name** will be populated automatically.

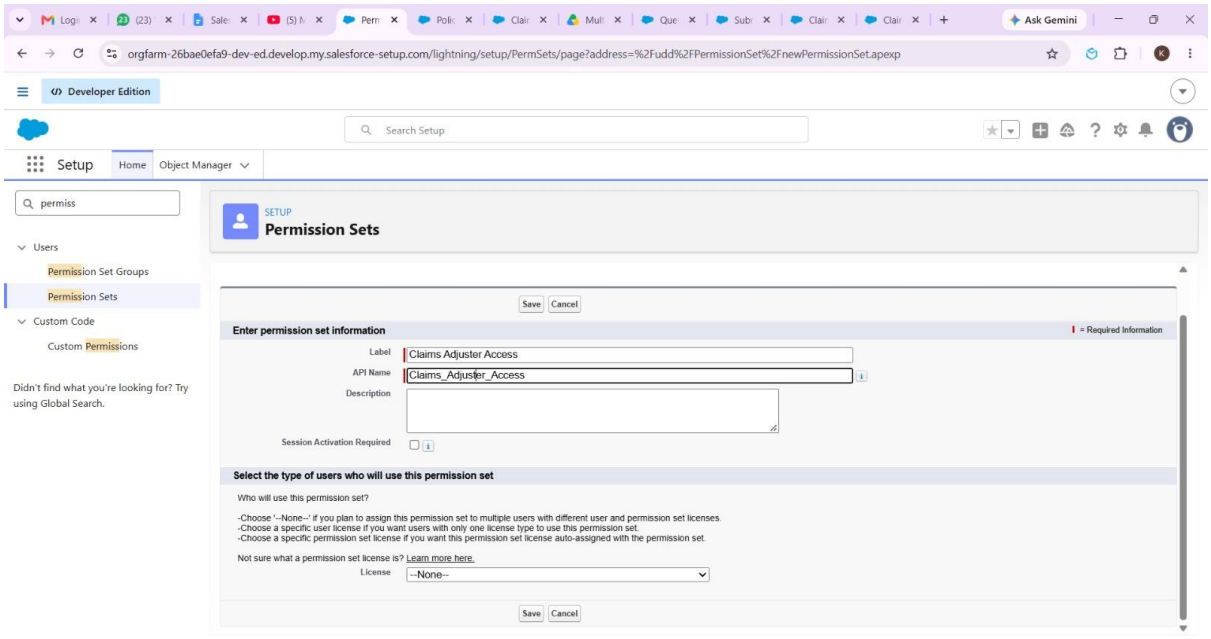


- Set the required **Object Permissions**.
- Give **Read** access to the **Policy** object.
- Give **Read** and **Edit** access to the **Contact** object.
- Give **Read, Edit, and Delete** access to the **Claim** object.
- Enable **Run Reports** under **System Permissions**.
- Enable **Manage Approvals** under **System Permissions**.

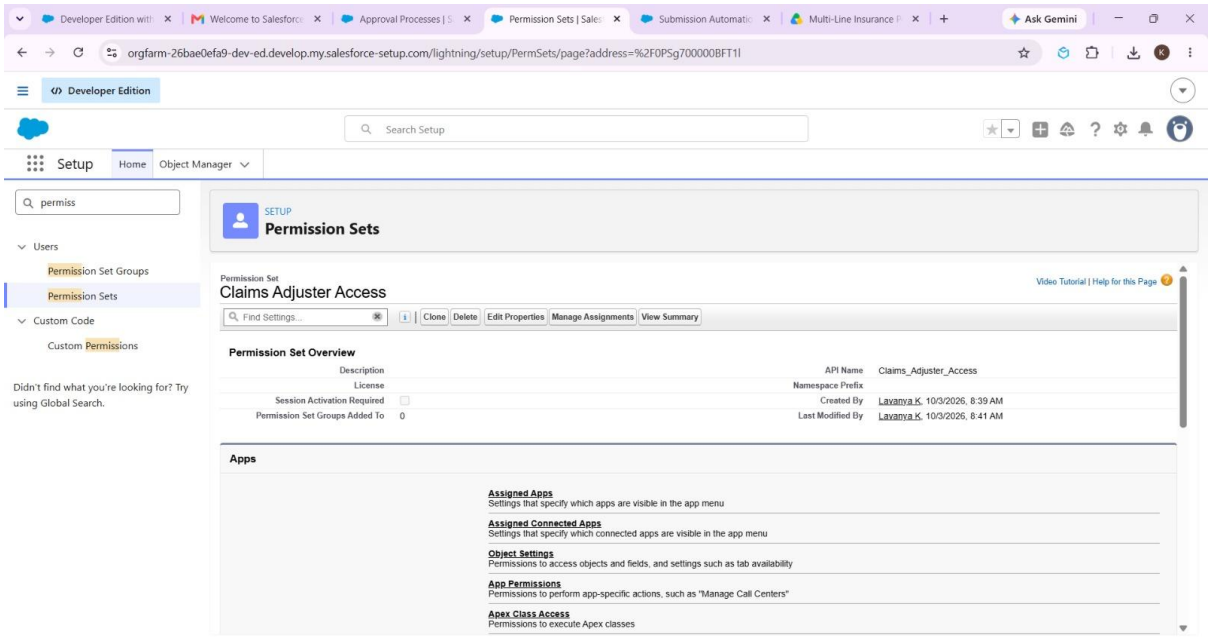


7.2 Create Claims Adjuster Access Permission Set

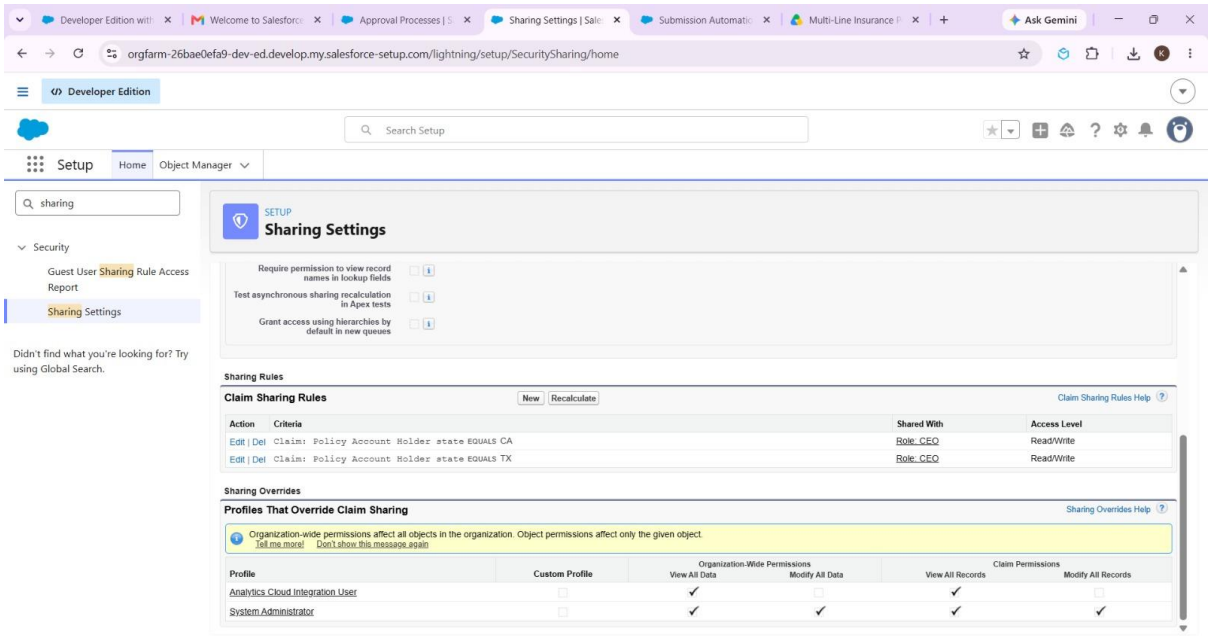
- Click **New**.
- Enter **Claims Adjuster Access** as the label.
- The **API Name** will be populated automatically.



- Set the required **Object Permissions**.
- Give **Read** access to the **Policy** object.
- Give **Read** and **Edit** access to the **Claim** object.



- Enable **ClaimsAdjusterController** under **Apex Classes**.



7. FUNCTIONAL & PERFORMANCE TESTING

The developed application was evaluated through different test scenarios to confirm that each configured Salesforce component behaved as expected. Testing covered data entry, policy and claim transactions, automated processing, approval activities, user access, and interface components. The results were examined for successful execution, correct record updates, proper navigation, and appropriate responses to different input conditions. Performance was also observed during repeated operations to ensure that the implemented solution remained stable and usable throughout the insurance workflow.

8. ADVANTAGES & DISADVANTAGES

Advantages

- **Reduced Administrative Work** – Repetitive insurance activities can be handled through configured Salesforce automation instead of relying entirely on manual actions.
- **Consistent Business Processing** – Defined workflows provide a common procedure for handling similar insurance transactions.
- **Centralized Activity Tracking** – Users can follow important policy and claim activities through their corresponding Salesforce records.
- **Faster Information Retrieval** – Related insurance information can be accessed directly from connected records rather than searching through separate sources.

- **Improved Process Control** – Approval stages and automated actions provide greater control over important insurance transactions.
- **Easier Issue Identification** – Validation and testing mechanisms help identify incorrect entries before they affect subsequent processing.
- **Better Team Coordination** – Agents, adjusters, and managers can work with information according to their respective responsibilities.
- **Customizable Configuration** – Salesforce components can be modified as organizational requirements change.
- **Reduced Record Duplication** – Maintaining related information within connected Salesforce records can minimize unnecessary duplicate entries.
- **Operational Transparency** – Managers can obtain a clearer understanding of ongoing insurance activities through Salesforce monitoring facilities.

Disadvantages

- **Configuration Dependency** – Changes to Salesforce configuration may require careful evaluation because multiple components can depend on the same records or fields.
- **Large-Scale Data Handling** – Processing a growing number of policy and claim records may require additional optimization and monitoring.
- **Customization Limitations** – Some highly specialized insurance requirements may need additional Salesforce development beyond standard configuration.
- **Testing Overhead** – Changes to flows, Apex, approval processes, or Lightning components must be tested again to prevent unintended effects.
- **Administrator Skill Requirement** – Maintaining the application effectively requires personnel who understand Salesforce administration and development.
- **Change Management** – Introducing new insurance rules or modifying existing processes may require updates across several Salesforce components.

9. CONCLUSION

The project successfully establishes a Salesforce environment for organizing the major activities involved in a multi-line insurance operation. By combining configurable Salesforce features with custom development, the application provides a structured environment in which insurance information can be created, processed, reviewed, and monitored.

The implementation demonstrates how a cloud-based CRM platform can be adapted to support insurance-specific business requirements. The developed components work together to provide controlled processing, automated actions, user-specific functionality, and improved accessibility of operational information.

The completed system therefore serves as a practical foundation for insurance organizations seeking to move from conventional handling methods toward a more organized digital process. Its configurable architecture also provides a suitable base for further improvements as business requirements evolve.

10. FUTURE SCOPE

- **Digital Document Management** – Future versions can include centralized storage and classification of policy documents, identification proofs, inspection reports, and claim-related files.
- **Notification Services** – Automated email, SMS, or in-app notifications can inform users about important policy and claim events.
- **Renewal Management** – The application can be extended to identify upcoming policy expirations and initiate renewal activities automatically.
- **Fraud Detection Support** – Additional analytical mechanisms could help identify unusual claim patterns and transactions requiring investigation.
- **Payment Processing** – Integration with suitable payment services could support premium collection and claim settlement activities.
- **Document Verification** – Automated document checking can be introduced to reduce manual verification during policy and claim processing.
- **Geographical Analysis** – Location-based reporting could help organizations study policy distribution, claim concentration, and regional insurance trends.
- **Customer Communication History** – Future functionality can maintain a structured history of important communications between customers and insurance personnel.

11. APPENDIX

- **Core Data:** Policy, Claim, Contact, User records
- **Insurance Configuration:** Vehicle, Property, Life policy classifications
- **Claim Configuration:** Accident, Property, Life claim classifications
- **Process Automation:** Policy quotation, claim processing, approval workflow
- **Custom Development:** Apex classes and Lightning Web Components
- **User Interface:** Salesforce Lightning pages and custom claim dashboard
- **Business Validation:** VIN and insurance record validation
- **Approval Management:** High-value claim approval workflow
- **Access Management:** Permission Sets and Sharing Rules
- **Quality Verification:** Apex testing and functional testing

THANK YOU